

Diskusi.5

Lakukan: Kirim balasan: 1**Jatuh tempo:** Minggu, 9 November 2025, 23:59

menampilkan balasan dalam bentuk bertingkat

Setelan v

Sudah mencapai batas waktu untuk mengirim ke forum ini sehingga Anda tidak dapat lagi mengirim ke forum ini.

Diskusi.5

Rabu, 28 Mei 2025, 10:31

Sebuah perusahaan teknologi sedang mengembangkan dua jenis sistem yang berbeda:

1. Sistem E-Commerce "QuickBuy" – Sebuah platform belanja online yang memungkinkan pengguna untuk memilih produk, menambahkan ke keranjang belanja, melakukan pembayaran, serta melacak pengiriman pesanan.
2. Sistem Kesehatan "MediCare" – Sebuah sistem injeksi insulin otomatis untuk penderita diabetes yang menggunakan sensor untuk memantau kadar gula darah dan secara otomatis memberikan dosis insulin yang sesuai.

Sebagai System Analyst, Anda ditugaskan untuk menganalisis dan memilih diagram UML yang paling relevan untuk mendesain kedua sistem ini. Namun, Anda menyadari bahwa jumlah dan jenis diagram yang diperlukan bisa sangat berbeda tergantung pada kompleksitas sistem yang dikembangkan.

Untuk Sistem E-Commerce "QuickBuy", kemungkinan hanya diperlukan beberapa diagram utama, karena sistem ini berfokus pada interaksi pengguna, transaksi, dan pengelolaan data produk.

Sedangkan untuk Sistem Kesehatan "MediCare", sistem ini melibatkan perangkat keras, sensor, algoritma pemantauan, dan keamanan tinggi, sehingga membutuhkan lebih banyak diagram seperti State Machine Diagram untuk menggambarkan alur kerja sistem yang kompleks dan interaksi antara perangkat serta pengguna.

Namun, tim proyek masih memiliki beberapa pertanyaan:

- Diagram UML apa saja yang paling penting untuk setiap sistem?
- Mengapa jumlah diagram yang dibutuhkan berbeda antara sistem e-commerce dan sistem injeksi insulin otomatis?
- Bagaimana cara menentukan jumlah diagram yang diperlukan dalam proyek berbasis rekayasa perangkat lunak berorientasi objek?

Pertanyaan Diskusi:

1. Diagram UML apa saja yang paling penting untuk setiap sistem? Jelaskan alasan pemilihan diagram untuk sistem e-commerce dan sistem kesehatan berbasis injeksi insulin otomatis.
2. Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda? Bagaimana kompleksitas sistem memengaruhi jumlah diagram yang dibutuhkan?
3. Bagaimana cara menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek pengembangan perangkat lunak?

Selamat berdiskusi!

Catatan: Saya sarankan, teman-teman mahasiswa menjawab langsung pada tempat yang disediakan, TIDAK mengupload jawaban berupa file, termasuk TIDAK mengupload jawaban dengan GAMBAR ya. Terima kasih!

Tautan permanen



Re: Diskusi.5

oleh [054416248 PANDU WIJAYA](#) - Senin, 3 November 2025, 16:53

1. Diagram UML yang Paling Penting untuk Setiap Sistem

A. Sistem E-Commerce "QuickBuy"

Fokus utama sistem ini adalah interaksi pengguna, transaksi, dan pengelolaan data produk.

Diagram UML yang paling relevan:

1. Use Case Diagram

→ Menunjukkan interaksi antara pengguna (pembeli, admin, sistem pembayaran) dengan sistem.

Alasan: Memudahkan analisis kebutuhan dan identifikasi fitur utama seperti login, pencarian produk, checkout, dan pelacakan pesanan.

2. Class Diagram

→ Menggambarkan struktur data dan hubungan antar entitas seperti User, Product, Order, Payment.

Alasan: Penting untuk merancang basis data dan logika pemrosesan transaksi.

3. Sequence Diagram

→ Menunjukkan alur komunikasi antar objek (misalnya saat pengguna melakukan pembelian).

Alasan: Berguna untuk memahami urutan proses dalam transaksi.

4. Activity Diagram (opsional tapi berguna)

→ Menggambarkan alur proses bisnis seperti proses checkout dan pembayaran.

Alasan: Membantu mengidentifikasi langkah-langkah operasional dan keputusan.

B. Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Sistem ini mencakup perangkat keras (sensor dan pompa insulin), algoritma kontrol, serta aspek keamanan dan keselamatan pasien.

Diagram UML yang diperlukan lebih kompleks:

1. Use Case Diagram

→ Menjelaskan interaksi antara pengguna (pasien, dokter) dan sistem perangkat keras.

Alasan: Menggambarkan kebutuhan fungsional dari perspektif pengguna.

2. Class Diagram

→ Menunjukkan struktur objek seperti Sensor, Controller, InsulinPump, UserProfile, AlertSystem.

Alasan: Membangun dasar arsitektur perangkat lunak dan komunikasi antar komponen.

3. Sequence Diagram

→ Menggambarkan interaksi waktu nyata antara sensor, pengendali, dan aktuator (pompa insulin).

Alasan: Penting untuk sistem real-time yang harus merespons perubahan kadar gula darah dengan cepat.

4. State Machine Diagram

→ Menjelaskan perubahan status sistem seperti Monitoring, Calculating Dose, Injecting, Error State.

Alasan: Krusial untuk sistem otomatis yang memiliki banyak kondisi dan transisi.

5. Activity Diagram

→ Menggambarkan alur kerja pemantauan dan pemberian insulin otomatis.

Alasan: Membantu memahami logika pengambilan keputusan sistem.

6. Component Diagram / Deployment Diagram

→ Menunjukkan bagaimana perangkat keras dan perangkat lunak saling terhubung (sensor, mikrokontroler, aplikasi).

Alasan: Diperlukan untuk sistem yang menggabungkan hardware dan software.

2. Mengapa Jumlah dan Jenis Diagram UML Berbeda?

Jumlah dan jenis diagram bergantung pada kompleksitas sistem dan tujuan pengembangannya.

-Faktor kompleksitas sistem E-Commerce "QuickBuy" sedang (berbasis web) sedangkan Sistem Kesehatan "MediCare" sangat tinggi (real-time, safety-critical)

-Faktor Interaksi pengguna sistem E-Commerce "QuickBuy" Manusia-sistem sedangkan Sistem Kesehatan "MediCare" Manusia-mesin-sensor

-Faktor Risiko kesalahan sistem E-Commerce "QuickBuy" Transaksi gagal sedangkan Sistem Kesehatan "MediCare" keselamatan pasien

-Faktor Jenis komponen sistem E-Commerce "QuickBuy" lunak saja sedangkan Sistem Kesehatan "MediCare" Perangkat keras + perangkat lunak

-Faktor Jumlah diagram sistem E-Commerce "QuickBuy" Sedikit (3-4 diagram utama) sedangkan Sistem Kesehatan "MediCare" Banyak (5-7 diagram utama)

• Kesimpulan:

Semakin kompleks dan kritis suatu sistem, semakin banyak jenis diagram yang dibutuhkan untuk menjelaskan semua aspek — mulai dari interaksi, alur kerja, status, hingga struktur teknisnya.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Relevan

Langkah-langkah sistematis untuk menentukan diagram UML yang diperlukan:

1. Analisis kebutuhan sistem

→ Tentukan apakah sistem berbasis web, real-time, atau berisiko tinggi.

-Jika berfokus pada pengguna dan transaksi → gunakan Use Case, Class, dan Sequence Diagram.

-Jika melibatkan kontrol otomatis, perangkat keras, atau kondisi sistem → tambahkan State Machine, Activity, dan Deployment Diagram.

2. Identifikasi tujuan desain

→ Apakah untuk memahami kebutuhan, desain logika, atau implementasi teknis?

-Tahap awal: Use Case dan Activity.

-Tahap desain: Class dan Sequence.

-Tahap implementasi: Component dan Deployment.

3. Perhatikan tingkat risiko dan regulasi

→ Sistem medis memerlukan lebih banyak dokumentasi visual untuk validasi dan keamanan.

4. Gunakan prinsip efisiensi dokumentasi

→ Gunakan hanya diagram yang memberikan nilai tambah. Tidak semua 14 diagram UML perlu dibuat.

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**

oleh [AJAY SUPRIADI 04001670](#) - Selasa, 4 November 2025, 09:37

Analisis Anda sudah bagus, namun belum dilengkapi dengan referensi dan kesimpulan. Coba tambahkan agar argumen Anda lebih kuat.

[Tautan permanen](#) [Tampilkan induk](#)

M

Re: Diskusi.5

oleh [MUHAMMAD FARHAN 051501474](#) - Senin, 3 November 2025, 17:32

Ijin menanggapi diskusi.

Diagram UML Paling Penting untuk Setiap Sistem

Pemilihan diagram harus sejalan dengan metodologi berorientasi objek, suatu strategi pembangunan perangkat lunak yang mengorganisasikan perangkat lunak sebagai kumpulan objek yang berisi data dan operasi.

1. Sistem E-Commerce "QuickBuy"

Sistem ini berfokus pada interaksi pengguna, transaksi, dan aliran kerja bisnis.

* Use Case Diagram: Diagram ini merupakan pemodelan untuk perilaku (behavior) sistem informasi yang akan dibuat.

Penting untuk memetakan fungsionalitas utama dari sudut pandang pengguna, seperti proses pembelian dan pembayaran.

* Activity Diagram: Diagram aktivitas menggambarkan aliran kerja (workflow) atau aktivitas dari sebuah sistem atau proses bisnis. Sangat berguna untuk memvisualisasikan langkah-langkah berurutan dalam proses checkout.

* Class Diagram: Diagram kelas menggambarkan struktur sistem dari segi pendefinisian kelas-kelas yang akan dibuat. Ini mendefinisikan entitas seperti Produk yang memiliki atribut (variabel-variabel) dan metode (fungsi-fungsi atau operasi).

* Sequence Diagram: Diagram sekuen menggambarkan perilaku objek pada use case dengan mendeskripsikan waktu hidup objek dan message yang dikirimkan dan diterima antar objek. Digunakan untuk memodelkan interaksi dinamis saat transaksi terjadi.

2. Sistem Kesehatan "MediCare"

Sistem ini adalah sistem tertanam (embedded), real-time, dan melibatkan hardware dan software kontrol yang kompleks.

- State Machine Diagram: Diagram ini termasuk dalam kategori Behavior Diagram. Diagram ini sangat penting untuk memodelkan perilaku berbasis status yang reaktif dari sistem injeksi insulin, menangani status seperti 'Memantau' atau 'Menyuntikkan' berdasarkan pembacaan sensor.

- Deployment Diagram: Diagram deployment menunjukkan konfigurasi komponen dalam proses eksekusi aplikasi. Diagram ini sangat penting untuk MediCare karena dapat digunakan untuk memodelkan sistem tambahan (embedded system) yang menggambarkan rancangan device, node, dan hardware (seperti sensor dan pompa).

- Use Case Diagram: Tetap diperlukan untuk memodelkan perilaku sistem dan interaksi antara pengguna dan sistem injeksi.

- Activity Diagram: Diperlukan untuk menggambarkan workflow atau algoritma kontrol yang kompleks dalam

penghitungan dan pengiriman dosis insulin.

Perbedaan Jumlah dan Jenis Diagram UML yang Digunakan

Perbedaan jumlah dan jenis diagram UML yang digunakan dalam proyek didasarkan pada kompleksitas, kebutuhan pemodelan, dan risiko sistem.

Bagaimana Kompleksitas Sistem Memengaruhi Jumlah Diagram:

- Pemodelan: Pemodelan adalah gambaran dari realita yang simpel yang dituangkan dalam bentuk pemetaan dengan aturan tertentu. Tujuannya adalah agar sekelompok manusia yang berkepentingan dapat mengerti ide yang akan dikerjakan menggunakan visual, serta untuk merencanakan suatu hal agar kegagalan dan risiko yang mungkin terjadi dapat diminimalisir.
- Kompleksitas Rendah (QuickBuy): Sistem transaksional e-commerce sebagian besar dapat dijelaskan dengan diagram yang fokus pada struktur (Structure Diagram) seperti Class Diagram dan perilaku (Behavior Diagram) seperti Activity dan Sequence Diagram. Jumlah diagram yang dibutuhkan relatif sedikit karena fokusnya adalah proses bisnis yang terdefinisi dengan baik.
- Kompleksitas Tinggi (MediCare): Sistem kritis-kehidupan dan tertanam (embedded) seperti MediCare memerlukan pemodelan yang lebih mendalam pada perilaku reaktif dan lingkungan fisik. Hal ini menyebabkan peningkatan jumlah diagram karena harus menyertakan:
 - * State Machine Diagram: Untuk memodelkan perilaku berbasis event yang sangat penting.
 - * Deployment Diagram: Untuk memodelkan integrasi hardware dan software.

Mengapa Jenis Diagram Berbeda?

Perbedaan jenis diagram muncul karena sifat sistem: QuickBuy adalah sistem client/server atau terdistribusi, sedangkan MediCare secara teknis adalah sistem tambahan (embedded system). Ini secara langsung menentukan perlunya Diagram Deployment dan Diagram State Machine untuk MediCare.

Cara Menentukan Jumlah dan Jenis Diagram UML

Cara menentukan jumlah dan jenis diagram yang paling relevan harus didasarkan pada pendekatan sistematis dan pragmatis.

- Pendekatan Metodologi Berorientasi Objek: Mulailah dengan mengorganisasikan perangkat lunak sebagai kumpulan objek yang berisi data dan operasi, kemudian gunakan pendekatan objek secara sistematis.
- Pemodelan Berdasarkan Kebutuhan: Gunakan pemodelan untuk memvisualisasikan ide dan mengurangi risiko kegagalan. Diagram UML terbagi menjadi Structure Diagram (seperti Class dan Deployment Diagram) dan Behavior Diagram (seperti Use Case, Activity, dan State Machine Diagram). Pilih diagram berdasarkan aspek yang paling membutuhkan visualisasi:
 - * Untuk struktur data \rightarrow Class Diagram.
 - * Untuk aliran kerja \rightarrow Activity Diagram.
 - * Untuk perilaku sistem reaktif \rightarrow State Machine Diagram.
 - * Untuk konfigurasi hardware \rightarrow Deployment Diagram.
- Prinsip Efisiensi: Keuntungan menggunakan metodologi berorientasi objek termasuk meningkatkan produktivitas, kecepatan pengembangan, dan meningkatkan kualitas perangkat lunak. Oleh karena itu, hanya gunakan diagram yang secara nyata membantu mencapai keuntungan tersebut, memastikan diagram yang dibuat memberikan gambaran yang efektif tentang realita sistem yang ingin dibangun.

Sumber: materi inisiasi 5 <https://elearning.ut.ac.id/mod/resource/view.php?id=17012882>

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Selasa, 4 November 2025, 09:38

Bagus, Anda memahami bahwa tidak semua sistem perlu semua jenis diagram. Pemikiran yang efisien!

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AHMAD MINAN NAJIB 050480425](#) - Senin, 3 November 2025, 19:51

Assalamualaikum wr wb

Izin menjawab pertanyaan dikusi 5

1. Diagram UML Paling Penting untuk Setiap Sistem

Sebagai System Analyst, pemilihan diagram UML harus didasarkan pada kebutuhan analisis, desain, dan implementasi sistem. Diagram dipilih untuk menggambarkan aspek-aspek kunci seperti interaksi pengguna, struktur data, alur proses, dan integrasi komponen. Berikut adalah diagram paling penting untuk masing-masing sistem, dengan alasan pemilihannya.

Sistem E-Commerce "QuickBuy"

Sistem ini berfokus pada interaksi pengguna, transaksi, dan pengelolaan data, sehingga diagram yang diperlukan lebih sederhana dan berorientasi pada fungsi bisnis. Diagram utama yang paling relevan adalah:

- Use Case Diagram: Menggambarkan interaksi antara pengguna (pembeli, admin) dan sistem, seperti "Memilih Produk", "Menambahkan ke Keranjang", "Melakukan Pembayaran", dan "Melacak Pengiriman". Alasan: Ini membantu mengidentifikasi kebutuhan fungsional utama dan stakeholder, yang penting untuk platform berbasis web yang interaktif.
- Class Diagram: Menunjukkan struktur objek seperti kelas "Produk", "Keranjang Belanja", "Pesanan", dan "Pengguna". Alasan: Sistem ini bergantung pada pengelolaan data relasional (misalnya, database produk), sehingga diagram ini mendukung desain basis data dan hubungan antar entitas.
- Sequence Diagram: Mengilustrasikan urutan interaksi waktu, seperti proses checkout dari pemilihan produk hingga konfirmasi pembayaran. Alasan: E-commerce melibatkan alur transaksi yang linier dan berbasis waktu, membantu memastikan integritas proses dan menghindari kesalahan seperti duplikasi pembayaran.
- Activity Diagram: Menggambarkan alur kerja umum, seperti "Belanja Online" dari pencarian produk hingga pengiriman. Alasan: Ini menyederhanakan pemahaman proses bisnis yang berulang, cocok untuk sistem yang lebih statis dan kurang melibatkan hardware.

Sistem ini mungkin hanya memerlukan 3-5 diagram utama, karena kompleksitasnya lebih rendah (fokus pada software dan UI/UX).

Sistem Kesehatan "MediCare"

Sistem ini melibatkan hardware (sensor, pompa insulin), algoritma real-time, keamanan tinggi, dan interaksi kritis (misalnya, pemantauan kadar gula darah), sehingga membutuhkan diagram yang lebih mendalam untuk menangani aspek teknis dan safety-critical. Diagram utama yang paling relevan adalah:

- Use Case Diagram: Menggambarkan interaksi seperti "Memantau Kadar Gula Darah", "Mengatur Dosis Insulin", dan "Mendapatkan Notifikasi Darurat" untuk pasien, dokter, dan sistem. Alasan: Ini mengidentifikasi kebutuhan fungsional dalam konteks kesehatan, termasuk skenario kegagalan (misalnya, sensor error), yang krusial untuk sistem medis.
 - Class Diagram: Menunjukkan struktur objek seperti kelas "SensorGlukosa", "PompaInsulin", "AlgoritmaKontrol", dan "DataPasien". Alasan: Sistem ini melibatkan integrasi hardware-software, sehingga diagram ini membantu mendesain antarmuka dan hubungan data yang kompleks, termasuk enkripsi untuk keamanan.
 - State Machine Diagram: Menggambarkan status sistem, seperti transisi dari "Normal" ke "Hipoglikemia" berdasarkan pembacaan sensor, yang memicu injeksi otomatis. Alasan: Sistem ini bergantung pada perilaku dinamis dan real-time (misalnya, respons terhadap fluktuasi gula darah), sehingga diagram ini penting untuk memodelkan logika kontrol dan mencegah risiko keselamatan.
 - Sequence Diagram: Mengilustrasikan interaksi waktu antara komponen, seperti sensor mengirim data ke algoritma, yang kemudian mengaktifkan pompa insulin. Alasan: Ini mendukung analisis timing dan sinkronisasi dalam sistem embedded, membantu mengidentifikasi bottleneck atau kegagalan komunikasi.
 - Component Diagram: Menunjukkan komponen seperti "Modul Sensor", "Unit Kontrol", dan "Interface Pengguna", termasuk dependensi hardware. Alasan: Sistem ini melibatkan perangkat keras fisik, sehingga diagram ini membantu desain integrasi dan modularitas.
 - Deployment Diagram: Menggambarkan bagaimana komponen dideploy, seperti sensor yang terhubung ke perangkat wearable dan server cloud untuk pemantauan jarak jauh. Alasan: Ini penting untuk sistem kesehatan yang mungkin melibatkan IoT dan keamanan jaringan, memastikan skalabilitas dan compliance regulasi (misalnya, HIPAA).
- Sistem ini mungkin memerlukan 6-10 diagram atau lebih, karena kompleksitasnya tinggi (safety-critical, real-time, hardware integration).

2. Mengapa Jumlah dan Jenis Diagram UML Berbeda Antara Sistem E-Commerce dan Sistem Injeksi Insulin Otomatis?

Jumlah dan jenis diagram berbeda karena kompleksitas sistem memengaruhi kedalaman analisis yang diperlukan.

Sistem e-commerce seperti QuickBuy umumnya lebih sederhana: berbasis software, dengan fokus pada interaksi pengguna dan data transaksi, sehingga diagram utama (seperti Use Case, Class, dan Sequence) cukup untuk

menangkap esensi tanpa perlu detail hardware atau perilaku dinamis yang rumit. Ini menghasilkan jumlah diagram yang lebih sedikit (3-5), karena sistemnya lebih statis dan dapat diprediksi.

Sebaliknya, sistem kesehatan seperti MediCare lebih kompleks: melibatkan hardware (sensor, pompa), algoritma real-time, keamanan tinggi, dan risiko keselamatan (misalnya, overdosis insulin bisa fatal). Kompleksitas ini memerlukan diagram tambahan seperti State Machine (untuk perilaku dinamis), Component (untuk integrasi hardware), dan Deployment (untuk infrastruktur fisik), sehingga jumlah diagram meningkat (6-10+). Faktor seperti domain aplikasi (bisnis vs. medis), tingkat safety-critical, dan integrasi teknologi (software-only vs. embedded/IoT) langsung memengaruhi: sistem yang lebih kompleks membutuhkan lebih banyak diagram untuk mengurangi risiko, memastikan akurasi, dan memfasilitasi komunikasi antar tim (misalnya, engineer hardware dan software).

Secara umum, kompleksitas sistem (dihitung berdasarkan jumlah komponen, interaksi, dan persyaratan non-fungsional seperti keamanan) menentukan skala diagram sistem sederhana cukup dengan diagram inti, sedangkan yang kompleks membutuhkan spesialisasi untuk aspek seperti concurrency, fault tolerance, atau deployment.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Paling Relevan dalam Suatu Proyek Pengembangan Perangkat Lunak

Penentuan ini dilakukan secara iteratif berdasarkan metodologi rekayasa perangkat lunak berorientasi objek (seperti UML atau Agile), dengan mempertimbangkan fase proyek, stakeholder, dan kompleksitas. Langkah-langkah praktis:

1. Identifikasi Kebutuhan Awal: Mulai dengan analisis kebutuhan (requirements gathering). Gunakan diagram inti seperti Use Case Diagram untuk memahami fungsi utama dan stakeholder. Ini memberikan baseline—misalnya, jika sistem melibatkan banyak interaksi pengguna, prioritaskan Use Case dan Sequence.
2. Evaluasi Kompleksitas Sistem: Nilai faktor seperti ukuran proyek (jumlah kelas/objek), domain (bisnis, medis, embedded), persyaratan non-fungsional (keamanan, performa, real-time), dan integrasi (hardware, API). Gunakan metrik seperti jumlah use case atau tingkat coupling untuk estimasi: sistem sederhana (<50 use case) mungkin butuh 3-5 diagram, sedangkan yang kompleks (>100 use case, safety-critical) butuh 6-15+.
3. Pilih Diagram Berdasarkan Fase Proyek:
 - Analisis: Use Case, Activity, Class.
 - Desain: Sequence, State Machine, Component.
 - Implementasi/Deployment: Deployment, Component.
- Hindari over-engineering, jangan gunakan diagram jika tidak diperlukan; fokus pada yang memberikan nilai (misalnya, State Machine hanya untuk sistem dengan perilaku dinamis).
4. Libatkan Stakeholder dan Iterasi: Diskusikan dengan tim (developer, tester, domain expert). Gunakan tools seperti Enterprise Architect atau Visual Paradigm untuk prototyping. Lakukan review iteratif: mulai minimal, tambah diagram jika muncul masalah (misalnya, tambah State Machine jika ada bug dalam alur).
5. Pertimbangkan Standar dan Best Practices: Ikuti panduan UML (OMG) atau metodologi seperti RUP. Untuk proyek besar, gunakan diagram subset (misalnya, 4+1 view model oleh Kruchten: logical, process, physical, development, use case). Evaluasi trade-off: lebih banyak diagram meningkatkan akurasi tapi memperlambat proyek—targetkan 80% coverage kebutuhan dengan 20% diagram.

Dengan pendekatan ini, jumlah diagram disesuaikan agar efisien dan cukup untuk mendukung desain tanpa redundansi, memastikan proyek on-time dan berkualitas.

Referensi

BMP MSIM4303 Modul 5

Pressman, R.S. (2001). Software engineering: A practitioner's approach. fifth edition. New York: Mc Graw Hill.

Sommerville, I. (2016). Software engineering. 10th edition. New York: McGraw-Hill.

Sukamto. R.A. (2018). Rekayasa perangkat lunak terstruktur dan berorientasi objek Bandung: Penerbit Informatika.

<https://www.omg.org>

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Selasa, 4 November 2025, 09:38

Jawaban Anda menunjukkan kematangan berpikir dalam perancangan sistem. Lanjutkan perjuangannya! referensi utamakan dari universitas terbuka

Tautan permanen Tampilkan induk

**Re: Diskusi.5**oleh [054276582 MUHAMMAD RIFKI ARSAH](#) - Senin, 3 November 2025, 21:22

1. Diagram UML paling penting meliputi Use Case, Class, Sequence, dan Activity karena memvisualisasikan kebutuhan, struktur, interaksi objek, serta alur proses sistem. Pemilihan diagram bergantung pada kompleksitas sistem dan fokus proyek.

Untuk Sistem E-commerce:

- Use Case Diagram: Penting untuk mengidentifikasi aktor (pelanggan, admin) dan fungsionalitas utama (melihat produk, menambahkan ke keranjang, checkout, mengelola pesanan).
- Class Diagram: Untuk memodelkan entitas seperti Produk, Pelanggan, Pesanan, dan Keranjang.
- Sequence Diagram: Untuk menjelaskan interaksi kompleks saat checkout, seperti aliran pesan antara Keranjang, Pesanan, dan Pembayaran.

Untuk Sistem Kesehatan Berbasis Injeksi Insulin Otomatis:

- Activity Diagram: Sangat vital untuk memodelkan alur kerja medis, seperti pengukuran glukosa, perhitungan dosis, dan proses injeksi otomatis.
- State Machine Diagram: Penting untuk memodelkan status perangkat (misalnya, Ready, Injecting, Low Battery, Error) dan transisi antar status tersebut.
- Sequence Diagram: Untuk menggambarkan interaksi antara sensor glukosa, unit kontrol, dan pompa insulin.

2. Jumlah dan Jenis Berbeda: Proyek berbeda membutuhkan representasi yang berbeda karena tingkat detail dan perspektif yang berbeda. Sistem sederhana mungkin hanya butuh beberapa diagram, sementara sistem kompleks membutuhkan lebih banyak diagram untuk menguraikan berbagai aspek.

Dampak Kompleksitas:

- Sistem Sederhana: Cukup dengan Use Case dan Class Diagram untuk menangkap kebutuhan dan struktur dasar.
- Sistem Kompleks: Memerlukan lebih banyak diagram (Sequence, Activity, State Machine, Component, Deployment) untuk menganalisis perilaku dinamis, interaksi kompleks, dan arsitektur perangkat keras/lunak.
- Sistem Kritis (misal, kesehatan): Fokus pada detail dan keandalan, sehingga diagram seperti State Machine menjadi sangat penting untuk memodelkan perilaku perangkat yang presisi.

3. Penentuan diagram UML yang relevan dilakukan melalui pendekatan berikut:

- Pahami Kebutuhan Bisnis dan Pengguna: Gunakan Use Case Diagram untuk mengidentifikasi semua fungsi dan interaksi aktor.
- Analisis Struktur Sistem: Identifikasi kelas-kelas utama, atributnya, dan hubungan antar kelas menggunakan Class Diagram.
- Modelkan Alur Kerja: Untuk proses bisnis atau alur kerja yang kompleks, gunakan Activity Diagram.
- Visualisasikan Interaksi Dinamis: Untuk interaksi antar objek yang berurutan dalam waktu, gunakan Sequence Diagram.
- Pahami Perilaku Objek yang Berubah Status: Untuk sistem dengan banyak keadaan (misalnya, perangkat elektronik atau status transaksi), State Machine Diagram sangat membantu.

Sumber:

<https://www.dicoding.com/blog/apa-itu-uml/>

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Selasa, 4 November 2025, 09:40

Bagus sekali, Anda sudah memahami perbedaan kompleksitas antara dua sistem. Sukses selalu dalam belajar!

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**oleh [MUHAMMAD RIZQI PRATAMA 053580633](#) - Senin, 3 November 2025, 21:29

1. Diagram UML yang penting untuk sistem QuickBuy dan MediCare.

QuickBuy adalah sistem yang berfokus pada interaksi pengguna, transaksi, dan pengelolaan data produk. Diagram UML yang paling cocok:

- QuickBuy

Use Case Diagram

Menggambarkan aktor lebih dari satu seperti admin, pelanggan, kurir dan lain-lain untuk mendefinisikan apa saja yang bisa dilakukan oleh aktor. Alasannya karena memudahkan pemahaman dan memudahkan identifikasi kebutuhan utama sistem seperti daftar, browse, checkout, track order, add to cart dan lain lain.

Class Diagram

dibuat sebagai Blueprint database dan logika bisnis, sebagai penghubung antar kelas-kelas seperti Pengguna, Pesanan, Produk, Payment, Shipment. Alasannya untuk mendesain struktur data dan relasi untuk implementasi database.

Sequence Diagram

Menggambarkan perilaku objek kepada use case dengan mendeskripsikan waktu hidup objek dan message yang dikirimkan dan diterima antar objek. Alasannya untuk menangani detail pesanan, atau jika terjadi error flows saat transaksi.

Activity Diagram

Menggambarkan workflow/aliran kerja atau aktifitas dari sebuah sistem atau proses bisnis atau menu yang ada pada perangkat lunak. Seperti proses validasi pembayaran dan penanganan jika gagal transaksi. Alasannya untuk visualisasi alur proses bisnis.

- MediCare

Use Case Diagram

Menggambarkan aktor lebih dari satu seperti dokter, pasien, apoteker dan lain-lain untuk mendefinisikan apa saja yang bisa dilakukan oleh aktor. Alasannya karena memudahkan pemahaman dan memudahkan identifikasi kebutuhan utama seperti sistem seperti monitor, alarm, inject insulin.

Class Diagram

dibuat sebagai Blueprint database dan logika sistem, sebagai penghubung antar kelas-kelas seperti model pasien, sensor, penentuan dosis. Alasannya untuk mendesain struktur data dan relasi untuk implementasi database.

Sequence Diagram

Menggambarkan perilaku objek kepada use case dengan mendeskripsikan waktu hidup objek dan message yang dikirimkan dan diterima antar objek. Alasannya untuk menangani atau merespons sensor, battery, koneksi.

Activity Diagram

Menggambarkan workflow/aliran kerja atau aktifitas dari sebuah sistem pada perangkat lunak MediCare. Seperti proses pengambilan keputusan dosis pengguna.

State Machine Diagram

Sebagai pemetaan transisi antar keadaan agar sistem tahu keadaan setiap saat, seperti Monitoring, Alert, Inject Insulin, Fault. Alasannya untuk mengetahui setiap keadaan karena sistem harus tahu pada setiap keadaan mengingat sistem ini sangat krusial karena menyangkut nyawa pasien.

Diagram Component

Digunakan untuk menghubungkan antar hardware, software, cloud service dan lain-lain. Alasannya sebagai penghubung antar perangkat keras dan software.

Diagram Deployment

sebagai penunjuk konfigurasi komponen dalam proses eksekusi aplikasi seperti embedded controller, gateway, cloud.

2. Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda?

Karena setiap sistem memiliki tujuan, kompleksitas dan konsekuensi kegagalan yang berbeda-beda. Contoh

Kompleksitas pada Sistem QuickBuy adalah terkait transaksional dan data. Banyak aturan bisnis, proses bisnis, banyaknya pengguna, dan banyaknya data. sehingga Resiko bisnis adalah kehilangan uang atau pelanggan tidak senang. Maka Diagram lebih fokus pada alur dan proses data.

Sedangkan kompleksitas MediCare adalah terkait perilaku dan real time. Pada Sistem ini merespon peristiwa di dunia nyata secara instan dan akurat. Sehingga resiko aplikasi ini adalah dapat mencelakai pasien. Maka diagram harus fokus pada status, waktu, lokasi, dan interaksi hardware.

Kesimpulan

Jumlah diagram akan meningkat jika jumlah sudut pandang untuk mengelola resiko sangat tinggi. MediCare lebih banyak diagram karena banyak sudut pandang seperti hardware, harus real-time, keamanan, pengambilan keputusan yang akurat.

3. Menentukan jumlah dan jenis diagram yang relevan

Analisis kebutuhan sistem dengan pendekatan atau cara pandang di mana tindakan dan keputusan didorong oleh penilaian risiko.

-Mulai dengan Diagram minimum seperti Diagram Use Case agar dapat memahami sistem yang dibuat akan diarahkan kemana. Kemudian Diagram Kelas untuk pemahaman struktur dasarnya.

-Identifikasi tujuan sistem

Apakah sistem berfokus pada pengguna dan transaksi maka buatlah diagram aktivitas. Kemudian apakah sistem bingung perlu pemanggilan API dulu atau layanan maka buat Diagram Sekuens untuk skenario spesifik.

Jika Pengembang bingung terhadap proses pesanan kapan selesai kapan dikirim kapan dibayar. maka perlu membuat Diagram Mesin Keadaan. Jika sistem perlu ada interaksi ke perangkat keras maka buat Diagram Komponen atau Deployment Diagram.

-Gunakan Diagram sebagai penyelesaian masalah pembuatan sistem. Diagram digunakan apabila terjadi kebingungan pada pembuatan sistem.

Kesimpulan

Jumlah diagram yang tepat adalah jumlah minimum yang diperlukan untuk membangun sistem yang benar, aman dan minim resiko. Gunakan Diagram sebagai pemecah kebuntuan pada proses pembuatan Sistem.

Sumber: Universitas Terbuka. BMP Rekayasa Perangkat Lunak (MSIM430301). Modul 5

Tautan permanen Tampilkan induk

**Re: Diskusi.5**

oleh [AJAY SUPRIADI 04001670](#) - Selasa, 4 November 2025, 09:40

Jawaban Anda singkat tapi padat. Lanjutkan semangat belajarnya dan semoga selalu diberi kelancaran!

Tautan permanen Tampilkan induk

**Re: Diskusi.5**

oleh [053929596 FEIZA RAHMAH ASSYIFA](#) - Selasa, 4 November 2025, 06:45

Nama: Feiza Rahmah Assyifa

NIM: 053929596

Prodi: Sistem Informasi

Selamat pagi pak Ajay Supriadi, S.Kom., M.Kom. Izinkan saya memberikan tanggapan atas soal pada diskusi sesi 5 ini.

Analisis Diagram UML untuk Sistem QuickBuy (E-Commerce) dan MediCare (Kesehatan)

1. Diagram UML yang Paling Penting untuk Setiap Sistem

UML yang dipilih adalah QuickBuy untuk sistem E-Commerce dan MediCare untuk sistem kesehatan. QuickBuy berfokus pada proses bisnis digital, pengelolaan data transaksi, dan pengalaman pengguna. Sementara MediCare menekankan pada pemantauan medis otomatis yang melibatkan perangkat keras dan algoritma pengendali insulin. Kedua sistem ini memiliki kebutuhan diagram yang berbeda karena kompleksitas dan tujuan desainnya tidak sama.

Sistem E-Commerce (QuickBuy)

Sistem QuickBuy merupakan platform belanja online yang dirancang untuk mempermudah pengguna dalam memilih produk, melakukan pembayaran, dan melacak pengiriman pesanan. Sistem ini berfokus pada kenyamanan pengguna, efisiensi transaksi, serta keamanan data pelanggan dalam setiap proses pembelian. Diagram UML yang digunakan difokuskan pada pemodelan aktivitas bisnis, alur kerja, serta relasi data antara komponen utama dalam sistem. Dengan pendekatan ini, rancangan sistem menjadi lebih terstruktur, mudah dikembangkan, dan mampu memenuhi kebutuhan bisnis digital.

Sistem ini bersifat digital sepenuhnya dan tidak berinteraksi dengan perangkat keras, sehingga diagram yang digunakan menitikberatkan pada aspek fungsional dan struktural. Tujuan utamanya adalah menggambarkan bagaimana data dan aktivitas saling terhubung dalam proses e-commerce. Diagram UML yang dipilih membantu tim pengembang memahami proses transaksi dari perspektif pengguna dan sistem. Hal ini menjadikan rancangan QuickBuy mudah diimplementasikan dan dikembangkan sesuai kebutuhan pasar online yang dinamis.

- Use Case Diagram (QuickBuy)

Diagram ini menggambarkan interaksi antara aktor seperti pelanggan, admin, dan sistem pembayaran dengan fitur utama seperti login, pemilihan produk, dan checkout. Diagram ini berfungsi untuk mengidentifikasi kebutuhan fungsional sistem secara menyeluruh. Dengan diagram ini, batasan sistem dan peran setiap aktor menjadi jelas sejak tahap analisis. Hal ini juga mempermudah komunikasi antara tim teknis dan pemangku kepentingan non-teknis.

- Class Diagram (QuickBuy)

Diagram ini menampilkan struktur kelas dan hubungan antar entitas seperti User, Product, Cart, Order, dan Payment. Diagram ini membantu menggambarkan bagaimana data disimpan dan diproses dalam sistem. Setiap kelas memiliki atribut dan operasi yang relevan dengan fungsi e-commerce. Dengan demikian, diagram ini menjadi dasar dalam perancangan basis data dan logika pemrograman sistem.

- State Machine Diagram (QuickBuy)

Diagram ini digunakan untuk menggambarkan perubahan status pesanan mulai dari Pending → Paid → Shipped → Delivered → Completed. Diagram ini memastikan bahwa setiap transisi status mengikuti aturan bisnis yang berlaku. Dengan pemodelan ini, pengembang dapat mengantisipasi kemungkinan kesalahan status dalam sistem. Selain itu, diagram ini membantu memantau siklus hidup pesanan dari awal hingga akhir.

- Sequence Diagram (QuickBuy)

Diagram ini menjelaskan urutan komunikasi antar objek selama proses transaksi berlangsung. Diagram ini menunjukkan bagaimana pengguna, sistem e-commerce, dan pihak ketiga seperti gateway pembayaran saling berinteraksi. Setiap langkah komunikasi divisualisasikan secara kronologis untuk menghindari ambiguitas proses. Dengan diagram ini, pengembang dapat memastikan integrasi sistem berjalan secara sinkron dan efisien.

- Activity Diagram (QuickBuy)

Diagram ini memvisualisasikan alur aktivitas bisnis seperti registrasi, pembelian, dan pelacakan pesanan. Diagram ini membantu memahami logika keputusan dan jalur aktivitas sistem secara menyeluruh. Dengan diagram ini, pengembang dapat mengidentifikasi area proses yang perlu dioptimalkan. Selain itu, diagram ini juga mendukung dokumentasi proses bisnis yang lebih transparan.

- Deployment Diagram (QuickBuy)

Diagram ini menggambarkan struktur fisik sistem, termasuk client, web server, dan database server. Diagram ini digunakan untuk memvisualisasikan bagaimana komponen perangkat lunak dideploy di lingkungan jaringan.

Dengan diagram ini, arsitektur sistem dapat dirancang agar efisien dan mudah diakses. Diagram ini juga membantu memastikan keamanan dan kestabilan koneksi antar komponen.

- **Sistem Kesehatan (MediCare)**

Sistem MediCare merupakan sistem otomatisasi medis yang dirancang untuk membantu penderita diabetes dengan memantau kadar gula darah secara real-time. Sistem ini bekerja menggunakan sensor dan pompa insulin otomatis yang dikendalikan oleh algoritma perangkat lunak. Diagram UML digunakan untuk menggambarkan hubungan antar komponen sistem dan alur kerja otomatis yang terjadi secara terus-menerus. Tujuan utamanya adalah memastikan keandalan, akurasi, dan keamanan sistem medis selama beroperasi.

Sistem ini melibatkan interaksi antara perangkat keras dan perangkat lunak, sehingga memerlukan diagram yang lebih kompleks dan terperinci. Diagram UML digunakan untuk menggambarkan perilaku sistem dalam berbagai kondisi fisiologis pasien. Kompleksitas ini menjadikan MediCare membutuhkan diagram tambahan seperti State Machine dan Deployment Diagram untuk mengilustrasikan integrasi sensor dan aktuator. Hal ini penting untuk menjaga keselamatan pasien dan efektivitas sistem secara keseluruhan.

- **Use Case Diagram (MediCare)**

Diagram ini menggambarkan peran aktor seperti pasien, dokter, dan sistem pemantauan medis. Diagram ini menjelaskan fungsi utama seperti pengukuran kadar gula, injeksi insulin otomatis, dan pengiriman laporan medis. Dengan diagram ini, kebutuhan sistem dari perspektif pengguna dapat teridentifikasi secara jelas. Hal ini juga membantu memastikan sistem memenuhi standar medis dan keamanan.

- **Class Diagram (MediCare)**

Diagram ini menunjukkan struktur komponen sistem seperti Sensor, Controller, Pump, dan AlertSystem. Diagram ini menjelaskan hubungan antar kelas dan metode yang digunakan untuk memproses data medis. Setiap kelas memiliki fungsi yang mendukung pengambilan keputusan otomatis berdasarkan data sensor. Diagram ini memastikan sistem berjalan secara modular dan mudah diuji.

- **State Machine Diagram (MediCare)**

Diagram ini menjelaskan perubahan status sistem seperti Idle → Monitoring → Injecting → Alert. Diagram ini membantu pengembang memahami bagaimana sistem merespons perubahan kadar gula darah pasien. Dengan diagram ini, setiap kondisi kritis dapat diidentifikasi dan ditangani dengan tepat. Hal ini memastikan sistem medis berfungsi secara aman dan stabil.

- **Sequence Diagram (MediCare)**

Diagram ini menggambarkan urutan interaksi antara sensor, kontroler, dan pompa insulin. Diagram ini memperlihatkan alur komunikasi data dari deteksi kadar gula hingga penyuntikan insulin. Dengan diagram ini, proses sinkronisasi antar komponen dapat dipastikan berjalan lancar. Diagram ini juga membantu meminimalkan kesalahan waktu dalam sistem real-time.

- **Activity Diagram (MediCare)**

Diagram ini menampilkan aliran aktivitas seperti pembacaan sensor, perhitungan dosis, dan pemberian insulin. Diagram ini menjelaskan logika proses yang dilakukan sistem berdasarkan input medis. Dengan diagram ini, alur kerja sistem dapat diuji dan diverifikasi dengan mudah. Diagram ini juga membantu mengoptimalkan efisiensi proses otomatisasi medis.

- **Deployment Diagram (MediCare)**

Diagram ini menggambarkan hubungan antara perangkat keras, perangkat lunak, dan jaringan komunikasi medis. Diagram ini menunjukkan bagaimana sistem sensor, pompa insulin, dan aplikasi pemantauan saling terhubung. Dengan diagram ini, arsitektur fisik sistem dapat dipahami secara menyeluruh. Diagram ini penting untuk memastikan setiap komponen medis beroperasi secara sinkron dan aman.

2. Mengapa Jumlah dan Jenis Diagram UML Berbeda

Alasan jumlah dan jenis diagram UML berbeda karena setiap sistem memiliki kebutuhan, risiko, dan tingkat kompleksitas yang bervariasi. Sistem seperti QuickBuy berfokus pada transaksi digital dan manajemen data, sementara MediCare menangani perangkat medis yang melibatkan aspek keselamatan pengguna. Perbedaan ini mengharuskan penggunaan diagram yang disesuaikan dengan karakteristik sistem. Semakin kompleks dan kritis suatu sistem, semakin banyak diagram yang dibutuhkan untuk menggambarkan setiap aspeknya secara detail.

- **Kompleksitas Domain**

Sistem e-commerce memiliki domain bisnis yang relatif sederhana, sedangkan sistem medis memiliki domain yang kompleks dengan aturan dan batasan biologis. Kompleksitas domain ini memengaruhi banyaknya diagram

yang dibutuhkan untuk menggambarkan setiap elemen sistem. Semakin kompleks domain, semakin rinci pemodelannya. Dengan demikian, sistem medis memerlukan pemodelan yang lebih luas dibandingkan sistem bisnis digital.

- Sifat & Interaksi Sistem

QuickBuy berinteraksi dengan pengguna melalui antarmuka web, sementara MediCare berinteraksi dengan perangkat keras dan sensor biologis. Perbedaan ini menyebabkan sistem medis memerlukan lebih banyak diagram perilaku untuk menggambarkan interaksi antar komponen. Diagram tambahan dibutuhkan untuk memastikan setiap interaksi berjalan sesuai tujuan. Hal ini penting untuk menjamin akurasi dalam sistem otomatis.

- Pesyaratan Kualitas

Sistem medis memiliki standar kualitas yang tinggi, seperti keamanan dan reliabilitas sistem yang harus terjamin. Dokumentasi diagram lebih banyak dibutuhkan untuk mendukung proses validasi dan sertifikasi medis.

Sementara itu, sistem e-commerce hanya membutuhkan dokumentasi fungsional utama. Hal ini menjadikan jumlah diagram berbeda tergantung pada standar kualitas yang diterapkan.

- Tipe Pemodelan

E-commerce menggunakan pemodelan struktural dan fungsional yang menggambarkan hubungan antar data. Sebaliknya, sistem medis menggunakan pemodelan perilaku dan arsitektur untuk menjelaskan logika kontrol dan perubahan status. Perbedaan ini menyebabkan variasi jumlah diagram yang digunakan. Semakin kompleks tipe pemodelan, semakin banyak diagram yang dibutuhkan.

- Interaksi dan Lingkungan

Sistem e-commerce beroperasi di lingkungan digital dengan data yang stabil, sementara sistem medis bekerja di lingkungan fisik dengan variabilitas tinggi. Kondisi ini membuat sistem medis memerlukan lebih banyak diagram untuk memetakan kemungkinan perubahan lingkungan. Diagram seperti State Machine dan Deployment menjadi penting. Dengan demikian, pemodelan dapat mencakup seluruh kondisi sistem.

- Jenis Kompleksitas

Kompleksitas sistem dapat berupa struktural, fungsional, atau dinamis. QuickBuy hanya memiliki kompleksitas fungsional, sedangkan MediCare mencakup ketiganya secara bersamaan. Kompleksitas yang lebih tinggi memerlukan pemodelan tambahan untuk menjaga keakuratan sistem. Oleh karena itu, MediCare membutuhkan lebih banyak diagram dibandingkan QuickBuy.

Bagaimana Kompleksitas Sistem Mempengaruhi Diagram

Kompleksitas sistem sangat memengaruhi jumlah dan jenis diagram UML yang dibutuhkan. Semakin besar jumlah komponen dan tingkat integrasi, semakin banyak diagram yang diperlukan untuk menggambarkan perilaku dan struktur sistem dengan detail. Pemodelan yang tepat membantu meminimalkan risiko kesalahan analisis dan desain. Oleh karena itu, pemilihan diagram harus disesuaikan dengan tingkat kompleksitas proyek.

- QuickBuy

Sistem ini memiliki kompleksitas menengah karena hanya berfokus pada aktivitas pengguna dan transaksi online. Diagram yang digunakan terbatas pada pemodelan fungsional dan data. Hal ini membuat QuickBuy hanya membutuhkan beberapa diagram utama. Dengan pendekatan sederhana, sistem ini mudah dikembangkan dan dikelola.

- MediCare

Sistem ini memiliki kompleksitas tinggi karena berinteraksi langsung dengan tubuh manusia melalui sensor dan aktuator. Diagram tambahan seperti State Machine dan Deployment diperlukan untuk menggambarkan perilaku sistem yang dinamis. Setiap komponen harus dimodelkan secara detail untuk menghindari kesalahan fungsi. Hal ini menjadikan MediCare membutuhkan pemodelan UML yang lebih luas dan mendalam.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Paling Relevan

Menentukan jumlah dan jenis diagram UML sangat penting agar desain sistem efisien, terarah, dan mudah dipahami. Diagram yang tepat membantu tim menghindari kesalahan analisis dan mempercepat proses pengembangan. Pemilihan jenis diagram harus disesuaikan dengan kompleksitas, tujuan, dan risiko sistem. Diagram juga berfungsi sebagai alat komunikasi antar tim teknis dan non-teknis. Cara Menentukan jumlah dan jenis UML yang paling relevan adalah:

Analisis Domain, Tujuan, dan Risiko

- Menentukan ruang lingkup dan tujuan utama sistem.

- Mengidentifikasi tingkat risiko dari kesalahan sistem.
- Menyesuaikan jumlah diagram dengan kompleksitas domain.
- Menggunakan diagram tambahan bila sistem bersifat kritis.
- Mengurangi diagram jika sistem sederhana.

Analisis Stakeholder, Kebutuhan, dan Area Kritis

- Memahami kebutuhan pengguna utama.
- Menentukan area sistem yang berpengaruh besar pada operasi.
- Memilih diagram yang menjelaskan proses utama.
- Menyelaraskan pemodelan dengan ekspektasi stakeholder.
- Menggunakan diagram sebagai alat validasi kebutuhan.

Gunakan Pendekatan Iteratif dan Prinsip Minimal Yet Sufficient

- Membuat diagram bertahap sesuai perkembangan proyek.
- Menghindari diagram yang tidak diperlukan.
- Menambahkan diagram jika sistem berkembang lebih kompleks.
- Menjaga dokumentasi tetap efisien.
- Mengevaluasi relevansi setiap diagram secara berkala.

Tahapan Proyek

- Menggunakan Use Case dan Activity Diagram di tahap analisis.
- Menggunakan Class dan Sequence Diagram di tahap desain.
- Menggunakan Deployment Diagram di tahap implementasi.
- Memeriksa konsistensi diagram pada tahap pengujian.
- Memperbarui diagram pada tahap pemeliharaan.

Terapkan Prinsip "Diagram sebagai Alat Komunikasi"

- Menjadikan diagram sebagai media komunikasi lintas tim.
- Menghindari istilah teknis berlebihan untuk stakeholder non-teknis.
- Membuat diagram mudah dipahami secara visual.
- Menggunakan diagram untuk diskusi dan presentasi proyek.
- Menstandarkan format diagram di seluruh proyek.

Perimbangan Standar Tim dan Industri secara Kebutuhan dan Regulasi

- Menyesuaikan diagram dengan standar industri terkait.
- Menggunakan format UML yang seragam dalam proyek.
- Menambahkan diagram bila diwajibkan untuk audit.
- Memastikan dokumentasi sesuai peraturan resmi.
- Menggunakan perangkat lunak UML yang diakui industri.

Menerapkan View 4+1 Philippe Kruchten

- Logical View menggambarkan struktur kelas dan objek.
- Process View menjelaskan interaksi antar proses.
- Physical View memperlihatkan struktur deployment sistem.
- Development View menunjukkan komponen pengembangan.
- Scenario View menggambarkan perilaku sistem dalam konteks nyata.

Sumber Referensi:

- Booch, G., Rumbaugh, J., & Jacobson, I. (1999). The Unified Modeling Language User Guide (1st ed.). Addison-Wesley. https://books.google.com/books/about/The_Unified_Modeling_Language_User_Guide.html?id=BqFQAAAAMAAJ

- Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The Unified Modeling Language Reference Manual (2nd ed.). Addison-Wesley. <https://repository.unikom.ac.id/37898/1/Addison%20Wesley%20-%20UML%20Reference%20Manual.pdf>
- Object Management Group. (2017). Unified Modeling Language (UML) Specification Version 2.5.1. <https://www.omg.org/spec/UML/2.5.1/>
- Kruchten, P. (1995). Architectural Blueprints—The “4+1” View Model of Software Architecture. IEEE Software, 12(6), 42–50. <https://www.cs.ubc.ca/~gregor/teaching/papers/4%2B1view-architecture.pdf>

Tautan permanen Tampilkan induk

**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Selasa, 4 November 2025, 09:41

Penjelasan Anda menunjukkan pemahaman yang baik tentang OOP dan UML. Terus asah kemampuan analitisnya!

Tautan permanen Tampilkan induk

**Re: Diskusi.5**oleh [MUHAMMAD FERDIANSYAH 053535258](#) - Selasa, 4 November 2025, 21:14

Izin menjawab

Diagram UML Penting untuk Setiap Sistem

Untuk sistem e-commerce "QuickBuy" yang fokus pada interaksi pengguna, transaksi, dan pengelolaan data produk, diagram UML yang penting biasanya meliputi:

- Use Case Diagram: Menggambarkan kebutuhan dan interaksi utama antara pengguna dan sistem.
- Class Diagram: Menjelaskan struktur data dan relasi antar objek dalam sistem.
- Sequence Diagram: Menunjukkan alur interaksi antara objek selama proses transaksi.
- Activity Diagram: Menggambarkan proses bisnis seperti alur pembelian dan pembayaran.

Sedangkan pada sistem kesehatan "MediCare" yang melibatkan perangkat keras, sensor, algoritma kompleks, dan keamanan tinggi, dibutuhkan diagram yang lebih lengkap, seperti:

- Use Case Diagram: Untuk menetapkan kebutuhan pengguna dan fungsi sistem.
- Class Diagram: Untuk struktur data dan komponen sistem.
- Sequence dan Communication Diagram: Untuk interaksi antara software dan hardware.
- State Machine Diagram: Penting untuk menggambarkan alur kerja perangkat yang kompleks, termasuk perubahan status berdasarkan input sensor dan kondisi medis.
- Deployment Diagram: Untuk memetakan perangkat keras yang terlibat dan hubungan fisik antar komponen.
- Activity Diagram: Untuk menjelaskan proses otomatisasi injeksi insulin dan monitoring kondisi pasien secara real-time.

Perbedaan Jumlah dan Jenis Diagram UML

Jumlah dan jenis diagram berbeda karena kompleksitas dan sifat sistem yang dikembangkan. Sistem e-commerce lebih fokus pada proses bisnis dan interaksi pengguna yang cukup dapat direpresentasikan dengan beberapa diagram fungsional dan struktural. Sedangkan sistem kesehatan seperti injeksi insulin otomatis mempunyai karakteristik sistem embedded, real-time, dan kritis yang memerlukan representasi lebih mendalam terhadap perilaku sistem, keadaan dinamis, dan arsitektur fisik, sehingga perlu diagram tambahan seperti State Machine dan Deployment Diagram. Semakin kompleks dan kritis suatu sistem, semakin banyak jenis dan jumlah diagram UML yang diperlukan untuk memastikan desain yang akurat dan lengkap.

Menentukan Jumlah dan Jenis Diagram UML yang Diperlukan

Untuk menentukan jumlah dan jenis diagram UML dalam proyek berorientasi objek, perlu mempertimbangkan beberapa aspek:

- Karakteristik Sistem: Apakah sistem bersifat interaktif, otomatis, atau embedded.

- Kompleksitas Sistem: Sistem sederhana biasanya membutuhkan diagram lebih sedikit dibanding sistem kompleks.
 - Tujuan Dokumentasi: Fokus pada diagram yang memberikan nilai tambah dalam memahami kebutuhan, desain, dan implementasi.
 - Stakeholder dan Tim Pengembangan: Diagram harus memfasilitasi komunikasi efektif antar pemangku kepentingan dan pengembang.
- Prinsip utama adalah menggunakan diagram UML yang relevan dan cukup untuk menjelaskan sistem tanpa dokumentasi berlebihan. Misalnya, sistem yang lebih sederhana dapat cukup dengan Use Case, Class, dan Sequence Diagram, sementara sistem kompleks memerlukan tambahan diagram perilaku dan fisik untuk representasi lengkap.

Kesimpulan

Misalnya, untuk "QuickBuy", fokus pada use case, kelas, aktivitas, dan sequence sudah cukup untuk mencakup kebutuhan pengembangan. Sedangkan untuk "MediCare", harus ditambah diagram state machine untuk mengelola status perangkat dan deployment diagram untuk hardware.

Dengan cara ini, tim proyek dapat memilih diagram UML yang sesuai tanpa membuat dokumentasi berlebihan yang justru membingungkan atau tidak berguna.

Demikian penjelasan mengenai diagram UML yang penting untuk kedua sistem serta alasan perbedaan jumlah dan jenis diagram yang dibutuhkan berdasarkan kompleksitas sistem dan prinsip efisiensi dalam rekayasa perangkat lunak berbasis objek. Semoga membantu!

Sumber Referensi: <https://penerbitgoodwood.com/index.php/jisted/article/download/2298/654/11266>

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Rabu, 5 November 2025, 10:42

Jawaban Anda cukup jelas, hanya perlu diperkuat dengan referensi dari universitas terbka

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [052471106 MUHAMAD FUJI SUBEKTI](#) - Selasa, 4 November 2025, 22:17

Nama: Muhamad Fuji Subekti

NIM: 052471106

Ijin Menanggapi

1. Diagram UML apa saja yang paling penting untuk ssetiap sistem?

A. Sistem E-Commerce "QuickBuy"

Berdasarkan Modul 5 BMP MSIM4303, sistem yang bersifat interaktif dan focus pada layanan transaksi umumnya cukup dimodelkan dengan beberapa diagram utama:

- Use Case Dagram

Menjelaskan fungsionalitas sistem dari sudut pandang actor eksternal. Digunakan untuk mengidentifikasi kebutuhan pengguna seperti login, pencarian produk, checkout, dan pelacakan pesanan.

- Class Diagram

Memodelkan struktur sistem secara statis termasuk atribut dan relasi antar kelas seperti produk, pengguna, keranjang, dan pesanan.

- Sequence Dagram

Menjelaskan interaksi antar objek untuk proses seperti transaksi pembelian, proses pembayaran dan penerimaan notifikasi.

- Activity Dagram (jika alur proses cukup kompleks)

Sesuai modul 5, ini digunakan untuk menggambarkan alur kerja procedural seperti proses checkout atau pengembalian barang.

Referensi modul 5 menjelaskan bahwa sistem bersifat transaksi dan client-server sederhana cukup dimodelkan dengan Use Case, dan Sequence Diagram sebagai inti dari desain sistem.

B. Sistem Kesehatan "MediCare"

Sesuai Modul 5, sistem dengan komponen fisik (hardware), proses otomatis, dan tanggung jawab kritis (life-critical sistem) memerlukan diagram yang lebih beragam dan mendalam:

- Use Case Diagram

Untuk mendefinisikan peran pengguna seperti pasien, dokter, dan fungsi utama seperti monitoring kadar gula, penyuntikan otomatis, dan pengiriman alert.

- Class Diagram

Untuk mendeskripsikan objek seperti Sensor, Insulin Controller, Data Pasien, dengan relasi dan logika yang kuat.

- Sequence Diagram

Menjelaskan bagaimana objek sering berinteraksi dalam pengambilan Keputusan otomatis.

- State Machine Diagram

Sangat dianjurkan dalam Modul 5 untuk sistem reaktif atau event-driven. Misalnya: status Standby, Monitoring, Menghitung Dosis, Inject, dan Emergency Alert.

- Activity Diagram

Untuk alur proses seperti pengecekan kadar gula dan validasi dosis insulin.

- Deployment Diagram

Diperlukan untuk menjelaskan arsitektur sistem yang melibatkan komponen perangkat keras dan perangkat lunak.

Modul menyebutkan bahwa sistem tertanam (embedded sistem) atau sistem dengan interaksi kompleks dan keamanan tinggi wajib dimodelkan dengan lebih banyak diagram seperti State Machine, Deployment, dan Activity Diagram.

2. Mengapa Jumlah dan Jenis Diagram UML yang Digunakan dalam Proyek Berbeda?

Menurut BMP MSIM4303 Modul 5:

- Perbedaan kompleksitas sistem menjadi dasar utama. Sistem yang bersifat interaktif (seperti e-commerce) cukup menggunakan diagram fungsional dan structural, sementara sistem yang bersifat otomatis dan kritis (seperti alat Kesehatan) memerlukan diagram perilaku (behavioral) dan fisik (deployment) secara lebih mendalam.

- Modul juga menjelaskan bahwa diagram UML dipilih berdasarkan karakteristik sistem termasuk:

- Sifat sistem (manual, otomatis, embedded)
- Lingkup interaksi antar objek dan perangkat
- Resiko dan konsekuensi dari kesalahan sistem.

Semakin kompleks dan real-time sistem tersebut, semakin banyak jenis diagram UML yang dibutuhkan untuk menggambarkan perilaku dan struktur sistem dengan akurat.

3. Bagaimana Menentukan Jumlah dan Jenis Diagram UML dalam Proyek?

Modul 5 BMP MSIM4303 menyebutkan beberapa prinsip penting:

1. Analisis Kebutuhan

Gunakan Use Case Diagram untuk identifikasi kebutuhan awal pengguna dan sistem.

2. Tujuan Pengembangan

Apakah sistem akan bersifat web, embedded, mobile, atau real-time?

Ini menentukan apakah diperlukan Deployment Diagram, State Machine, dsb.

3. Tingkat Kompleksitas

Sistem sederhana = cukup 3-4 diagram.

Sistem kompleks = bisa 6 diagram atau lebih, termasuk diagram interaksi dan fisik.

4. Iterasi dan Dokumentasi Bertahap

Tidak semua diagram harus dibuat sekaligus. Modul menyarankan penggunaan iteratif dan incremental, disesuaikan dengan kebutuhan tiap fase pengembangan.

5. Efisiensi Dokumentasi

Hindari dokumentasi berlebihan jika tidak memberikan nilai tambah. Fokus pada diagram yang membantu pemahaman tim dan pemangku kepentingan.

Modul menerapkan prinsip "gunakan hanya diagram yang relevan untuk mencapai tujuan pengembangan" yakni efisiensi dan efektivitas desain sistem.

Kesimpulan (Mengacu pada Modul 5)

- Sistem QuickBuy cukup dimodelkan dengan Use Case, Class, dan Sequence Diagram, karena sifatnya transaksional dan client-server standar.
- Sistem MediCare Membutuhkan tambahan State Machine, Deployment, dan Activity Diagram, karena kompleksitasnya yang lebih tinggi dan interaksi dengan perangkat keras.
- Jumlah dan jenis diagram tergantung pada Tingkat kompleksitas sistem, jenis sistem dan Tingkat resiko/kritisnya sistem tersebut.
- Modul 5 menyerankan pendekatan fleksibel dan bertahan dalam pemilihan diagram UML, dengan mengutamakan kebutuhan sistem dan kejelasan dokumentasi.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Rabu, 5 November 2025, 10:43

Sangat baik pemahamannya, tapi jangan lupa tambahkan referensi dan kesimpulan untuk melengkapi analisis Anda.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [054313301 ADISTI MELANI DIVARA MAHARANI](#) - Selasa, 4 November 2025, 22:26

Nama : Adisti Melani Divara Maharani

NIM : 054313301

1. Diagram UML yang Paling Penting untuk Setiap Sistem

A. Sistem E-Commerce "QuickBuy"

a.) Use Case Diagram

- Menjelaskan interaksi antara pengguna (pelanggan, admin, kasir online) dengan sistem.

alasan : Penting untuk menggambarkan fitur utama seperti "login", "pemesanan produk", "pembayaran", dan "pelacakan pesanan".

b.) Class Diagram

- Menjelaskan struktur data dan relasi antar objek seperti *Produk*, *Pengguna*, *Pesanan*, dan *Pembayaran*.

alasan : Penting untuk perancangan basis data dan arsitektur aplikasi.

c.) Sequence Diagram

- Menjelaskan alur proses seperti bagaimana pengguna menambahkan produk ke keranjang dan melakukan pembayaran.

alasan : Menunjukkan urutan interaksi antar objek.

d.) Activity Diagram

- Menjelaskan alur kerja logis suatu proses, misalnya alur pembelian dari memilih produk hingga pembayaran selesai.

alasan : Cocok untuk menggambarkan proses bisnis.

B. Sistem Kesehatan "MediCare"

a.) Use Case Diagram

- Menggambarkan interaksi pengguna (pasien, tenaga medis, sistem monitoring) dengan sistem.

alasan : Untuk mendefinisikan fungsi sistem dari sudut pandang pengguna.

b.) Class Diagram

- Menunjukkan hubungan antar komponen perangkat lunak seperti *Sensor*, *Controller*, *PumpUnit*, *DataLogger*.

alasan : Diperlukan untuk desain sistem berorientasi objek.

c.) State Machine Diagram

- Menjelaskan perubahan status sistem, misalnya kondisi kadar gula naik, normal, atau turun, serta respon sistem (dosis insulin).

alasan : Penting karena sistem bergantung pada perubahan keadaan (state-based).

d.) Sequence Diagram

- Menggambarkan urutan komunikasi antar komponen seperti sensor → kontrol → pompa insulin.

alasan : Memperlihatkan interaksi waktu nyata antar modul.

e.) Component Diagram

- Menjelaskan hubungan antar modul perangkat lunak dan perangkat keras.

alasan : Penting untuk sistem tertanam (embedded system).

f.) Deployment Diagram

- Menunjukkan distribusi perangkat keras (sensor, mikrokontroler, server data) dan hubungan jaringan.

alasan : Penting untuk integrasi antara *hardware* dan *software*.

2. Mengapa Jumlah dan Jenis Diagram UML Berbeda?

- Kompleksitas Sistem : Sistem yang lebih kompleks seperti *MediCare* memiliki lebih banyak komponen, kondisi, dan interaksi yang harus dimodelkan.

- Jenis Sistem (Software vs Embedded) : Sistem e-commerce murni perangkat lunak; sedangkan *MediCare* melibatkan perangkat keras dan sensor, sehingga butuh diagram tambahan (Deployment, State Machine, Component).

- Kritikalitas Sistem : Sistem medis memiliki risiko tinggi terhadap keselamatan pengguna, sehingga desain harus lebih detail dan tervalidasi.

- Kebutuhan Stakeholder : Setiap pihak (pengguna, teknisi, analis, manajer proyek) mungkin membutuhkan pandangan berbeda tentang sistem, sehingga memerlukan diagram yang beragam.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Diperlukan

a. Analisis kebutuhan sistem (Requirement Analysis) = Tentukan ruang lingkup, fungsi, dan batas sistem. Dari sini akan diketahui berapa banyak interaksi dan modul yang harus dimodelkan.

b. Identifikasi tipe sistem = Apakah sistem berbasis web, mobile, real-time, atau embedded? Ini menentukan apakah butuh Deployment Diagram, Component Diagram, atau cukup Use Case saja.

c. Tentukan level kompleksitas = Semakin tinggi kompleksitas (misalnya sistem medis atau otomasi industri), semakin banyak diagram yang harus digunakan untuk mencegah kesalahan desain.

d. Fokus pada tujuan diagram = Gunakan diagram yang benar-benar dibutuhkan:

- *Use Case* → kebutuhan pengguna
- *Class* → struktur data
- *Sequence* → interaksi
- *State Machine* → sistem berbasis keadaan.

e. Pertimbangkan waktu dan sumber daya proyek = Proyek kecil tidak perlu semua 14 jenis UML diagram, cukup yang paling representatif dan efisien.

Sumber Referensi :

- Fadilah, N., & Wahyudi, T. (2020). *Perancangan Diagram UML untuk Sistem Informasi Penjualan Berbasis Web*. Jurnal Teknologi dan Sistem Informasi.
- Kurniawan, A., & Sari, D. P. (2021). *Analisis dan Perancangan Sistem Menggunakan Unified Modeling Language (UML) pada Aplikasi Berbasis Android*. Jurnal Informatika dan Komputer
- Nugroho, A. (2019). *Rekayasa Perangkat Lunak Menggunakan UML dan Java*. Yogyakarta: Andi.
- Pratama, A. B. (2022). *Desain dan Analisis Sistem Berorientasi Objek dengan UML*. Bandung: Informatika.
- Rahmawati, N., & Hidayat, R. (2020). *Analisis Kebutuhan dan Desain User Interface Menggunakan UML pada Sistem Informasi Akademik*. Jurnal Teknologi Informasi dan Komunikasi

**Re: Diskusi.5**

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:17

Sangat baik! Anda menunjukkan pemikiran yang mandiri. Semoga Allah memudahkan perjalanan akademik Anda.

Tautan permanen Tampilkan induk

**Re: Diskusi.5**

oleh [TRIANA PUTRI WAHYUNI 053839459](#) - Selasa, 4 November 2025, 23:51

1. Diagram UML apa saja yang paling penting untuk setiap sistem? Jelaskan alasan pemilihan diagram untuk sistem e-commerce dan sistem kesehatan berbasis injeksi insulin otomatis.

Sistem E-Commerce "QuickBuy"

Sistem ini berfokus pada kegiatan memilih produk, memesan, dan membayar.

Jadi diagram yang dibutuhkan lebih menekankan pada siapa yang memakai sistem dan bagaimana proses pembelian berlangsung.

Diagram yang cocok adalah

1. Use Case Diagram yaitu untuk menunjukkan siapa yang memakai sistem (user, admin) dan apa saja yang bisa mereka lakukan (memilih produk, pesan, bayar).
2. Class Diagram yaitu untuk menggambarkan data seperti produk, keranjang, pesanan, dan pembayaran.
3. Activity Diagram yaitu untuk menunjukkan langkah-langkah proses belanja dari awal sampai selesai.
4. Sequence Diagram yaitu untuk menjelaskan urutan interaksi saat pengguna melakukan pemesanan hingga pembayaran.

Alasannya adalah karena sistem e-commerce lebih banyak berhubungan dengan proses transaksi dan pengelolaan data, jadi diagram yang menunjukkan aliran proses dan data sudah cukup.

Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Sistem ini lebih rumit karena melibatkan sensor, perangkat keras, perhitungan dosis, dan menyangkut keselamatan pengguna.

Diagram yang cocok adalah

1. Use Case Diagram yaitu untuk menjelaskan siapa yang terlibat (pasien, dokter, sistem sensor).
2. Class Diagram yaitu untuk memodelkan bagian-bagian sistem (sensor, alat injeksi, modul kontrol).
3. State Machine Diagram yaitu untuk menunjukkan perubahan kondisi sistem, misalnya, mendeteksi → menghitung → memberi insulin.
4. Sequence Diagram yaitu untuk melihat alur interaksi antara sensor dan alat penyuntik.
5. Deployment Diagram yaitu untuk menggambarkan hubungan perangkat keras (sensor – alat mikrokontroler – pompa insulin).

Alasannya adalah karena sistem ini bersifat otomatis dan kritis, maka perlu diagram yang menjelaskan keadaan sistem dan hubungan antar perangkat secara lebih detail.

2. Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda? Bagaimana kompleksitas sistem memengaruhi jumlah diagram yang dibutuhkan?

Jumlah dan jenis diagram yang digunakan berbeda karena setiap sistem punya tingkat kerumitan yang tidak sama.

Sistem e-commerce seperti "QuickBuy" lebih sederhana, karena hanya mengatur proses belanja, pembayaran, dan pengiriman. Jadi, diagram yang dipakai cukup yang menunjukkan alur belanja dan pengelolaan data. Sedangkan sistem "MediCare" jauh lebih rumit karena melibatkan sensor, alat medis, dan harus bekerja otomatis untuk kesehatan pasien. Sistem ini butuh diagram tambahan untuk menjelaskan bagaimana alat bekerja, bagaimana kondisi berubah, dan bagaimana data dari sensor diproses. Semakin rumit sistemnya, maka semakin banyak diagram yang diperlukan agar cara kerja sistem bisa dipahami dengan jelas.

3. Bagaimana cara menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek pengembangan perangkat lunak?

Jumlah dan jenis diagram UML ditentukan berdasarkan kebutuhan sistem yang akan dibuat. Jika sistemnya sederhana, maka cukup gunakan diagram yang dapat menjelaskan fungsi utama dan alur proses, seperti Use Case, Activity, dan Class Diagram. Tetapi jika sistemnya lebih kompleks atau berisiko tinggi, seperti sistem kesehatan, maka perlu menambahkan diagram yang menggambarkan kondisi sistem, hubungan antar perangkat, dan aliran kerja yang lebih

detail, misalnya State Machine dan Deployment Diagram. Jadi, cara menentukan diagram UML adalah dengan melihat seberapa rumit proses sistem, seberapa banyak komponen yang terlibat, dan seberapa penting ketepatan sistem dalam bekerja. Semakin rumit sistem, semakin banyak diagram yang diperlukan untuk menjelaskan cara kerjanya dengan jelas.

Maka kesimpulan dari semua jawaban diatas adalah Sistem QuickBuy (e-commerce) hanya butuh beberapa diagram karena sistemnya lebih sederhana, seperti diagram untuk menunjukkan siapa yang menggunakan sistem, bagaimana proses belanja terjadi, dan bagaimana data disimpan.

Sedangkan sistem MediCare (injeksi insulin otomatis) butuh lebih banyak diagram karena sistemnya lebih rumit, melibatkan sensor, alat medis, dan harus bekerja sangat tepat demi keselamatan pasien.

Perbedaan jumlah diagram terjadi karena tingkat kerumitan sistem berbeda. Sistem yang lebih rumit membutuhkan penjelasan lebih detail.

Untuk menentukan jumlah diagram, cukup melihat seberapa rumit sistemnya:

Sistem sederhana → diagram sedikit.

Sistem rumit → diagram lebih banyak dan lebih detail.

Referensi

Nugroho, E. (2019). Analisis perancangan sistem informasi menggunakan metode UML (Unified Modeling Language). Journal of Information System, Applied, Management, Accounting and Research, 3(2), 21-29.

Universitas Terbuka.Rekayasa Perangkat Lunak (MSIM430301). Tangerang Selatan: Universitas Terbuka.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Rabu, 5 November 2025, 10:44

Penjelasan Anda tentang sistem MediCare sangat mendalam, menunjukkan analisis kritis. Terus tingkatkan!

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [054518659 DHAFFA NIZHAR ADINDHA PUTRA](#) - Rabu, 5 November 2025, 01:47

1. Diagram UML yang Paling Penting untuk Setiap Sistem

A. Sistem E-Commerce "QuickBuy":

Diagram yang paling penting meliputi:

- Use Case Diagram: Digunakan untuk mensimulasikan interaksi pengguna (pelanggan atau administrator) dengan sistem. Menampilkan operasi utama seperti mencari produk, menambah barang ke keranjang, melakukan pembayaran, dan mengawasi pesanan.
- Class Diagram: digunakan untuk menunjukkan struktur data pengguna, transaksi, produk, dan keranjang belanja.
- Sequence Diagram : digunakan untuk menunjukkan alur transaksi dan interaksi pengguna-sistem selama proses pembelian.

Alasan: Sistem ini berfokus pada data transaksi dan interaksi pengguna, yang membutuhkan visualisasi yang jelas dari alur proses dan struktur data. Class diagram menunjukkan data utama sistem, sedangkan use case dan sequence menunjukkan urutan tindakan dan aktivitas utama.

B.Sistem Kesehatan "MediCare":

Diagram yang paling penting meliputi:

- State machine diagram: dapat digunakan untuk menunjukkan berbagai keadaan perangkat dan transisi antar keadaan, seperti idle, monitoring, dosis, dan alarm.
- Sequence Diagram: Menunjukkan cara sensor, algoritma pemantauan, dan mekanisme pemberian insulin berinteraksi satu sama lain.
- Component Diagram: Digunakan untuk mensimulasikan komunikasi antar komponen, perangkat lunak, sensor, dan perangkat keras.

Alasan adalah bahwa sistem ini sangat kompleks dan terdiri dari banyak kondisi dan transisi (perangkat lunak dan perangkat keras). Sequence dan component diagram membantu menggambarkan interaksi antar komponen dan

sistem, tetapi state machine diperlukan untuk menunjukkan logika alur kerja yang berurutan dan kondisi dinamis.

2. Kenapa jumlah dan jenis diagram UML berbeda? Bagaimana kompleksitas sistem memengaruhi?

Ada sejumlah alasan utama mengapa jumlah dan jenis diagram dari dua sistem di atas berbeda:

- Tingkat kompleksitas sistem: Semakin banyak elemen yang harus dimodelkan—seperti perangkat keras, sensor, real-time, state machine, dan kondisi medis—semakin banyak jenis diagram yang diperlukan untuk menangkap elemen tersebut.
- Kebutuhan stakeholder yang berbeda: Untuk komunikasi antara pengembang dan bisnis, sistem sederhana yang hanya berfokus pada transaksi mungkin hanya membutuhkan diagram utama (use case, class, atau sequence). Sistem yang sangat penting untuk keamanan, seperti injeksi insulin otomatis, harus memodelkan keadaan, interaksi perangkat keras, penempatan fisik, dan keamanan. Akibatnya, pihak yang bertanggung jawab (insinyur perangkat keras, regulator, klinis), membutuhkan artefak yang lebih lengkap.
- Tingkat perubahan dan variabilitas: Agar desain dapat divalidasi, sistem dengan banyak state, banyak interaksi, dan banyak kondisi kesalahan (juga dikenal sebagai "state error") memerlukan diagram yang menggambarkan dinamika (state machine, timing, dan komponen).
- Risiko dan keamanan: Kesalahan dapat berbahaya karena sistem medis sangat penting untuk keamanan. Untuk membuat diagram lebih lengkap, model harus mencakup skenario eksternal, keadaan kesalahan, dan fallback.
- Arsitektur sistem: Jika sistem memiliki banyak deployment, komponen terdistribusi, sensor, dan aktuator, diagram deployment dan komponen menjadi penting. Sistem "sederhana" mungkin hanya berjalan di web/database lokal tanpa banyak perangkat terdistribusi.

Secara ringkas, sistem yang lebih "kompleks" (dari sisi domain, teknologi, interaksi, real-time, keamanan) membutuhkan lebih banyak diagram UML untuk memodelkan berbagai aspeknya (struktur statis, perilaku, interaksi, keadaan, deployment).

3. Menentukan Jumlah dan Jenis Diagram UML yang Relevan

Cara untuk mengidentifikasi diagram yang paling relevan termasuk:

- Identifikasi kebutuhan pengguna dan stakeholder: Fokus pada elemen penting yang ingin dipahami, seperti interaksi pengguna (case use), struktur data (class diagram), alur proses (sequence), atau keadaan sistem (state machine).
- Analisis kompleksitas sistem: Sistem dengan banyak state, perangkat keras, dan proses otomatis membutuhkan diagram yang lebih besar, seperti diagram state machine dan komponen.
- Iterasi dan validasi: Mulai dengan diagram dasar, perbaiki secara bertahap sesuai kebutuhan, dan lakukan evaluasi dengan tim pengembang dan pengguna akhir.
- Tujuan dokumentasi: Pilih diagram yang mudah dipahami, dikomunikasikan, dan digunakan, dan jangan terlalu banyak yang membuatnya terlalu banyak.

Dengan mengikuti prinsip-prinsip ini, tim pengembang dapat memastikan bahwa jumlah dan jenis diagram yang dipilih efektif dan tepat sasaran sesuai dengan tingkat kompleksitas dan kebutuhan proyek. Pemilihan diagram UML sangat dipengaruhi oleh kompleksitas sistem dan fokus penggunaannya, termasuk proses bisnis, struktur data, dan kondisi dan interaksi perangkat keras. Sebuah pendekatan pemilihan diagram bertahap dan analitis akan memastikan dokumentasi yang lengkap dan efektif untuk pengembangan perangkat lunak.

Sumber referensi

- Modul 5 Rekayasa perangkat lunak berorientasi objek, MSIM 4303 Rekayasa Perangkat Lunak.
- [Materi inisiasi 5](#)

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Rabu, 5 November 2025, 10:45

Bagus sekali, Anda sudah memahami perbedaan kompleksitas antara dua sistem. Sukses selalu dalam belajar!

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**oleh [HASAN RAPI 053782613](#) - Rabu, 5 November 2025, 03:02

Assalamualaikum, izin menanggapi diskusinya bapak / ibu tutor.

1. Diagram yang Relevan Untuk Setiap Sistem**Sistem E-Commerce "QuickBuy"****Fokus utama :**

Interaksi pengguna, transaksi, dan pengelolaan data produk

Diagram yang relevan diantara sebagai berikut:

- Use Case Diagram

Menunjukkan hubungan antara aktor (pelanggan , admin sistem pembayaran, kurir) dan fungsi utama (login, memilih produk checkout, pembayaran , pelacakan, pesanan). **Alasan** : sangat penting karena untuk menggambarkan kebutuhan pengguna dan batasan sistem.

- Class Diagram

Menunjukkan struktur data seperti kelas produk, pelanggan , pesanan dan pembayaran. **Alasan** : karena dapat membantu pengembang memahami hubungan antar entitas dan atribut yang akan di terapkan ke database.

- Sequence Diagram

Menggambarkan aliran proses misalnya dari saat pengguna menambahkan produk ke keranjang hingga pembayaran selesai. **Alasan**: Menggambarkan logika bisnis dan urutan interaksi antar objek secara dinamis.

- Activity Diagram

untuk memperlihatkan alur kerja seperti " proses checkout" atau "verifikasi pembayaran". **Alasan** : karena mudah difahami oleh tim non-teknis dan dapat digunakan untuk validasi proses bisnis.

Sistem Kesehatan (MediCare)**Fokus utama :**

otomatisasi, sensor, pengendalian perangkat keras dan keselamatan pasien. untuk diagram yang relevan diantaranya :

- Use Case Diagram

Menunjukkan interaksi antara pasien , tenaga medis dan sistem injeksi otomatis. alasan nya karena menggambarkan kebutuhan utama pengguna dan batas sistem medis

- Class Diagram

Dapat menunjukkan struktur sistem : sensor gula, kontrol insulin , Database pasien, Alarm dan Log data,

Alasan: penting untuk representasi arsitektur sistem dan dependensi antar komponen

- State Machine Diagram

Menggambarkan state perangkat misalnya (*Idle, Monitoring, Injecting, Error/Alert*)

Alasan : penting untuk sistem real-time dan perangkat keras karena perilaku berubah berdasarkan kondisi sensor.

- Sequence Diagram

Menunjukkan alur interaksi antara sensor, kontrol logika dan pompa insulin, Alasan nya karena : membantu memahami urutan aksi sistem saat kadar gula berubah.

- Component Diagram & Deployment Diagram

Menunjukkan bagaimana perangkat keras, modul, perangkat lunak, dan jaringan komunikasi terhubung. Alasannya: wajib untuk sistem yang melibatkan hardware dan integrasi perangkat medis.

2. Mengapa Jumlah dan jenis Diagram Berbeda Antar Sistem ?

E- Commerce (QuickBuy)

- **Kompleksitas**, Relatif sederhana, fokus pada transaksi dan tampilan antarmuka pengguna
- **Tujuan Utama** : Memberikan kemudahan dan kecepatan transaksi bagi pengguna
- **Jenis Interaksi** : Interaksi antar manusia dan sistem web
- **Risiko kesalahan** : Umumnya berdampak pada finansial (misal kesalahan pembayaran)
- **Jumlah Diagram UML** : lebih sedikit sekitar 3-4 diagram utama (Case, Activity dan Class Diagram)

Sistem Kesehatan (MediCare)

- **Kompleksitas** : Sangat kompleks, melibatkan perangkat, sensor dan kontrol otomatis
- **Tujuan Utama** : Menjamin keamanan, ketepatan dosis dan keandalan sistem medis
- **Jenis Interaksi** : Interaksi antar sensor, aktuator, dan pengguna
- **Risiko kesalahan** : Bisa berdampak pada keselamatan pasien (risiko medis tinggi)
- **Jumlah diagram** : lebih banyak (sekitar 5-7 diagram termasuk State Machine, Deployment, dan Sequence Diagram)

3. Cara menentukan Jumlah dan jenis diagram UML yang diperlukan

Untuk menentukan jumlah dan jenis diagram UML, gunakan pendekatan berikut :

1. Analisis kebutuhan sistem

tentukan apakah sistem lebih fokus ke pengguna (UI/UX) proses bisnis atau kontrol perangkat

2. Evaluasi Kompleksitas Teknis

Sistem yang memiliki realtime proses, multikomponen atau keamanan tinggi memerlukan diagram tambahan seperti state machine, atau deployment diagram.

3. Pertimbangkan Audiens pengguna Diagram

untuk stakeholder non-teknis gunakan diagram visual tinggi (use case, activity), dan untuk pengembang gunakan diagram teknis (Class, Sequence, Deployment)

4. Gunakan prinsip As Needed

Tidak semua 14 diagram UML harus digunakan, pilih yang benar-benar membantu memahami dan membangun sistem.

kesimpulannya adalah : perbedaan jumlah diagram muncul karena tingkat kompleksitas, risiko dan interaksi antar komponen sistem berbeda, sedangkan jumlah diagram bergantung kepada kebutuhan analisis, kompleksitas serta siapa yang akan menggunakan diagram tersebut.

Referensi:

- Modul BMP MSIM430301: Modul 5 (Hal 5.16 : UML)
- *Unified Modeling Language (UML) Specification, Version 2.5.1.*
Retrieved from: <https://www.omg.org/spec/UML/2.5.1/> menjelaskan setiap diagram dan kapan sebaiknya digunakan.
- **Shelly, G. B., & Rosenblatt, H. J. (2012).**
Systems Analysis and Design (9th Edition). Course Technology.

Menjelaskan metode memilih diagram sesuai kebutuhan proyek, termasuk sistem e-commerce dan sistem medis.

Hide sidebars

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Rabu, 5 November 2025, 10:45

Analisis Anda logis dan relevan. Tambahkan sedikit penjelasan tentang hubungan antar diagram agar lebih utuh.

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [ELVIRA CITRA PARDENY 051115567](#) - Rabu, 5 November 2025, 13:09

1. Diagram UML yang paling penting untuk setiap sistem

a. Sistem E-Commerce QuickBuy

Untuk sistem e-commerce yang berfokus pada transaksi, interaksi pengguna, dan pengelolaan data produk, diagram yang paling relevan adalah:

1. Use Case Diagram

Menjelaskan interaksi antara pengguna (customer admin sistem pembayaran) dengan sistem, seperti melihat produk menambahkan ke keranjang melakukan pembayaran dan melacak pesanan.

Alasan: mempermudah analisis kebutuhan fungsional sistem dari sisi pengguna.

2. Class Diagram

Menjelaskan struktur kelas seperti Produk, User, Pesanan, Pembayaran dan relasinya.

Alasan: sangat penting untuk perancangan database dan logika program berbasis objek.

3. Sequence Diagram

Menggambarkan urutan interaksi antar objek saat proses tertentu, misalnya saat pelanggan melakukan transaksi.

Alasan: membantu pengembang memahami aliran data dan pesan antar objek.

4. Activity Diagram

Memperlihatkan alur aktivitas pengguna seperti dari proses login hingga pembayaran selesai.

Alasan: memperjelas aliran logika proses bisnis dalam sistem e-commerce.

b. Sistem Kesehatan MediCare (Sistem Injeksi Insulin Otomatis)

Untuk sistem medis yang kompleks, melibatkan perangkat keras, sensor, algoritma, dan keamanan tinggi, diagram berikut lebih dominan:

1. Use Case Diagram

Menjelaskan interaksi antara pengguna (pasien, dokter, sistem sensor) dengan sistem injeksi otomatis.

2. Class Diagram

Menampilkan struktur objek seperti Sensor Gula Darah, Controller, Insulin Pump, User Interface, dan Database.

3. State Machine Diagram

Menggambarkan perubahan keadaan sistem, misalnya dari Idle, Monitoring, Injecting, Alert Mode.

Alasan: sangat penting untuk sistem real-time atau sistem tertanam (embedded system).

4. Sequence Diagram

Menunjukkan urutan komunikasi antara sensor kontroler dan aktuator (pompa insulin).

5. Deployment Diagram

Memvisualisasikan arsitektur fisik sistem mencakup perangkat keras sensor dan koneksi dengan sistem cloud atau mobile.

Alasan: sistem medis berbasis perangkat keras membutuhkan diagram yang menggambarkan keterkaitan antar komponen fisik dan perangkat lunak.

2. Mengapa jumlah dan jenis diagram UML berbeda antara kedua sistem

Jumlah diagram UML berbeda karena kompleksitas dan karakteristik sistem juga berbeda.

1. Sistem E-Commerce relatif sederhana dan berorientasi pada transaksi data dan interaksi pengguna. Fokus utamanya pada alur bisnis, data produk, dan proses pembayaran. Maka, diagram yang dibutuhkan lebih sedikit dan berfokus pada use case, class, dan activity.

2. Sistem Injeksi Insulin Otomatis (MediCare) adalah sistem tertanam (embedded system) yang menggabungkan perangkat keras, perangkat lunak, dan algoritma kontrol. Dibutuhkan lebih banyak diagram karena sistem ini harus menggambarkan keadaan sistem, komunikasi antar sensor, keamanan, dan reliabilitas tinggi.

Dengan kata lain, semakin tinggi kompleksitas, risiko, dan interaksi antar komponen, semakin banyak jenis diagram UML yang diperlukan untuk memastikan rancangan yang lengkap dan akurat.

3. Cara menentukan jumlah dan jenis diagram UML yang relevan

Penentuan jumlah dan jenis diagram UML dalam proyek dilakukan berdasarkan beberapa faktor:

1. Kebutuhan Fungsional dan Non-Fungsional
2. Gunakan Use Case Diagram untuk menurunkan kebutuhan sistem dari sisi pengguna.
3. Kompleksitas Sistem

Jika sistem melibatkan banyak komponen atau perubahan status (seperti sistem medis), tambahkan diagram seperti State Machine dan Deployment.

4. Tahapan SDLC (System Development Life Cycle)

1. analisis - Use Case Diagram
2. Tahap desain - Class, Sequence dan Activity Diagram
3. Tahap implementasi - Component, Deployment Diagram
5. Tujuan Komunikasi Tim

Pilih diagram yang membantu komunikasi antar peran (analyst, programmer, dan stakeholder).

6. Kebutuhan Dokumentasi dan Regulasi

Untuk sistem medis seperti MediCare, dokumentasi visual yang detail diperlukan untuk memenuhi standar keamanan dan audit.

Kesimpulan

1. QuickBuy (E-Commerce): cukup menggunakan Use Case, Class, Sequence, dan Activity Diagram.
2. MediCare (Injeksi Insulin Otomatis): memerlukan tambahan State Machine dan Deployment Diagram untuk memodelkan perilaku sistem dan hubungan perangkat keras-perangkat lunak.
3. Perbedaan jumlah dan jenis diagram dipengaruhi oleh kompleksitas, risiko, dan jenis sistem yang dikembangkan.
4. Penentuan diagram yang relevan harus disesuaikan dengan tahapan pengembangan dan kebutuhan analisis sistem.

Sumber Referensi:

1. Modul 5 Halaman 5.16 BMP MSIM4303 – Rekayasa Perangkat Lunak (Universitas Terbuka)
2. Pressman, R. S. (2019). Software Engineering: A Practitioners Approach.
3. Sommerville, I. (2016). Software Engineering, 10th Edition.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:18

Analisis Anda menunjukkan kedewasaan berpikir. Jadikan ilmu ini jalan menuju keberhasilan dan keberkahan

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [RISKI SEPTIADI MURTI WAHYU 052218499](#) - Rabu, 5 November 2025, 14:23

1. Diagram UML yang terpenting untuk setiap sistem dan alasan memilihnya

A. Sistem E-Commerce "QuickBuy"

Fokus utama sistem ini adalah interaksi pengguna, pengelolaan transaksi, serta pengelolaan data produk. Oleh karena itu, diagram UML yang paling penting adalah:

1) Use Case Diagram

- Menjelaskan fungsi-fungsi utama seperti login, mencari produk, membeli, membayar, dan melacak pesanan.
- Menunjukkan hubungan antara aktor (pembeli, admin, sistem pembayaran) dengan sistem.
- Alasannya: Diagram ini penting untuk memahami kebutuhan fungsional dari sudut pandang pengguna.

2) Class Diagram

- Menampilkan struktur data serta hubungan antar kelas seperti Produk, Pengguna, Pesanan, dan Pembayaran.
- Alasannya: Cocok digunakan untuk merancang basis data dan logika objek dalam sistem e-commerce.

3) Sequence Diagram

- Menggambarkan urutan interaksi antar objek (misalnya antara User Interface, Payment Gateway, dan Database) saat pengguna melakukan transaksi.
- Alasannya: Memudahkan analisis alur proses bisnis seperti proses checkout dan konfirmasi pembayaran.

4) Activity Diagram

- Menggambarkan alur kerja dari suatu proses (misalnya alur pemesanan produk).
- Alasannya: Berguna untuk menjelaskan logika proses yang rumit secara visual.

Kesimpulan:

Sistem e-commerce lebih mengutamakan alur transaksi dan manajemen data, sehingga hanya membutuhkan beberapa diagram utama untuk menampilkan interaksi dan proses bisnis.

B. Sistem Kesehatan "MediCare" (Sistem Injeksi Insulin Otomatis)

Sistem ini bekerja secara real-time, menggunakan sensor, dan memiliki tingkat keamanan serta keselamatan yang tinggi. Untuk itu, dibutuhkan diagram yang menjelaskan keadaan, kontrol perangkat keras, dan interaksi sistem cerdas.

1) Diagram Use Case

- Menerangkan hubungan antara pengguna (pasien, dokter, teknisi) dengan sistem injeksi otomatis.
- Alasannya: Untuk memahami fungsi utama seperti pemantauan kadar gula darah, pemberian insulin, dan pemberitahuan darurat.

2) Diagram Class

- Menunjukkan struktur kelas seperti SensorGula, KendaliInsulin, DataPasien, dan Alarm.
- Alasannya: Merancang hubungan antar komponen sistem yang terintegrasi antara perangkat lunak dan perangkat keras.

3) Diagram State Machine

- Menggambarkan perubahan status sistem, misalnya dari state Idle ke Monitoring, lalu ke Injecting, dan setelah itu ke Alert.
- Alasannya: Penting untuk sistem yang berjalan secara otomatis dan bergantung pada kondisi sensor.

4) Diagram Sequence

- Menggambarkan interaksi antara sensor, modul kontrol, dan aktuator dalam satu siklus kerja.
- Alasannya: Membantu memahami urutan aktivitas sistem secara real-time.

5) Diagram Component

- Menunjukkan cara modul-modul (software dan hardware) saling berinteraksi.
- Alasannya: Relevan karena sistem ini terdiri dari perangkat keras (seperti sensor dan aktuator) dan perangkat lunak kontrol.

6) Diagram Deployment

- a. Menggambarkan distribusi komponen ke dalam node fisik seperti sensor unit, controller unit, dan server cloud monitoring.
- b. Alasannya: Dibutuhkan untuk merancang arsitektur sistem fisik dan integrasi perangkat.

Kesimpulan:

Sistem MediCare membutuhkan banyak diagram karena melibatkan kompleksitas sistem tertanam (embedded), proses real-time, dan keamanan pengguna.

2. Mengapa jumlah dan jenis diagram UML berbeda dalam proyek

Perbedaan jumlah dan jenis diagram UML tergantung pada tingkat kesulitan sistem, tujuan proyek, dan detail yang ingin dicakup.

A. Pemilihan Berdasarkan Tingkat Kesulitan Sistem

- 1) Sistem yang sederhana (QuickBuy): Fokus pada proses bisnis dan cara pengguna berinteraksi cukup dengan beberapa diagram seperti Use Case, Class, dan Sequence.
- 2) Sistem yang rumit (MediCare): Sistem ini melibatkan perangkat keras, algoritma, keamanan, dan keselamatan membutuhkan diagram tambahan seperti State Machine dan Deployment.

B. Kebutuhan dan Risiko Proyek

- 1) Jika sistem berisiko tinggi terhadap keselamatan manusia (seperti MediCare), maka perlu dokumen dan model UML yang lebih detail.
- 2) Jika risiko bisnis rendah (seperti QuickBuy), maka dokumen bisa lebih sederhana.

C. Faktor Tim dan Tahapan Proyek

- 1) Dalam proyek besar, diagram UML digunakan di berbagai tahap: analisis kebutuhan, desain sistem, hingga coding.
- 2) Semakin banyak tim dan subsistem yang terlibat, semakin banyak diagram yang dibutuhkan untuk memudahkan komunikasi antar tim.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Tepat

Langkah-langkahnya adalah sebagai berikut:

1) Analisis Kebutuhan Sistem (Analisis Kebutuhan Perangkat Lunak)

- a. Tentukan kebutuhan fungsional dan non-fungsional.
- b. Gunakan diagram Use Case untuk menampilkan apa yang diinginkan pengguna dari sistem.

2) Tentukan Tingkat Kompleksitas Sistem

- a. Jika sistem memiliki alur yang sederhana, cukup gunakan diagram dasar seperti Use Case, Class, atau Activity.
- b. Jika sistem melibatkan pengendalian real-time atau berbagai komponen, tambahkan diagram State Machine, Component, dan Deployment.

3) Pertimbangkan Tujuan Penggunaan Diagram

- a. Jika tujuannya untuk menjelaskan kebutuhan pengguna, gunakan diagram Use Case.
- b. Jika tujuannya untuk menampilkan struktur sistem, gunakan diagram Class.
- c. Jika tujuannya untuk menunjukkan interaksi antar objek, gunakan diagram Sequence.
- d. Jika tujuannya untuk menampilkan proses atau alur kerja sistem, gunakan diagram Activity atau State.
- e. Jika tujuannya untuk menunjukkan arsitektur teknis, gunakan diagram Component dan Deployment.

4) Konsultasikan dengan Tim Proyek dan Pemangku Kepentingan

- a. Jumlah dan jenis diagram yang dibutuhkan juga tergantung pada persyaratan komunikasi antar anggota tim pengembang, analis, dan pemilik produk.

Sumber:

1. Universitas Terbuka. (2020). BMP STSI4202 – Rekayasa Perangkat Lunak. Modul 3 & 4: Analisis dan Perancangan Sistem Berorientasi Objek dengan UML.

2. Pressman, R. S. (2015). Software Engineering: A Practitioner's Approach. McGraw-Hill.
3. Sommerville, I. (2016). Software Engineering (10th Edition). Pearson.

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:18

Jawaban Anda menunjukkan kedisiplinan berpikir. Semoga terus semangat menimba ilmu dengan niat baik.

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [RATIH NIKEN PRATIWI RAMADHANI 053714635](#) - Rabu, 5 November 2025, 15:04

Izinkan saya menjawab diskusi 5 ini, jika ada kesalahan mohon dikoreksi. Terima Kasih

💡 Diagram UML Paling Penting untuk Setiap Sistem

Pemilihan diagram UML seharusnya didasarkan pada kebutuhan untuk merepresentasikan struktur dan perilaku sistem dari berbagai perspektif.

I. Sistem E-Commerce "QuickBuy"

Sistem ini berorientasi pada bisnis dan menekankan interaksi pengguna, proses transaksi, dan pengelolaan data.

- Use Case Diagram

alasan memilih: Kewajiban. Untuk memodelkan batasan sistem (scope) dan interaksi utama antara pengguna (contohnya, Pelanggan, Admin) dan fitur-fitur sistem (contohnya, Melakukan Pemesanan, Mengelola Inventaris).

- Class Diagram

Alasan Memilih: Kewajiban. Untuk memodelkan struktur data statis dan hubungan antar objek (contohnya, Produk, Keranjang Belanja, Pengguna, Pesanan). Ini sangat penting untuk desain basis data dan pemrograman.

- Sequence Diagram

Alasan Memilih: Sangat Penting. Untuk memodelkan urutan pesan yang rinci dan waktu (temporal) dalam skenario kunci, seperti proses Melakukan Pembayaran atau Checkout.

- Activity Diagram

Alasan Memilih: Sangat Penting. Untuk memodelkan alur kerja (workflow) yang melibatkan pengambilan keputusan dan percabangan, misalnya, langkah-langkah dalam proses Pemrosesan Pesanan dari tahap pembayaran hingga pengiriman.

II. Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Sistem ini beroperasi secara langsung, terintegrasi, dan sangat penting, melibatkan perangkat keras, sensor, dan logika kontrol yang rumit, serta memerlukan tingkat keamanan dan keandalan yang tinggi.

- Use Case Diagram

Alasan Memilih: Wajib. Untuk menggambarkan interaksi antara pengguna (Pasien, Dokter) dan fungsi sistem secara umum (seperti, Memantau Gula Darah, Menghitung Dosis Insulin).

- Class Diagram

Alasan Memilih: Wajib. Untuk mempresentasikan struktur data, mencakup objek seperti SensorGlukosa, PompaInsulin, AlgoritmaDosis, dan RekamMedisPasien.

- State Machine Diagram

Alasan Memilih: Sangat Penting/Wajib. Untuk merepresentasikan perilaku waktu nyata dan perubahan kondisi yang kompleks dari sistem pengendalian. Contoh: Peralihan antara status Memantau, Siaga, MenghitungDosis, dan MenyuntikkanInsulin. Ini krusial karena sifat perilaku sistem yang tertanam sangat kompleks.

- Deployment Diagram

Alasan Memilih: Sangat Penting. Untuk menggambarkan pengaturan perangkat keras (Sensor, Pompa, Unit Kontrol) dan cara komponen perangkat lunak disebar pada alat tersebut.

- Activity Diagram

Alasan Memilih: Sangat Penting. Untuk menggambarkan proses dalam algoritma perhitungan dosis insulin dan kalibrasi sensor.

- Sequence Diagram

Alasan Memilih: Sangat Penting. Untuk menggambarkan interaksi secara berurutan antara sensor, unit kontrol, dan

aktuator (pompa), terutama selama peristiwa penting seperti deteksi hipoglikemia.

Mengapa Jumlah dan Jenis Diagram Berbeda?

Perbedaan dalam jumlah dan jenis diagram yang diperlukan antar kedua sistem ini disebabkan oleh beberapa faktor utama: Kompleksitas Domain, Kebutuhan Non-Fungsional, dan Arsitektur Sistem.

I. Domain dan Karakteristik Sistem

- QuickBuy (E-Commerce):

- Fokus: Interaksi antara pengguna dan komputer (GUI), transaksi bisnis, serta pengelolaan data (CRUD).

- Kompleksitas: Sebagian besar berfokus pada antarmuka pengguna dan logika bisnis (contohnya, promosi, pajak, pengiriman).

- Diagram Tambahan: Lebih mengedepankan diagram Interaksi (Sequence, Activity) dan Struktur (Class).

- MediCare (Injeksi Insulin Otomatis):

- Fokus: Pengendalian secara real-time, sistem tertanam (embedded), serta integrasi antara perangkat keras dan perangkat lunak.

- Kompleksitas: Sebagian besar terletak pada reaksi real-time yang responsif, algoritma pengendalian, keandalan (tidak boleh ada kegagalan), serta keamanan fungsional (kesalahan dapat berakibat fatal).

- Diagram Tambahan: Memerlukan diagram Perilaku Tingkat Lanjut (State Machine) dan diagram Implementasi (Deployment, Component) untuk memodelkan interaksi yang kompleks antara komponen perangkat lunak dan perangkat keras.

II. Kebutuhan Non-Fungsional

- QuickBuy: Fokus utama adalah Kinerja (kecepatan pemuatan), Skalabilitas, dan Kemudahan Penggunaan.

- MediCare: Kebutuhan utama meliputi Keandalan (Reliability), Keamanan (Safety dan Security), Ketepatan Waktu (Timeliness), serta Ketersediaan (Availability). Kebutuhan ini lebih mendetail dan membutuhkan pemodelan alur kerja untuk kegagalan, transisi kondisi, serta desain perangkat keras yang lebih rinci.

Cara Menentukan Jumlah dan Jenis Diagram UML yang Relevan

Menentukan jumlah dan variasi diagram yang sesuai merupakan gabungan antara seni dan ilmu dalam pengembangan perangkat lunak, dan harus sesuai dengan tujuan proyek serta prinsip dokumentasi yang cukup.

Langkah-Langkah untuk Penentuan:

I. Analisis Kebutuhan Awal (Sistem Analisis)

- Identifikasi Stakeholder dan Tujuan: Tentukan pihak-pihak yang harus memahami model serta tujuan utama dari pemodelan (contohnya, komunikasi antar tim, spesifikasi kode, atau dokumentasi arsitektur).

- Klasifikasi Sistem: Tentukan karakteristik sistem (Transaksi/Bisnis, Real-Time/Embedded, Data-Intensive, dan lain-lain).

- QuickBuy → Bisnis dan Transaksi.

- MediCare → Real-Time dan Embedded.

- Identifikasi Kebutuhan Non-Fungsional yang Sangat Penting: Keamanan, Keandalan, dan Kinerja. (MediCare membutuhkan diagram yang memodelkan aspek keamanan dan keandalan, seperti State Machine Diagram).

II. Pilihlah Diagram untuk Berbagai Sudut Pandang Pemodelan

Pilihlah gambaran yang menggambarkan keempat sudut pandang kunci dari suatu sistem, dengan mengacu pada Prinsip "4+1 View Model" sebagai acuan (dengan beberapa penyesuaian):

Sudut Pandang Pertanyaan yang Dijawab Diagram UML yang Relevan

Fungsional/Kasus Penggunaan Apa yang dilakukan sistem untuk pengguna? Use Case Diagram

Struktur/Logis Apa komponen statis sistem dan datanya? Class Diagram, Component Diagram

Perilaku/Proses Bagaimana objek berinteraksi (urutan) dan bagaimana alur kerjanya? Sequence Diagram, Activity Diagram

Perilaku/Keadaan Bagaimana perilaku sistem berubah seiring waktu (transisi kondisi)? (Kritis untuk real-time) State Machine Diagram

Implementasi/Fisik Di mana dan bagaimana sistem diterapkan pada perangkat keras? Deployment Diagram

III. Terapkan Prinsip "Diagram Minimum Efektif"

- Mulailah dengan yang Penting: Selalu gunakan Use Case Diagram (untuk menentukan ruang lingkup) dan Class Diagram (untuk menunjukkan struktur data).

- Tambahkan Berdasarkan Tingkat Kesulitan:

□ Jika sistem memiliki proses yang rumit: Sertakan Activity Diagram.

□ Jika sistem memiliki alur pesan yang rumit antara objek: Sertakan Sequence Diagram.

□ Jika sistem beroperasi berdasarkan status atau kondisi (seperti MediCare): Sertakan State Machine Diagram.

□ Jika sistem melibatkan hardware dan tampilan distribusi (seperti MediCare): Sertakan Deployment Diagram.

- Berhenti Saat Tidak Ada Manfaat: Hanya tambahkan diagram tambahan jika diagram tersebut:

□ Mengatasi Ketidakjelasan: Menjelaskan bagian dari sistem yang tidak dapat diperjelas dengan diagram lain atau deskripsi tertulis.

□ Mendukung Komunikasi: Membantu tim pengembang dan pemangku kepentingan untuk mencapai kesepakatan mengenai desain.

Kesimpulan:

MediCare memerlukan lebih banyak diagram (terutama State Machine dan Deployment) karena tingginya permintaan akan keandalan, keamanan fungsi, dan sifat real-time yang terkait dengan perangkat keras. QuickBuy dapat dibatasi pada diagram yang merepresentasikan logika bisnis dan interaksi transaksi karena fokus utamanya adalah pada aplikasi lapisan atas.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:19

Penjelasan Anda menarik dan mudah dipahami. Semoga semangat ini tidak pernah padam.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [ILHAN RAHARJA NALANKO 051475472](#) - Rabu, 5 November 2025, 16:22

Izin menjawab diskusi kali ini bapak Tutor

1. Diagram UML yang Paling Penting untuk Setiap Sistem

Dalam pengembangan sistem berbasis objek, diagram UML digunakan untuk menggambarkan struktur dan perilaku sistem sehingga mudah dipahami oleh analis, perancang, dan pengembang. Pemilihan diagram bergantung pada kebutuhan dan kompleksitas sistem.

Pada sistem e-commerce “QuickBuy”, fokus utama adalah interaksi pengguna, proses transaksi, dan pengelolaan produk. Diagram UML yang paling penting mencakup **Use Case Diagram** untuk memetakan hubungan antara pengguna, admin, dan sistem pembayaran dengan fungsi seperti pemesanan dan pelacakan pesanan. **Class Diagram** menggambarkan struktur data utama seperti Produk, Pesanan, dan Pembayaran. **Sequence Diagram** menunjukkan urutan interaksi antara pengguna dan sistem selama transaksi, sedangkan **Activity Diagram** menjelaskan alur aktivitas pengguna dalam proses belanja. Keempat diagram ini cukup mewakili sistem yang bersifat bisnis dan transaksional.

Sementara pada sistem kesehatan “MediCare”, kompleksitasnya lebih tinggi karena melibatkan perangkat keras, sensor, dan kontrol otomatis. Diagram yang relevan meliputi **Use Case**, **Class**, dan **Sequence Diagram**, serta tambahan **State Machine Diagram** untuk menggambarkan perubahan status sistem berdasarkan kadar gula darah. **Component** dan **Deployment Diagram** juga digunakan untuk menunjukkan hubungan antar modul perangkat lunak dan perangkat keras. Diagram ini penting guna menjamin keandalan dan keamanan sistem medis.

2. Mengapa Jumlah dan Jenis Diagram Berbeda

Jumlah dan jenis diagram UML yang digunakan dalam proyek pengembangan perangkat lunak berbeda-beda karena dipengaruhi oleh tingkat kompleksitas dan karakteristik sistem. bahwa sistem yang kompleks membutuhkan dokumentasi dan model yang lebih mendetail agar dapat dianalisis, diuji, serta diimplementasikan dengan aman dan

efisien.

Sistem e-commerce seperti "QuickBuy" memiliki struktur proses yang relatif sederhana karena berfokus pada aktivitas bisnis seperti transaksi, pengelolaan data produk, dan interaksi pengguna. Oleh karena itu, sistem ini hanya memerlukan beberapa diagram utama yang menggambarkan perilaku pengguna serta struktur data internal. Diagram seperti *Use Case*, *Class*, *Sequence*, dan *Activity* sudah mampu merepresentasikan kebutuhan utama sistem.

Sebaliknya, sistem kesehatan otomatis seperti "MediCare" memerlukan lebih banyak diagram karena melibatkan kontrol real-time, perangkat keras, sensor, serta algoritma pengambilan keputusan yang berkaitan dengan keselamatan pasien. Sistem semacam ini tidak hanya membutuhkan diagram perilaku, tetapi juga diagram status dan komponen yang menggambarkan hubungan antar perangkat dan software. Dengan demikian, semakin kompleks interaksi antar elemen sistem, semakin banyak pula diagram UML yang dibutuhkan untuk memastikan sistem dapat berjalan dengan akurat, aman, dan terukur.

3. Menentukan Jumlah dan Jenis Diagram UML yang Diperlukan

Penentuan jumlah dan jenis **diagram UML (Unified Modeling Language)** dalam proyek pengembangan perangkat lunak ditentukan oleh **tujuan analisis, kompleksitas sistem, dan kebutuhan pemangku kepentingan**. UML berfungsi sebagai alat bantu visual untuk menggambarkan struktur, perilaku, serta interaksi antar komponen sistem agar mudah dipahami dan dianalisis oleh tim pengembang.

Langkah pertama dalam menentukan diagram yang diperlukan adalah **menganalisis ruang lingkup dan kebutuhan sistem**. Sistem yang kompleks seperti *MediCare* memerlukan lebih banyak diagram untuk menjelaskan perilaku sistem, interaksi sensor, serta kontrol otomatis, misalnya **State Machine Diagram**, **Sequence Diagram**, dan **Component Diagram**. Diagram tersebut penting karena sistem medis menuntut keandalan, keamanan, dan ketepatan fungsi yang tinggi.

Sementara itu, sistem e-commerce seperti *QuickBuy* lebih fokus pada proses bisnis, transaksi, dan interaksi pengguna, sehingga cukup menggunakan **Use Case Diagram**, **Class Diagram**, dan **Activity Diagram** untuk menggambarkan alur transaksi dan pengelolaan data.

Sekian Dari pendapat saya jika ada kekurangan nya mohon di berikan tambahan nya Bapak/ibu Tutor Terimakasih.

Sumber refrensi :

Universitas Terbuka. (2021). *Modul STSI4202 – Rekayasa Perangkat Lunak*. Tangerang Selatan: Universitas Terbuka. Modul 5 Rekayasa Perangkat Lunak Berorientasi Objek KB 2

<https://www.dicoding.com/blog/apa-itu-uml>

<https://binus.ac.id/bekasi/2024/10/jenis-jenis-diagram-dalam-unified-modeling-language-uml/>

<https://nevacloud.com/blog/apa-itu-uml/>

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:19

Analisis Anda sudah bagus, semoga menjadi kebiasaan untuk berpikir sistematis dalam setiap pekerjaan

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [053984087 ANIS LISTYADI](#) - Rabu, 5 November 2025, 19:21

1. Diagram UML yang Paling Penting untuk Setiap Sistem
 - a. Sistem E-Commerce "QuickBuy"

Diagram yang paling relevan:

- Use Case Diagram: Menggambarkan interaksi antara aktor (pelanggan, admin) dan sistem seperti login, pencarian produk, checkout.
- Class Diagram: Menjelaskan struktur data seperti Produk, Pengguna, Pesanan, dan relasi antar kelas.
- Sequence Diagram: Menunjukkan alur interaksi objek saat transaksi atau proses pembayaran.
- Activity Diagram: Menggambarkan alur proses bisnis seperti pemesanan atau pengembalian barang.

Alasan: Sistem ini berfokus pada interaksi pengguna dan pengelolaan data, sehingga diagram yang menekankan alur proses dan struktur data sudah cukup.

b. Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Diagram yang paling relevan:

- State Machine Diagram: Menggambarkan perubahan status perangkat (sensor aktif, injeksi, alarm) berdasarkan kondisi pasien.
- Component Diagram: Menjelaskan modul perangkat lunak seperti pemantauan glukosa, kontrol injeksi, dan enkripsi data.
- Deployment Diagram: Menunjukkan distribusi perangkat lunak pada perangkat keras seperti sensor, server, dan aplikasi mobile.
- Sequence Diagram: Untuk memodelkan interaksi antara sensor, sistem pemantauan, dan pengguna.
- Activity Diagram: Menggambarkan alur kerja pemantauan dan respons otomatis

Alasan: Sistem ini kompleks, melibatkan perangkat keras, algoritma real-time, dan keamanan tinggi, sehingga membutuhkan representasi visual yang lebih detail.

2. Mengapa Jumlah dan Jenis Diagram UML Berbeda?

- Kompleksitas sistem menentukan kebutuhan visualisasi. Sistem sederhana cukup dengan 3–4 diagram, sedangkan sistem kompleks bisa memerlukan 7–10 diagram.
- Sistem dengan banyak komponen, interaksi real-time, dan kebutuhan keamanan tinggi memerlukan diagram tambahan seperti Deployment dan State Machine Diagram.
- Tujuan utama adalah memastikan semua aspek fungsional, struktural, dan dinamis sistem terwakili dengan jelas.

3. Cara Menentukan Diagram UML yang Paling Relevan

1. Analisis kebutuhan fungsional dan non-fungsional: Tentukan fitur utama, interaksi pengguna, dan teknologi yang digunakan.
2. Identifikasi aktor dan skenario utama: Gunakan Use Case Diagram sebagai titik awal.
3. Evaluasi arsitektur sistem: Jika ada perangkat keras atau komponen terdistribusi, gunakan Deployment dan Component Diagram.
4. Tinjau alur kerja dan status sistem: Gunakan Activity dan State Machine Diagram untuk sistem dinamis atau berbasis sensor.
5. Pertimbangkan stakeholder: Diagram harus membantu tim pengembang, analis, dan pemangku kepentingan memahami sistem.

Sumber :

Modul STSI4202

<https://news.sundaxploit.id/jenis-diagram-uml-serta-contoh-dan-penjelasan/>

<https://fajarbaskoro.blogspot.com/2024/11/studi-kasus-uml-dan-e-commerce-web.html>

https://www.lucidchart.com/pages/examples/uml_diagram_tool

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:20

Pemahaman konsep UML Anda mengesankan. Teruslah menulis dan berpikir kritis — ilmu tumbuh dari kebiasaan

Tautan permanen Tampilkan induk

**Re: Diskusi.5**oleh [I NYOMAN ARYA YUDIHARTA 052254338](#) - Kamis, 6 November 2025, 09:10

Ijin Menanggapi Diskusi:

Pemilihan Diagram UML yang Tepat untuk Sistem E-Commerce dan Sistem Kesehatan

Analisis perbedaan kebutuhan diagram UML pada dua sistem yang berbeda kompleksitas, yaitu Sistem E-Commerce "QuickBuy" dan Sistem Kesehatan "MediCare".

1. Diagram UML yang Paling Penting untuk Masing-Masing Sistem**a. Sistem E-Commerce "QuickBuy"**

Pada sistem e-commerce, fokus utama adalah interaksi pengguna, transaksi online, dan pengelolaan data produk serta pesanan. Oleh karena itu, diagram UML yang paling relevan adalah:

1. **Use Case Diagram** – untuk menggambarkan hubungan antara aktor (pelanggan, admin, kurir, dan sistem pembayaran) serta fungsionalitas sistem seperti registrasi, login, pembelian, dan pelacakan pesanan.
 2. **Class Diagram** – untuk menunjukkan struktur data dan hubungan antar kelas, seperti kelas Produk, Pelanggan, Pesanan, dan Pembayaran.
 3. **Sequence Diagram** – untuk menggambarkan urutan interaksi antara pengguna, antarmuka, dan sistem backend saat melakukan transaksi pembelian.
 4. **Activity Diagram** – untuk menjelaskan alur proses bisnis seperti proses checkout dan konfirmasi pembayaran.
- Diagram tersebut sudah cukup mewakili sistem e-commerce karena fokusnya pada alur transaksi dan interaksi pengguna, bukan pada kontrol perangkat keras atau sensor.

b. Sistem Kesehatan "MediCare"

Berbeda dengan e-commerce, sistem ini memiliki tingkat kompleksitas dan risiko yang jauh lebih tinggi, karena berkaitan dengan alat medis, sensor, algoritma pemantauan, serta keamanan data pasien.

Diagram UML yang penting untuk sistem seperti ini antara lain:

1. **Use Case Diagram** – untuk menggambarkan interaksi antara pasien, dokter, dan sistem otomatis.
 2. **Class Diagram** – untuk memodelkan struktur data seperti Sensor, Kontrol Dosis, Pasien, dan Riwayat Kesehatan.
 3. **Sequence Diagram** – untuk memperlihatkan bagaimana sensor mengirim data ke modul analisis dan sistem memberikan dosis insulin.
 4. **State Machine Diagram** – untuk menunjukkan perubahan status sistem, misalnya: Idle → Monitoring → Injecting → Alert. Ini sangat penting dalam sistem real-time berbasis perangkat medis.
 5. **Component Diagram dan Deployment Diagram** – untuk memodelkan hubungan antara perangkat keras (sensor, pompa insulin) dan perangkat lunak yang mengendalikannya.
- Diagram tambahan seperti State Machine dan Deployment menjadi penting karena sistem ini melibatkan interaksi antara software dan hardware, serta memiliki skenario yang harus dikontrol dengan ketat demi keselamatan pasien.

2. Mengapa Jumlah dan Jenis Diagram UML Berbeda?

Perbedaan jumlah dan jenis diagram ditentukan oleh kompleksitas sistem, risiko yang ditangani, dan tingkat detail yang dibutuhkan.

- Pada sistem e-commerce, proses bisnis relatif terstruktur dan berulang, sehingga cukup menggunakan beberapa diagram inti.
 - Sebaliknya, sistem medis seperti "MediCare" bersifat real-time, berbasis sensor, dan memerlukan keandalan tinggi. Oleh karena itu, dibutuhkan lebih banyak diagram untuk memastikan setiap kondisi sistem, aliran data, dan tanggapan terhadap sensor telah dimodelkan secara tepat.
- Dengan kata lain, semakin tinggi kompleksitas dan risiko sistem, semakin banyak pula diagram UML yang dibutuhkan untuk menggambarkan semua aspek desain secara menyeluruh.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Relevan

Penentuan jumlah dan jenis diagram UML dalam proyek pengembangan perangkat lunak dilakukan dengan mempertimbangkan beberapa hal, yaitu:

1. Tujuan proyek dan ruang lingkup sistem – Diagram dibuat hanya untuk menjelaskan aspek yang benar-benar relevan dengan kebutuhan pengguna dan tim pengembang.
2. Kompleksitas sistem – Sistem dengan integrasi perangkat keras atau keamanan tinggi memerlukan diagram tambahan seperti State Machine dan Deployment.
3. Tingkat kebutuhan dokumentasi dan komunikasi – Semakin besar tim dan sistem, semakin banyak diagram yang dibutuhkan agar komunikasi desain lebih jelas.

4. Metodologi pengembangan – Dalam Agile development, biasanya hanya digunakan diagram penting (Use Case, Class, Sequence), sementara dalam Waterfall atau proyek bersertifikasi (misalnya di bidang medis), dokumentasi lebih lengkap diperlukan.

Dengan pendekatan ini, System Analyst dapat menyeimbangkan antara efisiensi dokumentasi dan kelengkapan pemodelan.

Kesimpulan

Perbedaan jumlah dan jenis diagram UML sangat bergantung pada kompleksitas, risiko, dan karakteristik sistem yang dikembangkan. Sistem e-commerce cukup dengan diagram utama yang fokus pada transaksi dan interaksi pengguna, sedangkan sistem kesehatan membutuhkan lebih banyak diagram untuk menjamin keamanan, reliabilitas, dan integrasi antara perangkat keras dan perangkat lunak.

Pendekatan yang tepat dalam memilih diagram akan membantu tim proyek memahami sistem secara menyeluruh dan mengurangi risiko kesalahan desain.

Sumber Referensi:

Sukamto, R. A. (2021). Rekayasa perangkat lunak. Universitas Terbuka.

Jogiyanto, H. M. (2008). Analisis dan Desain Sistem Informasi: Pendekatan Terstruktur Teori dan Praktek Aplikasi Bisnis. Yogyakarta: Andi Offset.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd Edition). Addison-Wesley.

Pressman, R. S. (2019). Software Engineering: A Practitioner's Approach (8th Edition). McGraw-Hill.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Kamis, 6 November 2025, 11:26

Analisis Anda sudah bagus, semoga menjadi kebiasaan untuk berpikir sistematis dalam setiap pekerjaan.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [053883097 SYAHRUL HASANUDIN](#) - Kamis, 6 November 2025, 14:50

Nama : Syahrul Hasanudin

NIM : 053883097

UPBJJ : UT Jakarta Timur

Jawaban No. 1

1. Sistem E-Commerce "QuickBuy" (Fokus pada Interaksi dan Transaksi)

Sistem E-Commerce bersifat data-driven dan user-centric. Diagram yang paling penting adalah yang fokus pada fungsionalitas dan struktur data.

Diagram UNML yang tepat untuk sistem E-Commerce ini adalah Use Case Diagram. Alasan pemilihannya adalah karena menetapkan batasan sistem dan memetakan semua fungsi utama (checkout, login, product search) dari sudut pandang pengguna (Pelanggan, Admin, Staf Gudang).

2. Sistem Kesehatan "MediCare" (Fokus pada Real-Time, State, dan Perangkat Keras)

Sistem ini bersifat mission-critical, memiliki interaksi real-time dengan sensor, dan bergantung pada perubahan kondisi (State).

Diagram UNML yang tepat untuk sistem kesehatan Medicare ini adalah State Machine Diagram. Karena model diagram ini paling penting untuk memodelkan real-time logic sistem. Menggambarkan transisi antar status berdasarkan pemicu dari sensor.

Jawaban No. 2

Jumlah dan jenis diagram UML yang digunakan sangat berbeda karena tingkat kompleksitas, risiko, dan jenis masalah yang harus dipecahkan oleh masing-masing sistem.

1. Perbedaan Fokus Sistem:

- QuickBuy (E-Commerce): Lebih fokus kepada interaksi pengguna, business logic, dan pengelolaan data. Dan diagram yang lebih dibutuhkan adalah Diagram Struktural (Class, Component) dan Perilaku (Use Case, Activity, Sequence).
- MediCare: Lebih fokus pada kepatuhan waktu (timeliness), keamanan, keandalan, state sistem real-time, dan integrasi hardware-software. Dan diagram yang lebih dibutuhkan adalah Diagram State Machine, Deployment, dan model Keamanan (meskipun bukan UML, wajib ada analisisnya).

2. Pengaruh Kompleksitas pada Jumlah Diagram:

Kompleksitas tinggi selalu membutuhkan jumlah diagram yang lebih banyak dan lebih detail karena:

- Peningkatan Risiko: Dalam sistem medis, kesalahan dapat berakibat fatal. Setiap state dan transisi harus didokumentasikan dengan Diagram State Machine untuk mengurangi risiko.
- Integrasi Teknologi: Sistem MediCare menggabungkan hardware, firmware, dan software. Diagram Deployment dan Component diperlukan untuk menjelaskan bagaimana unit software dipetakan ke unit hardware yang berbeda.
- Kebutuhan Stakeholder: Dokumentasi untuk sistem kritis harus melayani lebih banyak stakeholder (misalnya, tim regulasi medis, tim firmware, tim safety), sehingga membutuhkan diagram yang menjelaskan berbagai perspektif sistem.

Jawaban No. 3

Berikut ini cara menentukan jumlah dan jenis diagram yang paling relevan dalam proyek pengembangan rekayasa perangkat lunak:

1. Menentukan Kebutuhan (Needs) dan Risiko (Risks)

- Identifikasi Kebutuhan Komunikasi: Diagram adalah alat komunikasi. Tentukan siapa yang perlu memahami informasi tersebut.
- Identifikasi Area Risiko: Diagram harus fokus pada bagian sistem yang paling berisiko atau paling kompleks.

2. Prinsip "Just Enough" Modeling

Jangan membuat diagram hanya karena diagram itu ada. Terapkan prinsip "Just Enough" Modeling—gunakan jumlah diagram minimum yang diperlukan untuk:

- Mengomunikasikan desain kepada tim dan stakeholder lainnya.
- Memverifikasi bahwa persyaratan telah dipahami dengan benar.
- Mengevaluasi desain sebelum coding dimulai.

3. Pendekatan Berbasis Empat Pilar Utama

Dalam banyak proyek berorientasi objek, diagram dapat dipilih berdasarkan empat pilar desain utama:

- Interaktif (Dynamic): Tujuannya untuk menentukan cara bagaimana komponen berinteraksi (berdasarkan waktu).
- Fungsional (Use Case): Menentukan apa yang harus dilakukan sistem.
- Struktural (Static): Untuk memilih komponen apa saja yang menyusun sistem.
- Alur Kerja (Process): Menentukan bagaimana cara status sistem atau proses bisnis berubah.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:13

Tulisan Anda sudah rapi, tapi untuk memenuhi kaidah akademik, sebaiknya sertakan referensi dan kesimpulan.

[Tautan permanen](#) [Tampilkan induk](#)



Diskusi.5

oleh [MIA AULIA 051618649](#) - Kamis, 6 November 2025, 19:56

Assalamualaikum selamat malam pak, Izin melampirkan jawaban diskusi 5

Diagram UML apa saja yang paling penting untuk setiap sistem? Jelaskan alasan pemilihan diagram untuk sistem e-commerce dan sistem kesehatan berbasis injeksi insulin otomatis.

SISTEM E-COMMERCE "QUICKBUY"

- Use Case Diagram

Use Case Diagram ini adalah fondasi untuk memastikan kita semua—baik tim teknis maupun pihak bisnis—sepaham. Diagram ini memetakan semua pemain kunci (Pembeli, Penjual, Admin) dan interaksi utama mereka dengan sistem, seperti memilih produk atau melakukan pembayaran. Dengan begitu, kita bisa melihat alur dan cakupan sistem secara keseluruhan dalam satu tampilan yang jelas.

- Class Diagram

Class Diagram berperan sebagai backbone atau tulang punggung sistem. Ini adalah model data inti yang nantinya akan langsung menjadi acuan untuk pembuatan kode dan struktur database. Tantangan di QuickBuy adalah pada relasi data yang kompleks, dan diagram inilah yang menjabarkannya. Didefinisikan entitas bisnis utama seperti User, Product, Order dan yang terpenting, hubungan di antaranya. Contohnya, relasi di mana satu Order memiliki banyak OrderItem, adalah hal mendasar yang memastikan desain database tepat dan konsisten.

- Activity Diagram

Activity Diagram adalah alat yang digunakan untuk menyepakati dan mendokumentasikan alur kerja sistem. Dengan diagram ini, bisa menjabarkan proses seperti checkout atau pelacakan pengiriman menjadi langkah-langkah yang terperinci dan logis. Tujuannya adalah untuk memastikan tidak ada celah dalam logika proses dan memudahkan semua pihak, baik bisnis maupun teknis, untuk memahami alur tersebut secara menyeluruh.

- Sequence Diagram

Sequence Diagram untuk memetakan secara rinci bagaimana komponen-komponen sistem saling berbicara dalam sebuah alur. Transaksi e-commerce seperti checkout adalah proses multi-langkah yang rumit, yang melibatkan koordinasi banyak layanan secara berurutan. Dengan memvisualisasikan skenario seperti 'Checkout Berhasil', kami dapat memastikan bahwa interaksi antara UI, OrderService, PaymentGateway, dan lainnya, sudah dirancang dengan benar. Ini adalah alat kunci untuk mengidentifikasi titik kegagalan potensial dalam alur yang paling kritis bagi bisnis.

SISTEM KESEHATAN "MEDICARE"

- Use Case Diagram

Use Case Diagram memberikan gambaran tingkat tinggi tentang fungsi sistem dan interaksinya dengan pengguna utama, yaitu Pasien dan Dokter. Diagram ini memastikan kami memiliki pemahaman yang sama tentang apa yang harus dibangun dan untuk siapa fitur-fitur tersebut ditujukan.

- State Machine Diagram

Diagram yang paling kritis dalam pengembangan MediCare karena berkaitan langsung dengan keselamatan pasien. Sistem ini sangat dinamis dan harus selalu berada dalam 'kondisi' yang benar. Diagram ini memetakan setiap kondisi kritis mulai dari menyala, kalibrasi, hingga menyuntik dan keadaan darurat serta menetapkan secara tepat bagaimana sistem merespons suatu peristiwa, seperti pembacaan sensor tiba atau error terdeteksi. Tujuannya adalah untuk menjamin bahwa sistem secara fisik tidak mungkin dapat masuk ke kondisi yang membahayakan pasien

- Sequence Diagram

Sistem ini berfungsi untuk mensimulasikan dan menspesifikasikan interaksi real-time yang bergantung pada ketepatan waktu antar komponen. Karena ketepatan waktu adalah faktor penentu, diagram ini memetakan skenario kritis seperti 'Satu Siklus Monitoring & Injeksi' dengan urutan pesan yang sangat ketat antara Sensor Glukosa, Kontroler (Algoritma), Pompa Insulin, dan Logger. Tujuannya adalah untuk secara proaktif mengidentifikasi dan menghilangkan kemungkinan race condition atau deadlock dalam operasi yang sangat bergantung pada waktu, sehingga memastikan keandalan dan keamanan proses inti.

- Component Diagram

Mendefinisikan arsitektur modular sistem untuk memenuhi persyaratan keamanan dan keandalan yang ketat. Diagram ini menguraikan komponen-komponen fungsional (SensorInterface, DosisCalculationEngine, PumpController, AlertSystem) dan yang terpenting, menegaskan independensi mutlak dari komponen SafetyMonitor. Isolasi ini memastikan bahwa kemampuan untuk mendeteksi kegagalan dan memberi peringatan tetap operasional bahkan ketika komponen lain mengalami malfungsi, sehingga membatasi dampak kegagalan.

- Class Diagram

Memetakan struktur data inti MediCare, termasuk model untuk Pasien, Parameter Medis, dan Perangkat. Diagram ini memastikan semua komponen sistem memiliki pemahaman yang sama dan konsisten tentang data yang dikelola, yang menjadi dasar bagi operasi dan keamanan sistem.

Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda? Bagaimana kompleksitas sistem memengaruhi jumlah diagram yang dibutuhkan?

Pemilihan dan jumlah diagram UML dalam sebuah proyek sangat bergantung pada kompleksitas dan sifat sistem yang dibangun. Sistem yang rumit—seperti perangkat medis otomatis yang melibatkan perangkat keras, respons real-time, dan standar keamanan kritis—umumnya membutuhkan cakupan diagram yang lebih luas untuk memetakan semua aspek teknis secara mendetail. Sebaliknya, untuk sistem yang lebih sederhana dan berfokus pada alur digital seperti e-commerce, beberapa diagram inti seperti Use Case, Activity, Class, dan Sequence Diagram biasanya sudah cukup untuk mendokumentasikan kebutuhan sistem.

Kompleksitas sistem memengaruhi jumlah diagram yang dibutuhkan karena:

- Kompleksitas sistem embedded mengharuskan penggunaan lebih banyak diagram UML karena sistem tersebut perlu memodelkan tidak hanya perilaku dinamisnya (seperti melalui State Machine Diagram) tetapi juga arsitektur fisik dan lunaknya (melalui Component serta Deployment Diagram) secara bersamaan.
- Dokumentasi yang sangat rinci merupakan sebuah keharusan bagi sistem berisiko tinggi, karena hal ini menjadi prasyarat mendasar untuk memastikan keakuratan, menjamin keamanan, dan mendukung pelaksanaan pengujian yang ketat
- Kompleksitas yang ditandai dengan banyaknya aktor, proses bisnis yang rumit, dan interaksi real-time menuntut penggunaan diagram tambahan untuk memetakan secara rinci interaksi antar objek serta alur aktivitas sistem yang kompleks.

Penentuan jenis dan jumlah diagram UML dalam rekayasa perangkat lunak berorientasi objek didasarkan pada:

- Dalam rekayasa perangkat lunak berorientasi objek, jenis dan cakupan diagram UML yang digunakan ditentukan oleh kebutuhan untuk mendokumentasikan serta menyampaikan aspek fungsional dan struktural sistem dengan jelas dan tepat.
- Tujuan dokumentasinya apakah untuk memandu implementasi teknis, memfasilitasi pengujian yang rigor, atau memastikan komunikasi yang efektif dengan seluruh pemangku kepentingan.

- Pemodelan UML yang efektif berpegang pada prinsip efisiensi, dengan sengaja menghindari diagram yang berlebihan dan hanya fokus pada diagram yang benar-benar memberikan nilai tambah bagi proses pengembangan.
- Proses pemodelan UML bersifat iteratif dan adaptif, di mana jenis serta jumlah diagram dapat terus disesuaikan dan ditambahkan seiring dengan evolusi kompleksitas sistem dan penambahan kebutuhan teknis yang teridentifikasi selama siklus pengembangan.

Bagaimana cara menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek pengembangan perangkat lunak?

- Identifikasi Kebutuhan Sistem

Awali dengan menganalisis kebutuhan fungsional dan non-fungsional sistem untuk mengidentifikasi aktor utama, proses bisnis, serta interaksi kunci. Gunakan Use Case Diagram sebagai fondasi dalam memetakan dan mendokumentasikan kebutuhan pengguna secara menyeluruh.

- Tentukan Tingkat Abstraksi dan Fokus Dokumentasi

Pilih tingkat detail dan fokus dokumentasi yang tepat. Gunakan diagram struktur (seperti Class/Component) untuk arsitektur sistem, dan diagram perilaku (seperti Activity/Sequence/State Machine) untuk alur proses. Penyesuaian ini didasarkan pada tujuan utama—apakah untuk koordinasi tim, analisis kebutuhan, desain teknis, atau pengujian

- Sesuaikan dengan Kompleksitas Sistem

Sesuaikan pilihan diagram dengan kompleksitas sistem. Untuk sistem sederhana, gunakan Use Case, Activity, Class, dan Sequence Diagram. Sementara sistem kompleks yang melibatkan kondisi dinamis, perangkat keras, dan interaksi real-time memerlukan tambahan State Machine dan Deployment Diagram

- Gunakan Pendekatan Iteratif

Terapkan pendekatan iteratif dengan membuat diagram paling kritis terlebih dahulu, lalu tingkatkan detailnya dengan diagram tambahan seiring perkembangan desain dan kebutuhan teknis.

- Perhatikan Efisiensi dan Keterbacaan Dokumentasi

Utamakan efisiensi dan kejelasan dokumentasi. Hindari diagram berlebihan yang tidak bernilai, dan pastikan setiap diagram yang dibuat mudah dipahami, konsisten, serta dapat dimengerti oleh seluruh tim dan stakeholder.

Sumber:

BMP MSIM4303 Rekayasa Perangkat Lunak Modul 5

<https://clickup.com/id/blog/238455/contoh-diagram-uml>

<https://binus.ac.id/bekasi/2025/06/merancang-uml-dengan-baik-panduan-praktis-untuk-pemodelan-sistem-yang-efektif/>

https://repository.bsi.ac.id/repo/files/304602/download/12174915_AmeliaPrameswariWijaya.pdf

<https://www.softwareseni.co.id/blog/class-diagram-adalah>

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:14

Analisis Anda sangat baik, sudah mampu membedakan kebutuhan diagram berdasarkan kompleksitas sistem. Pertahankan semangat belajar Anda!

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [RIANDI SOLIHIN 051676705](#) - Kamis, 6 November 2025, 22:34

Izin menjawab diskusi tuton

1. Diagram UML yang Paling Penting untuk Setiap Sistem

Untuk sistem e-commerce "QuickBuy", diagram yang paling relevan adalah Use Case Diagram, Class Diagram, Sequence Diagram, dan Activity Diagram karena fokusnya pada interaksi pengguna, proses transaksi, serta aliran data produk dan pembayaran.

Sedangkan pada sistem kesehatan "MediCare", diagram pentingnya meliputi Use Case Diagram, Class Diagram, State Machine Diagram, Sequence Diagram, dan Deployment Diagram, karena sistem ini bersifat real-time, melibatkan sensor dan perangkat keras, serta membutuhkan penggambaran perubahan status dan kontrol otomatis.

2. Alasan Jumlah Diagram Berbeda

Jumlah dan jenis diagram berbeda karena kompleksitas sistemnya tidak sama. Sistem e-commerce lebih sederhana dan berfokus pada transaksi data, sementara sistem kesehatan lebih kompleks karena menggabungkan perangkat keras, sensor, algoritma, serta keamanan pasien, sehingga membutuhkan lebih banyak diagram untuk memodelkan seluruh perilakunya.

3. Penentuan Jumlah dan Jenis Diagram

Jumlah dan jenis diagram ditentukan oleh kompleksitas sistem, kebutuhan stakeholder, serta tahapan pengembangan. Semakin kompleks sistem dan semakin tinggi kebutuhan detail teknisnya, semakin banyak jenis diagram UML yang perlu dibuat untuk mendukung analisis dan desain yang jelas.

Referensi :

BMP MSIM4303 Rekayasa Perangkat Lunak Modul 5

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:15

Jawaban Anda singkat tapi padat. Lanjutkan semangat belajarnya dan semoga selalu diberi kelancaran!

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [MUHAMAD SATRIA RIZKY 051464226](#) - Kamis, 6 November 2025, 22:36

Izin menanggapi diskusi pa

Muhamad satria rizky

051464226

1. Diagram UML yang paling penting untuk setiap sistem

a. Sistem E-Commerce "QuickBuy"

Diagram yang paling relevan:

- Use Case Diagram → menggambarkan interaksi antara pengguna (customer, admin, sistem pembayaran) dengan sistem.
- Class Diagram → menunjukkan struktur data seperti produk, pesanan, pengguna, dan pembayaran.
- Sequence Diagram → menjelaskan alur proses pembelian hingga konfirmasi pesanan.
- Activity Diagram → memvisualisasikan alur kerja seperti proses checkout dan pengiriman.

Alasan:

Sistem e-commerce berfokus pada transaksi dan interaksi pengguna. Oleh karena itu, diagram perilaku (use case, activity, sequence) dan diagram struktur dasar (class) sudah cukup untuk mendeskripsikan proses bisnis serta aliran data dalam sistem.

b. Sistem Kesehatan “MediCare”

Diagram yang paling relevan:

- Use Case Diagram → menggambarkan peran pasien, dokter, dan sistem otomatis.
- Class Diagram → memodelkan komponen seperti sensor, modul kontrol, dan sistem pemantauan.
- State Machine Diagram → sangat penting untuk menggambarkan perubahan kondisi sistem (misalnya: Normal → High Glucose → Inject → Stable).
- Sequence Diagram → menjelaskan komunikasi antara sensor, pengendali, dan aktuator.
- Deployment Diagram → menunjukkan hubungan antara perangkat keras, sensor, dan perangkat lunak.

Alasan:

Sistem kesehatan bersifat kompleks dan kritis, karena melibatkan perangkat keras, sensor, serta algoritma kontrol dosis insulin. Oleh sebab itu, diperlukan lebih banyak diagram untuk memastikan keamanan, keandalan, dan ketepatan fungsi sistem.

2. Mengapa jumlah dan jenis diagram UML berbeda antar sistem

Perbedaan jumlah diagram disebabkan oleh kompleksitas dan risiko operasional.

- Sistem e-commerce bersifat transaksional dan berbasis web, sehingga fokusnya pada alur pengguna dan pengelolaan data.
- Sistem injeksi insulin otomatis memiliki interaksi waktu nyata (real-time), sensor fisik, dan tuntutan keamanan tinggi, sehingga memerlukan lebih banyak diagram untuk menjelaskan status, interaksi, dan konfigurasi sistem.

Semakin kompleks dan berisiko sistemnya, semakin detail dokumentasi pemodelan UML yang dibutuhkan untuk menggambarkan seluruh perilakunya secara akurat.

3. Cara menentukan jumlah dan jenis diagram UML yang paling relevan

Penentuan jumlah dan jenis diagram dilakukan berdasarkan:

1. Tujuan sistem dan kebutuhan pengguna.
2. Kompleksitas sistem dan teknologi yang digunakan.
3. Tahapan pengembangan perangkat lunak (analisis, desain, implementasi, dan deployment).
4. Kebutuhan dokumentasi dan komunikasi antar tim.

Prinsipnya, diagram yang dibuat harus relevan dan memberikan nilai informasi bagi pengembang serta stakeholder, bukan sekadar banyak.

Referensi:

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner’s Approach (9th ed.). McGraw-Hill.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.
- Materi Pengayaan “Rekayasa Perangkat Lunak Berorientasi Objek”, Universitas Terbuka (2024)

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:16

Sangat baik! Anda sudah mengaitkan jenis diagram dengan kebutuhan fungsional dan nonfungsional sistem

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [BUNGA NUR ROSEANA 050428449](#) - Jumat, 7 November 2025, 00:24

1. Diagram UML Paling Penting untuk Setiap Sistem

Pemilihan diagram bergantung pada apa yang perlu *dikomunikasikan* dan *dimodelkan*. Untuk dua sistem ini, prioritasnya sangat berbeda.

- Sistem Kesehatan "MediCare" (Fokus: Keamanan & Perilaku Real-Time)

Untuk sistem *cyber-physical* yang kritis terhadap keselamatan seperti ini, kita harus memodelkan *perilaku*, *status*, dan *struktur fisik*.

A. State Machine Diagram (Diagram Mesin Status): Ini adalah diagram yang paling penting untuk MediCare.

Sistem ini tidak didorong oleh pengguna, tetapi oleh *kejadian* (events) dan *status* (states).

- **Alasan:** Kita harus secara pasti mendefinisikan apa yang terjadi ketika status berubah. Misalnya: State: Monitoring -> (Event: Gula Darah > 180mg/dL) -> State: CalculatingDose -> State: Injecting -> State: Monitoring. Kita juga harus memodelkan status anomali seperti State: ErrorSensor, State: LowBattery, atau State: ReservoirEmpty. Diagram ini penting untuk membuktikan keamanan dan keandalan sistem.

Component Diagram (Diagram Komponen): Sistem ini terdiri dari *banyak bagian* yang harus bekerja sama.

- **Alasan:** Diagram ini memvisualisasikan arsitektur perangkat lunak dan keras. Kita akan memiliki komponen seperti SensorInterface, DosageAlgorithmEngine, PumpController, dan UserAlertModule. Ini menunjukkan antarmuka yang jelas di antara mereka, yang sangat penting untuk pengembangan dan pengujian modular.

Deployment Diagram (Diagram Penempatan): Diagram ini menunjukkan *di mana* perangkat lunak dijalankan.

- **Alasan:** Untuk menunjukkan bahwa DosageAlgorithmEngine berjalan pada EmbeddedProcessor, yang terhubung secara fisik ke BloodSensor (node) dan InsulinPump (node). Ini memetakan perangkat lunak ke perangkat keras dunia nyata.

Activity Diagram (Diagram Aktivitas): Berguna untuk satu hal spesifik: memodelkan algoritma.

- **Alasan:** Kita dapat menggunakan ini untuk memetakan logika kompleks dari *algoritma perhitungan dosis*. IF-THEN-ELSE (misalnya, jika GulaDarah > X dan Tren = Naik, maka Dosis = Y).

- Sistem E-Commerce "QuickBuy" (Fokus: Interaksi Pengguna & Alur Proses)

Untuk sistem transaksional yang digerakkan oleh pengguna, kita fokus pada *alur kerja*, *interaksi*, dan *data*.

A. Use Case Diagram (Diagram Kasus Penggunaan): Ini adalah diagram awal yang paling penting.

- **Alasan:** Diagram ini mendefinisikan *batasan* dan *ruang lingkup* sistem dari perspektif pengguna (Aktor: Pelanggan, Admin Penjual, Sistem Pembayaran). Ini adalah kontrak antara analis dan pemangku kepentingan tentang *apa* yang akan dibangun (misalnya, Kelola Keranjang, Lakukan Pembayaran, Lacak Pesanan).

Activity Diagram (Diagram Aktivitas): Sangat penting untuk memodelkan proses bisnis.

- **Alasan:** Diagram ini sempurna untuk menggambarkan alur kerja *end-to-end* yang kompleks, seperti proses "Checkout". Ini akan menunjukkan langkah-langkah, keputusan (misalnya, Stok Tersedia?), dan jalur paralel (misalnya, Proses Pembayaran dan Kirim Email Konfirmasi terjadi bersamaan).

Class Diagram (Diagram Kelas): Ini adalah cetak biru (blueprint) untuk database dan kode.

- **Alasan:** Sistem e-commerce pada dasarnya adalah sistem manajemen data. Diagram Kelas akan mendefinisikan entitas inti (Customer, Product, Order, ShoppingCart) dan hubungan di antara mereka (Customer memiliki banyak Order, Order berisi banyak Product).

Sequence Diagram (Diagram Urutan): Berguna untuk memvalidasi skenario spesifik.

- **Alasan:** Saat Anda ingin memahami *bagaimana* objek berkolaborasi untuk menyelesaikan satu Use Case. Misalnya, untuk Lakukan Pembayaran, Sequence Diagram akan menunjukkan pesan yang dikirim dari ShoppingCartService -> PaymentGateway -> OrderService -> Database secara berurutan.

2. Mengapa Kebutuhan Diagram Berbeda?

Perbedaan jumlah dan jenis diagram tidak ditentukan oleh "jumlah fitur", tetapi oleh **jenis kompleksitas** yang dihadapi.

- **Kompleksitas QuickBuy (E-Commerce):** Ini adalah **kompleksitas proses dan data**. Sistem ini memiliki banyak alur kerja pengguna, aturan bisnis (diskon, pajak, pengiriman), dan entitas data yang saling terkait. Oleh karena itu, kita sangat bergantung pada diagram *perilaku* (Activity) dan *struktural* (Class) untuk mengelolanya.
- **Kompleksitas MediCare (Kesehatan):** Ini adalah **kompleksitas real-time dan struktural (fisik)**. Sistem ini memiliki sedikit alur kerja "pengguna" tetapi memiliki *perilaku berbasis status* yang sangat kompleks dan kritis. Kesalahan dalam transisi status (misalnya, memberikan insulin pada waktu yang salah) berakibat fatal.
 - Inilah mengapa **State Machine Diagram** adalah wajib untuk MediCare tetapi hampir tidak relevan untuk QuickBuy.
 - Sebaliknya, **Component** dan **Deployment Diagram** sangat penting untuk MediCare untuk memodelkan interaksi hardware-software, sementara untuk QuickBuy (yang mungkin hanya berupa aplikasi web di server cloud), diagram ini kurang kritis dibandingkan Class Diagram.

Singkatnya, kompleksitas sistem mendikte alat (diagram) yang dibutuhkan untuk berpikir. Menggunakan palu untuk paku dan obeng untuk sekrup. Anda menggunakan Activity Diagram untuk proses dan State Machine Diagram untuk status. MediCare memiliki lebih banyak *jenis* masalah (status, hardware, algoritma) sehingga membutuhkan lebih banyak *jenis* alat.

3. Cara Menentukan Diagram yang Relevan

Berikut adalah proses 3 langkah untuk memutuskan:

1. Identifikasi Audiens (Untuk Siapa Diagram Ini?)

- **Pemangku Kepentingan Bisnis/Klien:** Mereka membutuhkan gambaran besar. Gunakan **Use Case Diagram** (untuk ruang lingkup) dan **Activity Diagram** tingkat tinggi (untuk alur proses bisnis).
- **Insinyur Perangkat Lunak (Developer):** Mereka membutuhkan detail teknis. Gunakan **Class Diagram** (untuk struktur data/database), **Sequence Diagram** (untuk logika interaksi), dan **Component Diagram** (untuk arsitektur).
- **Insinyur QA (Tester):** Mereka perlu tahu cara "mematahkan" sistem. **State Machine Diagram** dan **Activity Diagram** (terutama jalur pengecualian/error) adalah tambang emas untuk skenario pengujian.
- **Insinyur Sistem/DevOps:** Mereka peduli pada infrastruktur. **Deployment Diagram** adalah bahasa mereka.

2. Identifikasi Risiko & Kompleksitas Inti (Apa Bagian Tersulit?)

- Fokuskan upaya pemodelan Anda pada bagian yang paling rumit atau berisiko tinggi. Jangan buang waktu membuat diagram untuk halaman "Tentang Kami" yang sederhana.
- *Jika alur kerjanya rumit* (seperti checkout QuickBuy) -> Buat **Activity Diagram**.
- *Jika struktur datanya rumit* (seperti relasi produk/kategori) -> Buat **Class Diagram**.
- *Jika perilakunya bergantung pada status* (seperti MediCare) -> Buat **State Machine Diagram**.
- *Jika interaksi antar layanan tidak jelas* (seperti proses pembayaran) -> Buat **Sequence Diagram**.

3. Gunakan Pendekatan "Just-in-Time" (Tepat Waktu)

- Dalam metodologi modern (seperti Agile), Anda tidak perlu membuat semua diagram di awal.
- Buat **Use Case Diagram** di awal untuk menyepakati ruang lingkup.

- Saat Anda akan mengerjakan fitur "Checkout", buatlah **Activity Diagram** dan **Class Diagram** yang relevan *tepat sebelum* pengembangan dimulai.
- Jika tim bingung tentang bagaimana **PaymentService** harus berbicara dengan **OrderService**, *berkumpullah* dan buat **Sequence Diagram** di papan tulis untuk menjernihkan kebingungan.

Kesimpulan: Gunakan diagram UML bukan sebagai "dokumen yang harus dipenuhi", tetapi sebagai **alat untuk berpikir, berkomunikasi, dan memecahkan masalah** spesifik pada waktu yang tepat, untuk audiens yang tepat

Sumber Referensi

BMP MSIM4303 REKAYASA PERANGKAT LUNAK

<https://binus.ac.id/bekasi/2024/10/jenis-jenis-diagram-dalam-unified-modeling-language-uml/>

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:17

Anda sudah di jalur yang benar, terus tingkatkan kemampuan berpikir sistematisnya!

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [VEYSTI DICNORYA BERNANTI 052338378](#) - Jumat, 7 November 2025, 10:21

Selamat Pagi

Izin menjawab diskusi

1. Diagram UML yang Paling Penting untuk Setiap Sistem adalah :

a. Sistem E-Commerce "QuickBuy"

Sistem ini berfokus pada interaksi pengguna, transaksi online, dan pengelolaan data produk. Diagram UML yang paling penting meliputi :

1. Use Case Diagram

- Menunjukkan fungsi utama seperti login, memilih produk, checkout, pembayaran, dan pelacakan pesanan.

- Membantu memahami kebutuhan pengguna (customer, admin, kurir).

2. Class Diagram

- Menggambarkan struktur data: User, Produk, Keranjang, Pesanan, Pembayaran

- Penting untuk desain basis data dan hubungan antar entitas

3. Sequence Diagram

- Memperlihatkan alur interaksi antar objek ketika pengguna melakukan transaksi (misalnya proses checkout dan pembayaran).

- Berguna untuk mendesain logika bisnis dan alur sistem.

4. Activity Diagram

- Menggambarkan alur aktivitas dari mulai pemilihan produk hingga pengiriman selesai.

- Membantu analisis proses bisnis.

QuickBuy merupakan sistem berbasis transaksi dan interaksi pengguna, sehingga fokusnya pada alur proses dan struktur data, bukan pada kontrol perangkat keras atau algoritma kompleks.

b. Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Sistem ini kompleks, melibatkan sensor, aktuator, perangkat keras, algoritma kontrol dosis, serta keamanan data pasien. Diagram UML penting antara lain :

1. Use Case Diagram

Menunjukkan aktor seperti pasien, dokter, dan sistem sensor.

Menggambarkan fungsi seperti pemantauan kadar gula, injeksi otomatis, dan notifikasi.

2. Class Diagram

Mendeskripsikan struktur komponen seperti Sensor, Kontroler, Database Medis, Injektor, dan Modul Keamanan.

3. State Machine Diagram

Menggambarkan perubahan keadaan sistem, misalnya: Idle → Mendeteksi Gula Tinggi → Menghitung Dosis → Injeksi → Monitoring.

Sangat penting untuk sistem real-time dan berbasis perangkat keras.

4. Sequence Diagram

Menunjukkan interaksi antara sensor, pengontrol, dan aktuator secara waktu nyata.

5. Component Diagram & Deployment Diagram

Menggambarkan hubungan antar modul dan distribusi perangkat lunak-perangkat keras (misalnya sensor, mikrokontroler, aplikasi mobile).

MediCare membutuhkan lebih banyak diagram karena melibatkan integrasi sistem tertanam (embedded system), keamanan tinggi, dan kontrol otomatis. Sistem ini bersifat real-time, sensitif, dan berisiko tinggi, sehingga dokumentasi UML harus lebih rinci

2. Jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda karena Jumlah dan Jenis diagram tergantung pada kompleksitas dan karakteristik sistem:

Sistem E-Commerce:

Lebih fokus pada user interaction, transaksi, dan database. Diagram yang digunakan cukup sederhana, menekankan pada alur bisnis dan entitas data.

Sistem Kesehatan (MediCare):

Kompleksitas tinggi karena Adanya komponen fisik dan sensorik, Diperlukan reaksi waktu nyata dan keamanan data medis dan Membutuhkan validasi keadaan sistem (state) dan interaksi antar komponen perangkat keras.

Semakin kompleks sistem (terutama sistem embedded, real-time, atau safety-critical), maka semakin banyak diagram UML yang dibutuhkan untuk memastikan akurasi desain, keamanan, dan konsistensi sistem.

3. Cara Menentukan Jumlah dan Jenis Diagram UML yang Diperlukan

Beberapa pendekatan yang digunakan oleh System Analyst :

1. Analisis Kebutuhan Sistem (Requirement Analysis):

- Tentukan ruang lingkup, aktor, dan fungsi sistem.
- Pilih diagram yang membantu memvisualisasikan kebutuhan tersebut

2. Tingkat Kompleksitas Sistem :

- Sistem sederhana: cukup Use Case, Class, dan Sequence Diagram.
- Sistem kompleks (real-time, embedded): tambah State Machine, Deployment, dan Component Diagram

3. Pendekatan Use-Case Driven (seperti RUP :

Mulai dari Use Case Diagram lalu turunkan ke diagram lain yang mendukung implementasi tiap fungsi

4. Pertimbangan Stakeholder :

Diagram untuk komunikasi dengan pengguna (Use Case, Activity)

Diagram untuk tim teknis (Class, Sequence, Deployment)

Gunakan diagram secukupnya untuk menjelaskan sistem dengan jelas dan konsisten — tidak semua 14 jenis UML harus digunakan

Referensi :

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

BMP Rekayasa Perangkat Lunak Universitas Terbuka

**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:17

Cermat sekali! Anda sudah bisa menyesuaikan diagram UML dengan kompleksitas sistem.

[Tautan permanen](#) [Tampilkan induk](#)
**Re: Diskusi.5**oleh [MARTO 055035643](#) - Jumat, 7 November 2025, 14:48

Assalamu'alaikum

Mohon izin untuk memberikan tanggapan diskusi sesi 5

1. - Sistem E-Commerce fokus pada interaksi pengguna, alur kerja transaksi, dan struktur data. Diagram yang paling penting berorientasi pada aspek fungsional dan perilaku sistem dari sudut pandang pengguna.

Diagram yang paling penting untuk sistem E-Commerce adalah menggunakan Use Case Diagram, karena diagram tersebut mendeskripsikan sebuah interaksi antara satu atau lebih aktor dengan sistem informasi yang akan dibuat, fungsi utama seperti Mencari Produk, Membuat Pesanan, dan Melakukan Pembayaran. Diagram berikutnya yang paling penting adalah Activity Diagram, karena Memodelkan alur kerja transaksi atau proses bisnis, seperti proses checkout dan pembayaran. Ini menunjukkan langkah-langkah berurutan dan percabangan keputusan (misalnya, pembayaran berhasil atau gagal). Diagram berikutnya adalah Class Diagram, karena Menggambarkan struktur data dan relasi objek-objek utama (misalnya, Produk, Pelanggan, KeranjangBelanja, Pesanan). Ini penting untuk desain database dan struktur kode. Dan yang terakhir Sequence Diagram, karena Memodelkan interaksi antar objek dalam skenario tertentu, seperti "Menambahkan Produk ke Keranjang".

- Sistem kesehatan "Medicare" Sistem Injeksi Insulin Otomatis berfokus pada pemrosesan real-time, interaksi perangkat keras/sensor, dan perubahan keadaan sistem yang sensitif (kadar gula darah). Diagram yang paling penting digunakan adalah Use Case Diagram, sama seperti quickbuy, yaitu ini mendefinisikan fungsi utama seperti "Memantau Gula Darah, Menginjeksikan Insulin, dan Mengirimkan Peringatan. Kemudian Activity Diagram, Memodelkan alur kerja algoritma pemrosesan data sensor, perhitungan dosis insulin, dan mekanisme injeksi. Selanjutnya Class Diagram, Menggambarkan struktur objek data dan kontrol (misalnya, Sensor, PompaInsulin, DataPembacaan, AlgoritmaDosis).

2. Kenapa setiap proyek berbeda jumlah diagram yang digunakan, karena tergantung dari Domain dan Sifat Sistem yang berlaku atau yang digunakan, tergantung dari metodologi pengembangan yang digunakan, seperti misal Proyek dengan metodologi Agile cenderung menggunakan diagram minimalis dan hanya diagram yang esensial untuk sprint saat ini.

Pengaruh kompleksitas sistem terhadap jumlah diagram, Kompleksitas sistem adalah faktor tunggal yang paling memengaruhi jumlah diagram UML yang dibutuhkan. Semakin kompleks suatu sistem, semakin banyak perspektif yang perlu dimodelkan, sehingga memerlukan lebih banyak jenis diagram.

3. Menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek didasarkan pada prinsip dokumentasi secukupnya dan fokus pada kebutuhan komunikasi spesifik proyek.

Referensi : Buku Modul MSIM4303 REKAYASA PERANGKAT LUNAK Rosa Ariani Sukamto

Terima kasih

[Tautan permanen](#) [Tampilkan induk](#)
**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:18

Jawaban Anda menunjukkan kematangan berpikir dalam perancangan sistem. Lanjutkan perjuangannya!

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**oleh [YUSUP SUPRIATNA 052621317](#) - Jumat, 7 November 2025, 15:08**1. Diagram UML yang penting untuk setiap sistem:**

◦ Sistem E-Commerce "QuickBuy"

- Use Case Diagram → digunakan untuk menggambarkan interaksi antara pengguna (pelanggan, admin, kurir) dengan sistem, seperti proses login, memilih produk, checkout, dan pembayaran.
- Class Diagram → menjelaskan struktur data utama seperti *User*, *Produk*, *Pesanan*, dan *Pembayaran* serta hubungan antar kelas.
- Sequence Diagram → memperlihatkan urutan proses saat pengguna melakukan transaksi, dari pemilihan produk hingga konfirmasi pembayaran.
- Activity Diagram → menggambarkan alur aktivitas sistem seperti proses pembelian atau pelacakan pesanan. Alasan: Sistem e-commerce berfokus pada interaksi pengguna dan aliran data transaksi, sehingga cukup dengan diagram yang menyoroti proses bisnis dan hubungan data.

◦ Sistem Kesehatan "MediCare"

- Use Case Diagram → untuk mendeskripsikan peran pengguna (pasien, dokter, teknisi) serta interaksi dengan sistem otomatis.
- State Machine Diagram → menjelaskan perubahan status sistem, misalnya dari *Idle* → *Monitoring* → *Injecting* → *Alert*, tergantung kondisi gula darah pasien.
- Activity Diagram → menunjukkan alur kerja otomatis dari sensor membaca kadar gula hingga perangkat mengeluarkan dosis insulin.
- Component Diagram → menampilkan hubungan antar komponen seperti sensor, aktuator, modul kontrol, dan perangkat lunak.
- Deployment Diagram → digunakan untuk menggambarkan bagaimana perangkat keras dan perangkat lunak saling terhubung di lingkungan nyata. Alasan: Sistem ini kompleks dan menyangkut keselamatan pasien, sehingga membutuhkan diagram tambahan untuk memodelkan perilaku, status, dan hubungan antar komponen.

2. Perbedaan jumlah dan jenis diagram UML yang digunakan:

Jumlah diagram berbeda karena kompleksitas sistem berbeda.

- Pada sistem e-commerce, proses utamanya bersifat transaksi dan berbasis pengguna, sehingga cukup dengan beberapa diagram yang menjelaskan alur dan data.
- Pada sistem kesehatan, terdapat integrasi sensor, algoritma kontrol otomatis, dan perangkat keras. Setiap komponen memiliki kondisi dan transisi yang harus dipantau, sehingga membutuhkan diagram perilaku dan arsitektur yang lebih detail. Semakin rumit interaksi dan risiko sistem, semakin banyak diagram dibutuhkan untuk memastikan desain aman dan akurat.

3. Cara menentukan jumlah dan jenis diagram UML yang relevan:

- Analisis kebutuhan sistem: tentukan fungsi utama dan risiko setiap komponen.
- Tingkat kompleksitas: semakin banyak modul dan interaksi, semakin detail diagram yang diperlukan.
- Tujuan dokumentasi: pilih diagram yang membantu komunikasi antara pengembang, analis, dan pengguna.
- Prioritas fungsional: gunakan diagram yang paling membantu memahami logika utama sistem.

Kesimpulan:

Setiap sistem memerlukan diagram UML sesuai tingkat kompleksitas dan tujuannya. Sistem sederhana seperti e-commerce cukup dengan diagram interaksi dan struktur data, sedangkan sistem kompleks seperti *MediCare* memerlukan diagram tambahan untuk menggambarkan perilaku, status, serta hubungan antar perangkat keras dan perangkat lunak agar sistem lebih aman, akurat, dan mudah dipahami tim pengembang.

Sumber Referensi :

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

BMP Rekayasa Perangkat Lunak Universitas Terbuka

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:19

Anda sudah memahami inti perbedaan kompleksitas sistem. Semoga terus konsisten belajar!

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [MUHAMMAD DAFFA RIZKY PRASETYA 054818576](#) - Jumat, 7 November 2025, 18:07

NAMA: MUHAMMAD DAFFA RIZKY PRASETYA

NIM: 054818576

Dalam pengembangan sistem berbasis rekayasa perangkat lunak berorientasi objek, penggunaan diagram UML (Unified Modeling Language) menjadi hal penting untuk memodelkan struktur, perilaku, serta interaksi antar komponen sistem. Namun, jumlah dan jenis diagram yang digunakan akan sangat bergantung pada tingkat kompleksitas serta karakteristik sistem yang dikembangkan. Pada sistem e-commerce seperti "QuickBuy", diagram UML yang paling relevan dan esensial antara lain Use Case Diagram, Class Diagram, Sequence Diagram, dan Activity Diagram. Use Case Diagram digunakan untuk menggambarkan interaksi antara aktor seperti pelanggan, admin, dan kurir dengan sistem dalam aktivitas seperti pemesanan produk, pembayaran, dan pelacakan pengiriman. Class Diagram berfungsi untuk memodelkan entitas penting seperti produk, pesanan, dan pengguna, sedangkan Sequence Diagram dan Activity Diagram menjelaskan urutan proses bisnis serta alur logika aktivitas dalam transaksi pembelian. Diagram-diagram tersebut sudah mencukupi karena sistem e-commerce berfokus pada interaksi pengguna dan pengelolaan data transaksi tanpa melibatkan sensor atau perangkat keras.

Berbeda dengan itu, sistem kesehatan "MediCare" memiliki tingkat kompleksitas yang jauh lebih tinggi karena melibatkan perangkat keras, sensor, serta algoritma pengendalian dosis insulin otomatis. Oleh karena itu, diperlukan lebih banyak jenis diagram UML untuk menggambarkan interaksi yang kompleks di dalam sistem. Selain Use Case dan Class Diagram, sistem ini membutuhkan State Machine Diagram untuk menjelaskan perubahan status sistem, misalnya dari kondisi siaga, membaca kadar gula darah, hingga memberikan dosis insulin secara otomatis. Sequence Diagram digunakan untuk memperlihatkan komunikasi antara sensor dan aktuator, sementara Component Diagram dan Deployment Diagram digunakan untuk menjelaskan arsitektur fisik sistem, termasuk hubungan antara sensor, modul kontrol, dan aplikasi dokter. Dengan demikian, sistem MediCare memerlukan lebih banyak diagram untuk menjelaskan perilaku dinamis, integrasi perangkat keras, serta aspek keselamatan pasien.

Perbedaan jumlah dan jenis diagram UML antara kedua sistem tersebut muncul karena kompleksitas dan domain sistem yang berbeda. Sistem e-commerce merupakan sistem transaksional dengan fokus pada interaksi manusia dengan aplikasi digital, sedangkan sistem MediCare merupakan embedded system yang memerlukan pemodelan perilaku sistem real-time. Semakin kompleks interaksi antar komponen dan semakin tinggi risiko operasional sistem, maka semakin banyak diagram UML yang dibutuhkan agar rancangan sistem dapat dipahami dengan jelas oleh seluruh tim pengembang.

Penentuan jumlah dan jenis diagram UML yang digunakan dalam proyek dapat dilakukan dengan mempertimbangkan beberapa aspek penting. Pertama, dilakukan analisis kebutuhan pengguna untuk menentukan Use Case Diagram. Kedua, analisis struktur data dan hubungan antar objek dilakukan untuk memutuskan kebutuhan Class Diagram. Ketiga, analisis perilaku sistem digunakan untuk menentukan perlunya Activity, Sequence, atau State Machine Diagram. Terakhir, jika sistem melibatkan komponen fisik atau integrasi perangkat keras, maka perlu ditambahkan Component dan Deployment Diagram. Prinsip utamanya adalah efisiensi dan relevansi, yaitu menggunakan diagram yang benar-benar dibutuhkan untuk membantu pemahaman dan pengembangan sistem.

Secara keseluruhan, sistem e-commerce "QuickBuy" hanya memerlukan beberapa diagram utama karena fokusnya pada pengelolaan data transaksi dan interaksi pengguna, sedangkan sistem kesehatan "MediCare" membutuhkan lebih banyak diagram untuk menggambarkan perilaku dinamis dan hubungan antar komponen fisik. Jumlah dan jenis diagram UML harus disesuaikan dengan tingkat kompleksitas sistem agar proses analisis, desain, dan implementasi dapat dilakukan secara efektif dan terarah.

[Tautan permanen](#) [Tampilkan induk](#)**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:19

Analisis Anda menarik, tetapi belum ada referensi dan kesimpulan yang menegaskan hasil pemikiran Anda

[Tautan permanen](#) [Tampilkan induk](#)**Re: Diskusi.5**oleh [TRI WAHYU WIDIARTONO 052401701](#) - Jumat, 7 November 2025, 22:09

1. Diagram UML yang Paling Penting untuk Setiap Sistem

Sistem E-commerce "QuickBuy"

Sistem QuickBuy merupakan platform e-commerce yang fokus pada pengelolaan data, produk, transaksi, serta interaksi pengguna. Oleh karena itu, jenis diagram UML yang digunakan cenderung menggambarkan alur fungsional dan struktur data, seperti:

a. Use Case Diagram

Use Case Diagram ini berfungsi sebagai jembatan komunikasi antara pengguna dan pengguna sistem agar sistem memenuhi kebutuhan aktual pengguna. Diagram ini digunakan untuk memodelkan kebutuhan fungsional sistem, seperti aktivitas login, mencari produk, menambahkan ke keranjang, melakukan pembayaran, dan melacak pesanan.

b. Activity Diagram

Diagram ini digunakan untuk memperlihatkan alir aktivitas pengguna dari awal hingga akhir proses transaksi. Misalnya, alur dari pemilihan produk → checkout → pembayaran → konfirmasi pesanan. Diagram ini membantu menganalisis urutan langkah dan keputusan logis dalam sistem.

c. Sequence Diagram

Digunakan untuk menunjukkan urutan interaksi antara komponen seperti antarmuka pengguna, server aplikasi, dan basis data. Dalam kasus ini, sequence diagram ini dapat memperlihatkan bagaimana data dikirim dari pengguna ke sistem pembayaran dan kemudian disimpan dalam database transaksi.

d. Class Diagram

Class diagram merupakan inti dalam desain yang berorientasi pada objek karena menunjukkan struktur data sistem dan hubungan antar entitas. Diagram ini menjelaskan struktur dan hubungan antar kelas dalam sistem, seperti User, Produk, Pesanan, dan Pembayaran.

Sistem Kesehatan "MediCare"

Sistem medicare adalah sistem injeksi insulin otomatis yang menggunakan sensor untuk memantau kadar gula darah dan secara otomatis memberikan dosis insulin yang sesuai. Sistem ini memiliki kompleksitas tinggi karena melibatkan perangkat keras, sensor biologis, algoritma kontrol, dan tingkat keamanan tinggi. Oleh karena itu diagram UML yang dibutuhkan adalah:

a. Use Case Diagram

Diagram ini menggambarkan hubungan antara aktor seperti pasien, dokter, sensor glukosa, dan pompa insulin. Use Case ini membantu mengidentifikasi fungsionalitas utama sistem, seperti memantau gula darah, mengatur dosis insulin, dan mengirim data ke aplikasi dokter.

b. Class Diagram

Menunjukkan struktur internal sistem seperti kelas SensorGlukosa, Controller, PompaInsulin, Alarm, dan LogData. Diagram ini digunakan untuk memahami bagaimana komponen perangkat keras dan perangkat lunak saling terhubung.

c. State Machine Diagram

Diagram ini digunakan untuk menggambarkan bagaimana sistem bereaksi terhadap berbagai kejadian secara berurutan. Diagram ini sangat penting untuk sistem MediCare ini, karena sistem harus beroperasi berdasarkan kondisi sensor. Misalnya, status sistem dapat berubah dari Idle → Monitoring → Injecting → Alert.

d. Sequence Diagram

Digunakan untuk menunjukkan interaksi real-time antara sensor, controller, dan aktuator. Diagram ini menjelaskan urutan pesan dan waktu respons sistem yang sangat krusial untuk perangkat medis.

e. Activity Diagram

Digunakan untuk memodelkan alur logika pengambilan keputusan sistem dalam menentukan dosis insulin. Diagram ini juga membantu dalam validasi algoritma pengendalian dosis yang aman.

f. Component dan Deployment Diagram

Digunakan untuk menjelaskan integrasi antara modul perangkat keras dan perangkat lunak. Diagram ini juga menjelaskan bagaimana sistem terdistribusi secara fisik dan logis di lingkungan medis.

2. Perbedaan Jumlah dan Jenis Diagram UML

Perbedaan jumlah dan jenis diagram UML yang digunakan disebabkan oleh tingkat kompleksitas dan risiko dari masing-masing sistem. Sistem QuickBuy memiliki kompleksitas menengah karena berfokus pada proses bisnis digital seperti transaksi dan pengelolaan data produk. Sementara itu, sistem MediCare memiliki kompleksitas tinggi karena melibatkan komponen fisik serta algoritma kontrol otomatis yang harus bekerja secara akurat. Oleh karena itu, sistem medis memerlukan lebih banyak diagram untuk memastikan setiap komponen bekerja secara sinkron dan aman.

Selain kompleksitas, faktor risiko juga memengaruhi jumlah diagram yang dibutuhkan. Sistem QuickBuy memiliki risiko kegagalan yang relatif rendah karena hanya berkaitan dengan transaksi finansial, sedangkan sistem MediCare memiliki risiko yang sangat tinggi karena berkaitan dengan kesehatan manusia. Kesalahan kecil dalam perancangan sistem medis dapat berakibat fatal, sehingga dibutuhkan dokumentasi desain yang lebih lengkap dan mendetail. Selain itu, QuickBuy hanya melibatkan interaksi antara manusia dan sistem digital, sementara MediCare melibatkan interaksi antara manusia, sistem, dan perangkat fisik, sehingga membutuhkan diagram tambahan untuk menjelaskan hubungan tersebut secara menyeluruh.

3. Cara Menentukan Jumlah dan Jenis Diagram UML**a. Analisis Kompleksitas dan Tujuan Sistem**

Semakin kompleks sistem, semakin banyak diagram yang dibutuhkan untuk menggambarkan berbagai aspek dari sistem tersebut, seperti fungsionalitas, struktur, dan perilaku dinamisnya.

b. Pertimbangan Tahapan SDLC

Pada tahap analisis, digunakan Use Case Diagram dan Activity Diagram untuk memahami kebutuhan sistem. Pada tahap desain, digunakan Class Diagram, Sequence Diagram, dan State Machine Diagram. Sedangkan pada tahap implementasi dan deployment, digunakan Component dan Deployment Diagram.

c. Kebutuhan Stakeholder dan Risiko Proyek

Jumlah diagram juga disesuaikan dengan kebutuhan stakeholder. Jika proyek memiliki risiko yang tinggi, dokumentasi dan model yang dihasilkan harus lebih detail.

Referensi

Dicoding Intern. (2021). *Apa itu UML? Beserta Pengertian dan Contohnya*. Diakses pada 7 November 2025, dari <https://www.dicoding.com/blog/apa-itu-uml/>

Rosa Ariani Sukamto. (2021). MSIM4303 - Rekayasa Perangkat Lunak. Modul 5

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:20

Penjelasan Anda sangat komunikatif dan terstruktur. Pertahankan gaya ini!

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [MOCH RISWAN LUTFIN ANFA 051141089](#) - Jumat, 7 November 2025, 22:58

Nama: Moch Riswan lutfin Anfa

Nim: 051141089

Izin menjawab pertanyaan dari diskusi diatas, trimakasih

Analisis Diagram UML untuk Sistem E-Commerce "QuickBuy" dan Sistem Kesehatan "MediCare"

1. Diagram UML Paling Penting untuk Setiap Sistem.

A. Sistem E-Commerce "QuickBuy"

Sistem E-Commerce, meskipun berskala besar, secara fungsional berfokus pada interaksi pengguna, transaksi, dan manajemen data. Tiga diagram utama sudah cukup untuk memberikan cetak biru yang komprehensif:

- Use Case Diagram:

- Alasan: Ini adalah diagram awal dan fundamental. Tujuannya adalah mendefinisikan lingkup dan fungsionalitas sistem dari sudut pandang aktor (Pengguna, Admin, Mitra Kurir). Diagram ini dengan jelas memetakan apa yang harus dilakukan sistem (goal) misalnya, "Melakukan Pembayaran," "Mencari Produk," atau "Mengelola Stok" tanpa masuk ke detail bagaimana proses itu terjadi. Ini penting untuk mendapatkan persetujuan awal dari stakeholders.

- Class Diagram:

- Alasan: Sistem e-commerce adalah tentang data dan strukturnya. Diagram ini berfungsi sebagai cetak biru struktural untuk basis data dan kode program. Ia memodelkan entitas kunci seperti Pelanggan, Produk, Keranjang Belanja, Pesanan, dan Pembayaran, serta hubungan (misalnya, satu pelanggan dapat memiliki banyak pesanan). Diagram ini krusial untuk memastikan integritas dan konsistensi data transaksi.

- Sequence Diagram:

- Alasan: Proses paling kritis dalam e-commerce adalah transaksi checkout dan pembayaran, yang melibatkan banyak objek yang berinteraksi dalam urutan waktu tertentu. Sequence Diagram memvisualisasikan aliran pesan yang spesifik antar objek (misalnya, dari User Interface ke Order Processor ke Payment Gateway). Ini penting untuk mengidentifikasi bottleneck dan memastikan keamanan serta kecepatan transaksi.

B. Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Sistem ini bersifat kritis (life-critical), melibatkan kontrol real-time, dan merupakan sistem tertanam (embedded) yang berinteraksi dengan perangkat keras. Oleh karena itu, kebutuhan diagramnya jauh lebih banyak dan fokus pada perilaku yang sangat detail, keamanan, dan deployment fisik.

- State Machine Diagram (Paling Krusial):

- Alasan: Karena sistem ini event-driven (dipicu oleh perubahan kadar gula darah) dan reaktif, perilaku sistem berubah drastis berdasarkan statusnya saat ini. Diagram ini memodelkan semua status yang mungkin (misalnya, Idle, Memantau, Menghitung Dosis, Menginjeksikan, Status Peringatan Darurat) dan transisi yang memicunya. Dalam sistem yang kritis, memodelkan setiap status secara eksplisit adalah wajib untuk mencegah kegagalan fatal.

- Deployment Diagram:

- Alasan: Sistem ini adalah perpaduan perangkat keras dan perangkat lunak. Diagram ini memetakan topologi fisik sistem, menunjukkan di mana komponen perangkat lunak dijalankan (Node): di Sensor (perangkat keras), di Unit

Kontrol Pompa, dan di Server Komunikasi (jika ada). Ini penting untuk perencanaan integrasi fisik dan pengujian konektivitas.

- Activity Diagram:

- Alasan: Algoritma yang digunakan untuk menghitung dosis insulin adalah kompleks. Activity Diagram digunakan untuk memodelkan alur kerja prosedural dari algoritma ini, termasuk percabangan logika (decision point misalnya, "Apakah Gula Darah di Bawah Batas Aman?"), dan pemrosesan paralel (concurrency misalnya, memantau dan menginjeksikan secara bersamaan).

- Component Diagram:

- Alasan: Digunakan untuk memecah perangkat lunak MediCare menjadi unit logis yang dapat diganti dan dikelola, seperti komponen Sensor Data Manager, Dose Calculation Engine, dan Actuator Controller. Ini penting untuk modularitas dan pengujian unit secara terpisah.

2. Mengapa Jumlah dan Jenis Diagram UML yang Digunakan Berbeda?

Jumlah dan jenis diagram UML berbeda karena perbedaan fundamental dalam kompleksitas dan risiko kedua sistem:

- Sifat Sistem: Sistem QuickBuy adalah sistem informasi/transaksional yang soft, berfokus pada data, di mana kegagalan berdampak finansial. Sistem MediCare adalah sistem embedded dan life-critical yang hard, berfokus pada kontrol fisik dan waktu, di mana kegagalan berdampak pada nyawa.
- Kompleksitas Perilaku: Perilaku QuickBuy sebagian besar sekuensial (langkah demi langkah). Perilaku MediCare adalah reaktif dan concurrent; ia harus merespons perubahan kadar gula darah dalam real-time. Perilaku kompleks ini menuntut penggunaan diagram perilaku khusus seperti State Machine Diagram yang tidak diperlukan oleh sistem e-commerce.
- Kebutuhan Validasi: Karena risiko MediCare jauh lebih tinggi, tim Analis harus melakukan validasi yang lebih ketat dari setiap sudut pandang struktur, perilaku, dan fisik untuk meminimalkan potensi bug yang mematikan. Setiap diagram tambahan berfungsi sebagai alat validasi yang kritis.

3. Bagaimana Cara Menentukan Jumlah dan Jenis Diagram UML yang Paling Relevan?

Penentuan diagram yang relevan mengikuti prinsip "Just Enough UML" dan didasarkan pada analisis risiko dan kompleksitas:

- Mulai dari Inti (Use Case dan Class): Setiap proyek berorientasi objek harus dimulai dengan Use Case Diagram (untuk mendefinisikan apa yang dilakukan sistem) dan Class Diagram (untuk mendefinisikan struktur data dan kode). Ini adalah fondasi.
- Analisis Risiko dan Kritisitas: Jika sistem adalah sistem kritis (MediCare) atau memiliki regulasi yang ketat, tambahkan diagram yang memvalidasi keselamatan, seperti State Machine Diagram (untuk perilaku real-time) dan Deployment Diagram (untuk integrasi fisik).
- Modelkan Bottleneck dan Interaksi Kompleks: Identifikasi di mana interaksi antar objek atau aliran kontrol menjadi sangat rumit (misalnya, checkout di QuickBuy atau algoritma di MediCare). Di sinilah Sequence Diagram dan Activity Diagram ditambahkan untuk memecahkan kompleksitas tersebut.
- Prinsip Nilai Tambah: Diagram hanya dibuat jika ia menambah nilai nyata (misalnya, menyelesaikan ambiguitas, meredakan risiko, atau memfasilitasi komunikasi antar tim). Jika suatu aspek tidak berisiko atau sederhana, jangan buang waktu membuat diagram yang tidak perlu.

Intinya, QuickBuy hanya memerlukan diagram yang fokus pada transaksi dan data, sedangkan MediCare memerlukan spektrum diagram yang lebih luas untuk memodelkan semua aspek yang terikat waktu, fisik, dan kritis terhadap kehidupan.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:21

Jawaban Anda cukup jelas, hanya perlu diperkuat dengan referensi dan ditutup dengan kesimpulan agar lebih akademis.

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**oleh [ULIKTA MUTAWAFINA 051517222](#) - Jumat, 7 November 2025, 23:34**1. Diagram UML yang Paling Penting untuk Setiap Sistem**

Untuk Sistem E-Commerce "QuickBuy", fokus utama pengembangan adalah pada interaksi pengguna, transaksi online, serta pengelolaan data produk dan pesanan. Berdasarkan prinsip perancangan berorientasi objek dalam Modul MSIM430301 Universitas Terbuka (2023), sistem seperti ini memerlukan diagram UML yang menggambarkan hubungan antara aktor dan sistem, alur proses bisnis, serta struktur data yang digunakan.

Diagram UML yang paling penting untuk QuickBuy antara lain:

- Use Case Diagram, untuk menggambarkan fungsi utama sistem dan interaksi dengan aktor seperti pengguna, admin, dan payment gateway.
- Activity Diagram, untuk menunjukkan alur aktivitas seperti proses pemesanan produk, pembayaran, dan pelacakan pengiriman.
- Sequence Diagram, untuk memperlihatkan urutan komunikasi antar komponen seperti antarmuka pengguna, sistem pembayaran, dan database.
- Class Diagram, untuk mendeskripsikan struktur data dan hubungan antar kelas seperti User, Product, Order, dan Payment.
- Deployment Diagram bersifat opsional, digunakan bila diperlukan untuk memvisualisasikan konfigurasi fisik antara web server, database server, dan client.

Dengan demikian, sistem QuickBuy umumnya hanya memerlukan empat hingga lima diagram utama karena kompleksitasnya tergolong sedang dan lebih menekankan pada proses bisnis digital.

Berbeda dengan itu, Sistem Kesehatan "MediCare" memiliki kompleksitas yang jauh lebih tinggi. Sistem ini menggabungkan perangkat keras (sensor kadar gula darah dan pompa insulin), perangkat lunak (algoritma pengendali dosis), serta mekanisme keamanan dan komunikasi data. Oleh karena itu, berdasarkan modul yang sama, sistem seperti MediCare membutuhkan lebih banyak diagram UML agar setiap aspek perilaku dan struktur sistem dapat tergambarkan secara rinci.

Diagram penting untuk MediCare meliputi:

- Use Case Diagram, untuk menunjukkan interaksi antara pasien, dokter, sensor, dan sistem pengendali.
- Class Diagram, untuk menggambarkan hubungan antar entitas seperti Sensor, Controller, Pump, dan DataLogger.
- Sequence Diagram, untuk menjelaskan urutan interaksi antar komponen sistem dari pengukuran kadar gula hingga injeksi insulin otomatis.
- State Machine Diagram, untuk menggambarkan perubahan status sistem seperti keadaan Idle, Monitoring, Injection, hingga Error Handling.
- Activity Diagram, untuk memvisualisasikan alur kerja logis seperti proses deteksi kadar gula, analisis data, dan pemberian dosis.
- Component Diagram dan Deployment Diagram, untuk menjelaskan hubungan antar modul software-hardware serta distribusi sistem secara fisik di perangkat medis, aplikasi pengguna, dan server cloud.

Karena kompleksitas dan tingkat risiko sistem kesehatan jauh lebih tinggi, jumlah diagram UML yang digunakan untuk MediCare biasanya enam hingga tujuh diagram utama, dengan penekanan pada aspek perilaku dan keandalan sistem.

2. Alasan Jumlah Diagram Berbeda antara Sistem QuickBuy dan MediCare

Perbedaan jumlah diagram UML antara kedua sistem dijelaskan dalam Modul MSIM430301 – Rekayasa Perangkat Lunak (Universitas Terbuka, 2023). Jumlah dan jenis diagram ditentukan oleh kompleksitas, tujuan sistem, serta tingkat interaksi antara komponen perangkat keras dan perangkat lunak.

Sistem QuickBuy memiliki alur yang linear dan relatif sederhana. Prosesnya hanya melibatkan pengguna, server, dan database. Risiko kesalahan sistem berdampak pada aspek finansial, bukan keselamatan pengguna. Karena itu, diagram yang digunakan hanya berfokus pada model interaksi dan alur bisnis.

Sebaliknya, sistem MediCare beroperasi secara otomatis dan berinteraksi langsung dengan tubuh manusia melalui sensor dan aktuator. Sistem ini harus menggambarkan status, keputusan, dan reaksi terhadap perubahan kondisi secara real-time. Risiko kegagalan sistem juga sangat tinggi karena dapat berpengaruh pada kesehatan atau keselamatan pasien. Oleh karena itu, dibutuhkan diagram tambahan seperti State Machine Diagram, Component Diagram, dan Deployment Diagram untuk memodelkan seluruh perilaku dan struktur sistem secara menyeluruh.

Dengan kata lain, semakin kompleks dan berisiko tinggi suatu sistem, semakin banyak diagram UML yang diperlukan untuk menjelaskan semua aspek operasionalnya.

3. Cara Menentukan Jumlah Diagram UML yang Diperlukan dalam Proyek RPL Berorientasi Objek
Menurut Modul Universitas Terbuka MSIM430301 (2023) serta panduan dari Pressman (2020) dan Sommerville (2016), jumlah dan jenis diagram UML yang digunakan dalam sebuah proyek tidak ditentukan secara baku, melainkan bergantung pada tujuan analisis, tingkat kompleksitas, serta kedalaman desain yang dibutuhkan.

Langkah pertama adalah melakukan analisis kebutuhan sistem untuk mengidentifikasi fungsi utama dan interaksi pengguna. Dari hasil analisis ini, dibuat Use Case Diagram untuk mendefinisikan batas dan peran sistem. Setelah itu, apabila sistem memiliki proses bisnis yang kompleks, maka digunakan Activity Diagram dan Sequence Diagram untuk menggambarkan alur kerja dan urutan interaksi antar komponen.

Langkah kedua adalah menilai tingkat kompleksitas arsitektur sistem. Jika sistem mencakup banyak kelas atau komponen yang saling berinteraksi, maka Class Diagram dan Component Diagram digunakan untuk menggambarkan struktur dan hubungan antar bagian. Untuk sistem yang bekerja secara otomatis dan bergantung pada kondisi tertentu (seperti alat medis atau sistem industri), diperlukan State Machine Diagram untuk menggambarkan perubahan keadaan sistem secara detail.

Langkah ketiga adalah mempertimbangkan risiko dan tujuan pengembangan. Semakin tinggi risiko kegagalan sistem, semakin rinci diagram UML yang dibutuhkan. Sistem dengan risiko rendah seperti e-commerce cukup menggunakan diagram inti untuk mendukung komunikasi antar tim pengembang. Sementara sistem dengan risiko tinggi seperti MediCare memerlukan pemodelan lengkap agar dapat menjamin keandalan, keamanan, dan validasi sistem.

Kesimpulan

Sistem QuickBuy membutuhkan diagram UML utama seperti Use Case, Activity, Sequence, dan Class Diagram karena fokusnya pada proses transaksi dan interaksi pengguna.

Sementara MediCare memerlukan diagram yang lebih banyak dan mendalam seperti State Machine, Component, dan Deployment Diagram karena melibatkan perangkat keras, algoritma otomatis, serta keamanan tinggi.

Perbedaan jumlah diagram ini disebabkan oleh tingkat kompleksitas, risiko, dan kebutuhan pemodelan sistem.

Sumber Referensi

1. Universitas Terbuka. (2023). Modul MSIM430301 – Rekayasa Perangkat Lunak. Jakarta: Universitas Terbuka.
2. Pressman, R. S. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
3. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
4. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Sabtu, 8 November 2025, 05:22

Anda sudah menunjukkan kemampuan berpikir analitis yang bagus. Terus tingkatkan ya

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [054453816 ACH. RIYANTO](#) - Sabtu, 8 November 2025, 12:11

Nama : Achmad Riyanto
 NIM : 054453816
 MK : Rekayasa Perangkat Lunak
 UPBJJ : Malang

1. Diagram UML paling penting untuk setiap sistem

- Use Case Diagram (Menunjukkan hubungan antara pengguna (pelanggan, admin, penjual, payment gateway) dengan fitur sistem pemesanan, pembayaran, pelacakan) alasan pemilihan untuk memahami kebutuhan fungsional dan alur bisnis.
- Activity Diagram (Menggambarkan alur proses seperti pembelian, checkout, dan pengiriman barang) dipilih karena untuk mendeskripsikan workflow pengguna.
- Sequence Diagram (Menjelaskan interaksi waktu nyata antara pelanggan, sistem, dan payment gateway saat transaksi) berguna untuk merancang komunikasi antar modul.

2. Mengapa jumlah dan jenis diagram UML berbeda antara kedua sistem?

Kompleksitas, distribusi, dan real-time suatu sistem, semakin banyak diagram yang dibutuhkan untuk menangkap seluruh aspek perilaku, struktur, dan interaksi sistem.

- Kompleksitas sistem menentukan banyaknya aspek yang harus dimodelkan.
- QuickBuy bersifat transactional system (data-centric), sedangkan MediCare adalah control system (event-driven dan safety-critical).
- Dalam sistem seperti MediCare, dibutuhkan pemodelan status, interaksi antar komponen, dan penanganan kondisi darurat, sehingga diagram tambahan seperti State Machine dan Deployment diperlukan.

3. Cara menentukan jumlah dan jenis diagram UML yang relevan

- Analisis kebutuhan sistem
 - Tentukan apakah sistem bersifat data-driven, event-driven, atau real-time.
- Identifikasi aktor dan proses utama
 - Jika banyak interaksi pengguna → gunakan Use Case dan Activity Diagram.
- Tentukan aspek yang perlu dimodelkan
 - Struktur data → Class Diagram
 - Perilaku dan status → State Machine Diagram
 - Komunikasi antar komponen → Sequence Diagram
 - Distribusi perangkat → Deployment Diagram
- Pertimbangkan tujuan proyek
 - Fokus pada implementasi sistem kompleks (terintegrasi perangkat keras atau sensor)

Referensi

MATERI PENGAYAAAN MSIM4303 REKAYASA PERANGAKAT LUNAK, SESI 5 Rekayasa Perangkat Lunak Berorientasi Objek.

Universitas Terbuka. (2021). Modul STSI4202 – Rekayasa Perangkat Lunak. Tangerang Selatan: Universitas Terbuka.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Minggu, 9 November 2025, 16:29

Argumentasi Anda kuat. Semoga ilmu yang dipelajari hari ini bermanfaat untuk karier profesional Anda.

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**

oleh [NUGROHO DWI PUTRANTO 051123668](#) - Sabtu, 8 November 2025, 14:44

1. Diagram UML yang Paling Penting Masing-masing Sistem

- Sistem E-Commerce "QuickBuy"

Sistem ini focus pada interaksi pengguna, transaksi, dan pengelolaan data produk, maka diagram UML yang paling relevan adalah:

- Use Case Diagram, yang berfungsi menggambarkan hubungan antara pengguna (customer, admin, sistem pembayaran) dan fungsionalitas sistem. Diagram ini diperlukan untuk menjelaskan fitur utama seperti login, pemesanan, pembayaran, dan pelacakan pesanan.
- Class Diagram, yang berfungsi menunjukkan struktur data dan hubungan antar kelas (Produk, User, Order, Pembayaran). Karena E-commerce berbasis pada pengelolaan objek data, maka diperlukan Class Diagram.
- Sequence Diagram, yang berfungsi menggambarkan alur interaksi antar objek saat pengguna melakukan transaksi. Adanya Sequence Diagram penting untuk menjelaskan alur proses "checkout" atau "payment".
- Activity Diagram, yang berfungsi menunjukkan alur kerja proses bisnis seperti "proses pembelian" atau "proses pengiriman." Activity Diagram berguna untuk memahami logika bisnis dan aliran aktivitas.

- Sistem Kesehatan "MediCare"

Sistem ini lebih kompleks dan bersifat real-time, melibatkan perangkat keras (sensor, aktuator) serta algoritma kontrol otomatis., maka diagram UML yang paling relevan adalah:

- Use Case Diagram, yang berfungsi enunjukkan interaksi antara pasien, dokter, dan sistem (sensor, pompa insulin). Diagram ini diperlukan untuk memahami peran dan fungsi pengguna serta perangkat.
- Class Diagram, yang berfungsi menggambarkan struktur perangkat lunak seperti kelas Sensor, Kontrol Dosis, dan Pompa. Class Diagram diperlukan untuk memodelkan sistem yang berbasis komponen perangkat dan perangkat lunak..
- Sequence Diagram, yang berfungsi menunjukkan interaksi waktu nyata antara sensor, kontrol logika, dan aktuator. Adanya Sequence Diagram diperlukan untuk menjelaskan bagaimana sistem merespons perubahan kadar gula darah.
- State Machine Diagram, yang berfungsi menggambarkan perubahan status system. Adanya diagram ini sangat penting untuk sistem real-time yang bereaksi terhadap kondisi lingkungan.
- Component Diagram, yang berfungsi memetakan arsitektur sistem (sensor, modul kontrol, UI, jaringan). Diagram ini membantu integrasi antara software modules dan hardware components.
- Deployment Diagram, yang berfungsi menunjukkan bagaimana perangkat keras dan perangkat lunak ditempatkan di lapangan. Karena sistem ini memiliki elemen fisik (alat medis), maka Deployment Diagram diperlukan

2. Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda?

Perbedaan Jumlah dan jenis diagram UML yang digunakan dalam masing-masing proyek dipengaruhi oleh beberapa hal/faktor:

- Kompleksitas Fungsional:

Pada sistem E-Commerce kompleksitas fungsionalnya berupa transaksi online, data produk, user interface.

Sedangkan pada sistem Kesehatan: Algoritma medis, sensor real-time, keamanan tinggi.

- Komponen Fisik

Komponen fisik pada sistem E-Commerce murni perangkat lunak. Sedangkan pada sistem Kesehatan: merupakan gabungan hardware dan software.

- Risiko & Keamanan

Resiko Pada sistem E-Commerce Adalah resiko finansial. Sedangkan pada sistem Kesehatan Adalah resiko kesehatan pasien.

- Kebutuhan Validasi

Kebutuhan validasi pada sistem E-Commerce hanya pengujian fungsional standar. Sedangkan pada sistem Kesehatan: Harus memenuhi regulasi medis dan uji keselamatan.

Semakin tinggi kompleksitas dan tingkat risiko sistem, semakin banyak diagram UML yang dibutuhkan untuk menggambarkan perilaku, struktur, dan interaksi sistem secara lengkap.

3. Cara menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek pengembangan

perangkat lunak:

- Analisis Kebutuhan Sistem: Tentukan apakah sistem lebih fokus pada proses bisnis atau interaksi teknis.
- Identifikasi Komponen Utama: Apakah sistem melibatkan hardware, algoritma real-time, atau hanya software
- Tentukan Perspektif Pemodelan: Gunakan diagram struktur (Class, Component) untuk desain arsitektur, dan perilaku (Activity, Sequence, State) untuk interaksi dan dinamika sistem.
- Evaluasi Kompleksitas & Risiko: Semakin kompleks dan berisiko sistemnya, semakin detail dokumentasi yang dibutuhkan.
- Prinsip Efisiensi Dokumentasi: hanya gunakan diagram yang bermanfaat langsung untuk komunikasi antar tim dan pengembangan sistem. Hindari diagram yang tidak relevan.

Sumber referensi:

BMP MSIM4303 - Rekayasa Perangkat Lunak

<https://www.dicoding.com/blog/apa-itu-uml/>

<https://binus.ac.id/bekasi/2024/10/jenis-jenis-diagram-dalam-unified-modeling-language-uml/>

<https://csirt.teknokrat.ac.id/7-jenis-diagram-uml-yang-wajib-kamu-tahu/>

<https://sis.binus.ac.id/2019/05/15/model-model-diagram-uml/>

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Minggu, 9 November 2025, 16:30

Penjelasan Anda rinci dan terstruktur, seperti analis profesional. Pertahankan gaya analisis ini!

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [054208558 SASTRO MANURUNG](#) - Sabtu, 8 November 2025, 19:05

izin menjawab pada diskusi ini

Nama: Sastro Manurung

Nim: 054208558

1. Diagram UML apa saja yang paling penting untuk setiap sistem? Jelaskan alasan pemilihan diagram untuk sistem e-commerce dan sistem kesehatan berbasis injeksi insulin otomatis.

a. Sistem E-Commerce "QuickBuy"

Use Case Diagram:

Alasan: Sangat penting untuk memvisualisasikan interaksi antara pengguna (aktor) dan sistem. Ini membantu memahami fungsionalitas utama seperti "Melihat Produk," "Menambah ke Keranjang," "Melakukan Pembayaran," dan "Melacak Pesanan."

Class Diagram:

Alasan: Memodelkan struktur data dan hubungan antar entitas seperti "Produk," "Pengguna," "Keranjang Belanja," "Pesanan," dan "Pembayaran." Ini membantu dalam perancangan basis data dan implementasi logika bisnis.

Activity Diagram:

Alasan: Menggambarkan alur kerja proses bisnis utama seperti "Proses Pemesanan" atau "Proses Pembayaran." Ini membantu mengidentifikasi langkah-langkah dan keputusan yang terlibat dalam setiap proses.

Sequence Diagram:

Alasan: Menunjukkan interaksi antar objek dalam urutan waktu tertentu. Berguna untuk memvisualisasikan bagaimana objek-objek berkolaborasi untuk menjalankan use case, misalnya, bagaimana pengguna berinteraksi dengan keranjang belanja dan sistem pembayaran.

b. Sistem Kesehatan "MediCare" (Injeksi Insulin Otomatis)

Use Case Diagram:

Alasan: Sama pentingnya dengan sistem e-commerce, tetapi dengan fokus pada interaksi antara pasien, perangkat, dan petugas medis. Contoh use case: "Memantau Kadar Gula Darah," "Memberikan Dosis Insulin," "Mengatur Parameter Perangkat," dan "Memberikan Peringatan Kondisi Darurat."

Class Diagram:

Alasan: Memodelkan struktur data dan hubungan antar entitas seperti "Pasien," "Sensor Gula Darah," "Pompa

Insulin," "Dosis Insulin," "Log Data," dan "Petugas Medis." Struktur ini lebih kompleks dibandingkan sistem e-commerce karena melibatkan perangkat keras dan algoritma.

State Machine Diagram:

Alasan: Sangat penting untuk menggambarkan berbagai keadaan (states) dari sistem dan transisi antar keadaan tersebut. Contoh states: "Normal," "Kadar Gula Tinggi," "Kadar Gula Rendah," "Pemberian Insulin," "Mode Darurat." Diagram ini membantu memastikan sistem bereaksi dengan benar terhadap berbagai kondisi.

Activity Diagram:

Alasan: Menggambarkan alur kerja proses kritis seperti "Proses Pemantauan Gula Darah" dan "Proses Pemberian Insulin." Ini membantu memastikan bahwa algoritma dan perangkat keras bekerja secara sinkron.

Sequence Diagram:

Alasan: Menunjukkan interaksi antar objek dan perangkat dalam urutan waktu. Contoh: bagaimana sensor mengirim data ke pompa insulin, bagaimana pompa memberikan dosis, dan bagaimana sistem memberikan peringatan kepada pasien atau petugas medis.

Component Diagram:

Alasan: Memodelkan komponen perangkat keras dan perangkat lunak yang membentuk sistem, serta ketergantungan antar komponen. Ini membantu dalam perancangan arsitektur sistem yang kompleks.

Deployment Diagram:

Alasan: Menggambarkan bagaimana komponen perangkat keras dan perangkat lunak ditempatkan (deployed) dalam lingkungan fisik. Ini penting karena sistem melibatkan perangkat keras yang berinteraksi langsung dengan tubuh manusia.

2. Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda? Bagaimana kompleksitas sistem memengaruhi jumlah diagram yang dibutuhkan?

Perbedaan Jumlah dan Jenis Diagram:

Fokus Sistem: Sistem e-commerce berfokus pada interaksi pengguna dan pengelolaan data, sementara sistem kesehatan melibatkan perangkat keras, algoritma, dan keamanan tinggi.

Tingkat Kritis: Sistem kesehatan memiliki tingkat kritis yang lebih tinggi karena kesalahan dapat membahayakan nyawa. Ini memerlukan pemodelan yang lebih rinci dan komprehensif.

Kompleksitas Teknis: Sistem kesehatan melibatkan integrasi perangkat keras dan perangkat lunak, algoritma pemantauan, dan mekanisme keamanan yang kompleks.

Pengaruh Kompleksitas Sistem:

Semakin kompleks sistem, semakin banyak diagram yang diperlukan untuk memodelkan berbagai aspek sistem secara rinci.

Diagram tambahan membantu memastikan bahwa semua persyaratan dipahami dengan benar, risiko diidentifikasi, dan solusi dirancang dengan tepat.

3. Bagaimana cara menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek pengembangan perangkat lunak?

Analisis Kebutuhan:

Pahami persyaratan sistem secara mendalam. Identifikasi fungsionalitas utama, interaksi pengguna, dan batasan teknis.

Identifikasi Risiko:

Identifikasi area-area yang berpotensi menimbulkan risiko atau kompleksitas tinggi. Area-area ini memerlukan pemodelan yang lebih rinci.

Pilih Diagram yang Relevan:

Pilih diagram UML yang paling sesuai untuk memodelkan aspek-aspek penting dari sistem. Gunakan Use Case Diagram untuk memahami interaksi pengguna, Class Diagram untuk memodelkan struktur data, State Machine Diagram untuk menggambarkan alur kerja sistem yang kompleks, dan sebagainya.

Prioritaskan Diagram:

Prioritaskan diagram yang paling penting untuk keberhasilan proyek. Fokus pada diagram yang memberikan nilai terbesar dalam memahami dan mengkomunikasikan desain sistem.

Iterasi dan Evaluasi:

Selama proses pengembangan, terus evaluasi apakah diagram yang ada masih relevan dan memadai. Tambahkan diagram baru jika diperlukan, atau hapus diagram yang tidak lagi berguna.

Konsultasi dengan Tim:

Diskusikan dengan tim pengembangan untuk mendapatkan masukan dan perspektif yang berbeda. Pastikan semua anggota tim memahami dan setuju dengan pilihan diagram yang digunakan.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Minggu, 9 November 2025, 16:30

Analisis Anda sudah bagus, namun belum dilengkapi dengan referensi dan kesimpulan. Coba tambahkan agar argumen Anda lebih kuat.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [WENDI BUDIARSO 053647749](#) - Minggu, 9 November 2025, 06:08

Nama : Wendi Budiarmo

NIM : 053647749

Diskusi 5

Pertanyaan 1

a. Sistem E-Commerce "QuickBuy"

- Use Case Diagram : Menunjukkan interaksi antara pengguna dan sistem, serta fungsi utama seperti memilih produk, menambahkan ke keranjang, dan pembayaran.
- Class Diagram : Menggambarkan struktur data dan hubungan antara entitas seperti Produk, Pengguna, dan Keranjang Belanja.
- Sequence Diagram : Menunjukkan urutan interaksi antara objek dalam proses transaksi, dari pemilihan produk hingga pembayaran.
- Activity Diagram: Menggambarkan alur kerja proses pembelian, mulai dari pemilihan produk hingga pelacakan pengiriman.

Alasan : Sistem ini berfokus pada interaksi pengguna dan pengelolaan data, sehingga diagram yang diperlukan lebih sederhana dan berorientasi pada fungsionalitas.

b. Sistem Kesehatan "MediCare"

- Use Case Diagram: Menunjukkan interaksi antara pengguna (pasien, dokter) dan sistem, serta fungsi utama seperti pemantauan kadar gula darah dan pemberian insulin.
- Class Diagram: Menggambarkan struktur data dan hubungan antara entitas seperti Sensor, Dosis Insulin, dan Pengguna.
- State Machine Diagram: Menunjukkan status perangkat dan transisi antar status berdasarkan data sensor (misalnya, kadar gula darah rendah, normal, tinggi).
- Sequence Diagram: Menggambarkan interaksi antara sensor dan perangkat dalam memberikan dosis insulin.
- Activity Diagram: Menggambarkan proses pemantauan dan pemberian insulin secara otomatis.

Alasan : Sistem ini lebih kompleks karena melibatkan perangkat keras dan algoritma, sehingga memerlukan lebih banyak diagram untuk menggambarkan berbagai aspek interaksi dan status sistem.

Pertanyaan 2

a. Mengapa Jumlah dan Jenis Diagram Berbeda?

Jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda karena:

- Kompleksitas Sistem: Sistem yang lebih kompleks, seperti "MediCare", memerlukan lebih banyak representasi untuk menggambarkan interaksi antara komponen yang berbeda (perangkat keras, perangkat lunak, pengguna).
- Fungsionalitas: E-Commerce lebih fokus pada transaksi dan interaksi pengguna, sementara sistem kesehatan membutuhkan pemantauan dan respons otomatis yang lebih dinamis.
- Regulasi dan Keamanan: Sistem kesehatan seringkali memiliki persyaratan keamanan dan regulasi yang lebih ketat, sehingga memerlukan representasi yang lebih rinci.

Kompleksitas sistem sangat memengaruhi jumlah diagram yang dibutuhkan dalam pengembangan perangkat lunak. Berikut adalah beberapa cara kompleksitas memengaruhi kebutuhan diagram UML:

a. Jumlah Komponen

Sistem Sederhana: Jika sistem hanya memiliki beberapa komponen yang saling terhubung secara langsung, jumlah diagram yang diperlukan cenderung lebih sedikit. Misalnya, aplikasi sederhana mungkin hanya memerlukan Use Case dan Class Diagram.

Sistem Kompleks: Pada sistem dengan banyak komponen, interaksi, dan dependensi, lebih banyak diagram diperlukan untuk menggambarkan hubungan dan alur kerja. Ini bisa mencakup diagram tambahan seperti Sequence, Activity, dan State Machine.

b. Interaksi dan Alur Kerja

Sistem dengan Interaksi Terbatas: Jika interaksi pengguna dengan sistem terbatas, diagram yang diperlukan untuk menggambarkan alur mungkin minimal.

Sistem dengan Interaksi Dinamis: Sistem yang melibatkan banyak interaksi, seperti aplikasi real-time atau sistem perangkat medis, memerlukan lebih banyak diagram untuk menunjukkan bagaimana berbagai komponen berinteraksi satu sama lain dalam berbagai kondisi.

c. Kebutuhan Pemantauan dan Respons

Sistem Statik: Sistem yang berfungsi dengan cara yang cukup statis mungkin memerlukan diagram yang lebih sedikit karena tidak ada banyak perubahan status atau alur kerja.

Sistem Dinamis: Sistem yang memerlukan pemantauan status dan respons otomatis (seperti sistem kesehatan) memerlukan lebih banyak diagram, seperti State Machine Diagram, untuk menggambarkan bagaimana sistem beradaptasi dengan perubahan kondisi.

d. Regulasi dan Keamanan

Sistem dengan Persyaratan Rendah: Sistem yang tidak terikat pada regulasi ketat mungkin memerlukan lebih sedikit diagram untuk menggambarkan alur kerja dan keamanan.

Sistem dengan Persyaratan Tinggi: Sistem yang beroperasi dalam domain yang diatur (seperti kesehatan atau keuangan) memerlukan lebih banyak diagram untuk memastikan semua aspek kepatuhan dan keamanan ditangani secara memadai.

e. Kompleksitas Logika Bisnis

Logika Sederhana: Jika logika bisnis dalam sistem relatif sederhana, jumlah diagram yang diperlukan untuk menggambarkan alur dan keputusan akan lebih sedikit.

Logika Rumit: Jika sistem memiliki banyak aturan, kondisi, dan keputusan yang harus diambil, lebih banyak diagram seperti Decision Diagram atau Activity Diagram diperlukan untuk menggambarkan semua kemungkinan skenario. Dengan demikian, semakin kompleks sistem, semakin banyak diagram yang diperlukan untuk memastikan semua aspek sistem dapat dipahami dan dikelola dengan baik. Hal ini membantu dalam merencanakan, mengembangkan, dan memelihara sistem secara efektif.

Pertanyaan 3

Menentukan Jumlah dan Jenis Diagram UML

Untuk menentukan jumlah dan jenis diagram UML yang relevan, pertimbangkan hal-hal berikut:

- Analisis Kebutuhan: Identifikasi kebutuhan fungsional dan non-fungsional sistem. Apa yang harus dilakukan sistem dan bagaimana pengguna akan berinteraksi dengannya?
- Kompleksitas Sistem: Evaluasi kompleksitas sistem berdasarkan jumlah komponen, interaksi, dan kebutuhan keamanan.
- Stakeholder: Pertimbangkan siapa saja yang terlibat (developer, pengguna, manajer) dan informasi apa yang mereka butuhkan untuk memahami sistem.
- Iterasi dan Umpan Balik: Diagram dapat ditambahkan atau diubah selama proses pengembangan berdasarkan umpan balik dari tim dan stakeholder.

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Minggu, 9 November 2025, 16:31

Penjelasan sudah mengarah dengan baik, tapi sebaiknya tambahkan sumber referensi dan kesimpulan singkat di akhir tulisan.

Tautan permanen Tampilkan induk



Re: Diskusi.5

oleh [HAMDI 053837115](#) - Minggu, 9 November 2025, 13:29

1. Diagram UML apa saja yang paling penting untuk setiap sistem? Jelaskan alasan pemilihan diagram untuk sistem e-commerce dan sistem kesehatan berbasis injeksi insulin otomatis.

a. Sistem E-Commerce "QuickBuy"

Diagram UML yang paling penting:

Use Case Diagram → untuk menggambarkan interaksi antara pengguna (customer, admin, kurir) dan sistem.

Activity Diagram → untuk menunjukkan alur aktivitas dari proses pembelian, pembayaran, hingga pengiriman.

Sequence Diagram → untuk memperlihatkan urutan interaksi antar objek saat pengguna melakukan transaksi.

Class Diagram → untuk menampilkan struktur data seperti produk, pelanggan, dan pesanan.

Alasan:

Sistem e-commerce berfokus pada interaksi pengguna dan transaksi data, sehingga perlu diagram yang menekankan alur proses bisnis, relasi antar data, dan komunikasi antar objek sistem.

b. Sistem Kesehatan "MediCare"

Diagram UML yang paling penting:

Use Case Diagram → untuk memetakan peran dokter, pasien, dan sistem otomatis.

Class Diagram → untuk mendefinisikan struktur data (sensor, dosis insulin, pasien, log medis).

State Machine Diagram → untuk menggambarkan perubahan status sistem (misalnya, dari "mendeteksi kadar gula" → "menghitung dosis" → "menyuntikkan insulin").

Sequence Diagram → untuk menunjukkan interaksi antara perangkat keras (sensor, pompa insulin) dan sistem perangkat lunak.

Component Diagram → untuk menggambarkan hubungan antar modul sistem (sensor, kontroler, server data).

Alasan:

Sistem kesehatan lebih kompleks dan sensitif terhadap keamanan serta keakuratan, melibatkan sensor fisik, data medis, dan otomasi perangkat keras, sehingga memerlukan lebih banyak diagram untuk menjelaskan interaksi dan perilaku sistem secara rinci.

2. Mengapa jumlah dan jenis diagram UML yang digunakan dalam proyek berbeda? Bagaimana kompleksitas sistem memengaruhi jumlah diagram yang dibutuhkan?

Jumlah dan jenis diagram berbeda karena kompleksitas sistem, ruang lingkup, serta interaksi antar komponen tidak sama.

Sistem sederhana seperti e-commerce hanya fokus pada proses bisnis dan transaksi, sehingga cukup menggunakan beberapa diagram dasar.

Sistem kompleks seperti MediCare memerlukan koordinasi antara software, hardware, dan sensor, sehingga butuh lebih banyak diagram perilaku (behavioral) dan interaksi untuk memastikan sistem bekerja aman dan konsisten.

Semakin tinggi kompleksitas sistem, semakin banyak dan detail diagram yang diperlukan untuk meminimalkan kesalahan desain dan menjelaskan hubungan antar komponen dengan jelas.

3. Bagaimana cara menentukan jumlah dan jenis diagram UML yang paling relevan dalam suatu proyek pengembangan perangkat lunak?

Penentuan jumlah dan jenis diagram dilakukan berdasarkan:

1. Tujuan proyek dan kebutuhan pengguna (requirement analysis) — diagram dipilih sesuai aspek sistem yang ingin dijelaskan (fungsi, struktur, atau interaksi).

2. Tingkat kompleksitas sistem — semakin kompleks sistem, semakin banyak diagram yang diperlukan.

3. Tahapan pengembangan perangkat lunak —

Use Case dan Activity digunakan di tahap analisis,

Class dan Sequence digunakan di tahap desain,

Component dan Deployment digunakan di tahap implementasi.

4. Kebutuhan komunikasi tim — diagram dipilih agar mudah dipahami oleh pengembang, klien, dan pengguna.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [AJAY SUPRIADI 04001670](#) - Minggu, 9 November 2025, 16:31

Jawaban Anda cukup jelas, hanya perlu diperkuat dengan referensi dan ditutup dengan kesimpulan agar lebih akademis.

[Tautan permanen](#) [Tampilkan induk](#)



Re: Diskusi.5

oleh [054546411 AIMAAN YUSUF WICAKSONO](#) - Minggu, 9 November 2025, 19:52

1. Diagram UML Paling Penting untuk Setiap Sistem

Kita harus memilih diagram berdasarkan fokus utama sistemnya:

Sistem E-Commerce "QuickBuy" (Fokus: Belanja dan Transaksi):

- Use Case Diagram: Ini penting banget buat nunjukin fungsionalitas utama dari sudut pandang pembeli (misal: "Beli Produk", "Bayar") dan Admin (misal: "Kelola Stok").
- Activity Diagram: Dipakai untuk menjelaskan alur langkah-langkah yang detail, terutama proses checkout atau pengembalian barang, dari awal sampai selesai.
- Class Diagram: Fungsinya untuk memetakan struktur data utama, seperti bagaimana data Produk, Keranjang Belanja, dan Pelanggan saling terhubung. Ini jadi dasar bikin database.

Sistem Kesehatan "MediCare" (Fokus: Kontrol Alat dan Keselamatan):

- State Machine Diagram: Ini yang paling wajib. Karena sistem ini alat injeksi, perilakunya sangat tergantung pada status (misal: "Mode Diam", "Mode Mengukur Gula", "Mode Menyuntik"). Diagram ini memodelkan perubahan state ini.
- Component Diagram: Penting untuk menunjukkan komponen fisik (sensor, pompa, baterai) dan software yang ada di dalam alat itu, serta bagaimana mereka berinteraksi.
- Sequence Diagram: Digunakan untuk menggambarkan urutan waktu komunikasi yang sangat presisi antar komponen (misal: sensor mengirim data -> algoritma menghitung -> pompa bergerak). Karena ini menyangkut nyawa, urutan waktu harus benar.

2. Mengapa Jumlah Diagram UML Berbeda?

Perbedaan jumlah diagram ini intinya karena tingkat dan jenis kompleksitasnya berbeda:

QuickBuy lebih fokus ke interaksi pengguna dan alur bisnis. Walaupun banyak, alurnya masih berupa permintaan dan respons. Kita cukup pakai diagram inti yang memodelkan siapa melakukan apa dan datanya seperti apa.

MediCare lebih fokus ke perilaku real-time, keamanan, dan kontrol perangkat keras. Ini sistem kritis yang harus bereaksi seketika terhadap perubahan (misal: gula darah naik). Kompleksitasnya ada di state dan waktu. Oleh karena itu, kita butuh diagram tambahan seperti State Machine dan Component Diagram untuk memastikan sistem bekerja dengan aman dan benar di dunia nyata.

3. Cara Menentukan Jumlah dan Jenis Diagram

Untuk menentukan diagram mana yang perlu dibuat, kita gunakan prinsip "Cukup Saja, tapi Tepat":

1. Mulai dari yang Dasar: Selalu mulai dengan Use Case dan Class Diagram. Ini adalah fondasi.
2. Lihat Kebutuhan Proyek:
 - Perlu Detail Alur Kerja? $\rightarrow$ Buat Activity Diagram.
 - Perlu Detail Komunikasi Antar Objek? $\rightarrow$ Buat Sequence Diagram.
 - Perlu Kontrol State dan Perilaku Kritis? $\rightarrow$ Wajib buat State Machine Diagram.
 - Ada Perangkat Keras/IoT? $\rightarrow$ Wajib buat Component atau Deployment Diagram.
3. Jangan Berlebihan: Jangan buat semua 14 diagram UML kalau tidak ada yang memerlukannya. Setiap diagram harus punya tujuan jelas dan membantu komunikasi tim (misal: developer butuh Class Diagram, tester butuh Activity Diagram).

Tentu! Anggap saja ini jawaban ringkas dan mudah dipahami yang saya susun sebagai rekan mahasiswa. Saya akan langsung menjawab poin-poin utama Anda tanpa menggunakan tabel.

Jawaban Diskusi UML (Gaya Mahasiswa)

1. Diagram UML Paling Penting untuk Setiap Sistem

Kita harus memilih diagram berdasarkan fokus utama sistemnya:

Sistem E-Commerce "QuickBuy" (Fokus: Belanja dan Transaksi):

- **Use Case Diagram:** Ini penting banget buat nunjukkin fungsionalitas utama dari sudut pandang pembeli (misal: "Beli Produk", "Bayar") dan Admin (misal: "Kelola Stok").
- **Activity Diagram:** Dipakai untuk menjelaskan **alur langkah-langkah** yang detail, terutama proses *checkout* atau pengembalian barang, dari awal sampai selesai.
- **Class Diagram:** Fungsinya untuk memetakan **struktur data** utama, seperti bagaimana data *Produk*, *Keranjang Belanja*, dan *Pelanggan* saling terhubung. Ini jadi dasar bikin *database*.

Sistem Kesehatan "MediCare" (Fokus: Kontrol Alat dan Keselamatan):

- **State Machine Diagram:** Ini yang **paling wajib**. Karena sistem ini alat injeksi, perilakunya sangat tergantung pada *status* (misal: "Mode Diam", "Mode Mengukur Gula", "Mode Menyuntik"). Diagram ini memodelkan perubahan *state* ini.
- **Component Diagram:** Penting untuk menunjukkan **komponen fisik** (sensor, pompa, baterai) dan *software* yang ada di dalam alat itu, serta bagaimana mereka berinteraksi.
- **Sequence Diagram:** Digunakan untuk menggambarkan **urutan waktu komunikasi** yang sangat presisi antar komponen (misal: sensor mengirim data $\$to\$$ algoritma menghitung $\$to\$$ pompa bergerak). Karena ini menyangkut nyawa, urutan waktu harus benar.

2. Mengapa Jumlah Diagram UML Berbeda?

Perbedaan jumlah diagram ini intinya karena **tingkat dan jenis kompleksitasnya berbeda**:

- **QuickBuy** lebih fokus ke **interaksi pengguna dan alur bisnis**. Walaupun banyak, alurnya masih berupa permintaan dan respons. Kita cukup pakai diagram inti yang memodelkan *siapa melakukan apa* dan *datanya seperti apa*.
- **MediCare** lebih fokus ke **perilaku real-time, keamanan, dan kontrol perangkat keras**. Ini sistem kritis yang harus bereaksi seketika terhadap perubahan (misal: gula darah naik). Kompleksitasnya ada di *state* dan waktu. Oleh karena itu, kita butuh diagram tambahan seperti **State Machine** dan **Component Diagram** untuk memastikan sistem bekerja dengan aman dan benar di dunia nyata.

3. Cara Menentukan Jumlah dan Jenis Diagram

Untuk menentukan diagram mana yang perlu dibuat, kita gunakan prinsip **"Cukup Saja, tapi Tepat"**:

1. **Mulai dari yang Dasar:** Selalu mulai dengan Use Case dan Class Diagram. Ini adalah fondasi.
2. **Lihat Kebutuhan Proyek:**
 - **Perlu Detail Alur Kerja?** $\$to\$$ Buat Activity Diagram.
 - **Perlu Detail Komunikasi Antar Objek?** $\$to\$$ Buat Sequence Diagram.
 - **Perlu Kontrol State dan Perilaku Kritis?** $\$to\$$ Wajib buat State Machine Diagram.
 - **Ada Perangkat Keras/IoT?** $\$to\$$ Wajib buat Component atau Deployment Diagram.
3. **Jangan Berlebihan:** Jangan buat semua 14 diagram UML kalau tidak ada yang memerlukannya. Setiap diagram harus punya **tujuan jelas** dan **membantu komunikasi** tim (misal: *developer* butuh Class Diagram, *tester* butuh Activity Diagram).

Intinya, kita cuma bikin diagram yang benar-benar **bermanfaat** untuk memecahkan masalah dan meminimalisir risiko kegagalan sistem.

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**

oleh [AJAY SUPRIADI 04001670](#) - Senin, 10 November 2025, 10:21

Tulisan Anda kurang rapi terlihat dari jenis huruf yang berbeda, upayakan konsisten saat menulis jawaban, untuk memenuhi kaidah akademik, sebaiknya sertakan referensi dan kesimpulan.

[Tautan permanen](#) [Tampilkan induk](#)

**Re: Diskusi.5**

oleh [054284802 YULI ASMARAWATI](#) - Minggu, 9 November 2025, 20:37

Nama: Yuli Asmarawati

NIM: 054284802

1. Diagram UML yang paling penting untuk sistem E-Commerce "QuickBuy" adalah Structure Diagram karena diagram ini digunakan untuk menggambarkan suatu struktur statis dari sistem yang dimodelkan pada platform belanja online yang memungkinkan pengguna untuk memilih produk, menambahkan ke keranjang belanja, melakukan pembayaran, serta melacak pengiriman pesanan. Diagram yang mendukung diantaranya:

- a. Class Diagram untuk menggambarkan struktur data dan hubungan antara objek-objek dalam sistem.
- b. Object Diagram menggambarkan struktur sistem dari segi penamaan objek dan jalannya objek dalam sistem
- c. Component Diagram untuk menunjukkan organisasi dan ketergantungan di antara kumpulan komponen dalam sebuah sistem. untuk memodelkan hal-hal berikut: source code program perangkat lunak, komponen executable yang dilepas ke user, basis data secara fisik, sistem yang harus beradaptasi dengan sistem lain, framework sistem dibuat untuk memudahkan pengembangan dan pemeliharaan aplikasi, contohnya seperti prinsip desain Model-View-Controller (MVC).
- d. Deployment Diagram digunakan untuk memodelkan hal-hal berikut: Sistem tambahan (embedded system) yang menggambarkan rancangan device, node, dan hardware. Sistem client/server misalnya seperti gambar berikut. Sistem terdistribusi murni.

Diagram UML yang paling penting untuk sistem Kesehatan "MediCare" adalah Behavior Diagram karena diagram ini digunakan untuk menggambarkan perilaku sistem atau rangkaian perubahan yang terjadi pada sebuah sistem seperti pada sistem injeksi insulin otomatis untuk penderita diabetes yang menggunakan sensor untuk memantau kadar gula darah dan secara otomatis memberikan dosis insulin yang sesuai. Diagram yang mendukung diantaranya:

- a. Use Case Diagram untuk menggambarkan interaksi antara pengguna dan sistem.
- b. Activity Diagram untuk menggambarkan proses bisnis yang terjadi dalam sistem.
- c. Sequence Diagram untuk menggambarkan interaksi antara objek-objek dalam sistem

2. Jumlah diagram yang dibutuhkan berbeda antara sistem e-commerce dan sistem injeksi insulin otomatis karena dipengaruhi oleh kompleksitas sistem. Sistem e-commerce memiliki kompleksitas yang lebih rendah dibandingkan dengan sistem injeksi insulin otomatis. Sistem e-commerce hanya berfokus pada interaksi pengguna, transaksi, dan pengelolaan data produk, sehingga diagram yang dibutuhkan hanya beberapa. Sedangkan, sistem injeksi insulin otomatis memiliki kompleksitas yang lebih tinggi, sistem ini melibatkan perangkat keras, sensor, algoritma pemantauan, dan keamanan tinggi, seperti: Pengumpulan data dari sensor, analisis data untuk menentukan dosis insulin yang tepat, pengontrolan pompa insulin, pengiriman notifikasi ke pengguna. Oleh karena itu, sistem injeksi insulin otomatis mungkin memerlukan diagram yang lebih banyak.

3. Cara menentukan jumlah diagram yang diperlukan dalam proyek berbasis rekayasa perangkat lunak berorientasi objek adalah dengan mempertimbangkan beberapa faktor, seperti:

- a. Kompleksitas Sistem: Sistem yang lebih kompleks memerlukan lebih banyak diagram untuk menggambarkan struktur dan perilaku sistem.
- b. Tujuan Proyek: Diagram yang diperlukan dapat berbeda-beda tergantung pada tujuan proyek, seperti pengembangan sistem, integrasi sistem, atau pemeliharaan sistem.
- c. Metodologi Pengembangan: Metodologi pengembangan yang digunakan, seperti Agile atau Waterfall, dapat mempengaruhi jumlah diagram yang diperlukan.
- d. Ukuran Tim: Ukuran tim pengembangan dapat mempengaruhi jumlah diagram yang diperlukan, karena tim yang lebih besar memerlukan lebih banyak diagram untuk komunikasi dan koordinasi.
- e. Teknologi yang Digunakan: Teknologi yang digunakan, seperti bahasa pemrograman dan platform, dapat mempengaruhi jumlah diagram yang diperlukan.

Beberapa pedoman umum untuk menentukan jumlah diagram yang diperlukan adalah:

- a. Diagram Use Case: 1-5 diagram, tergantung pada kompleksitas sistem dan jumlah aktor.
- b. Diagram Aktivitas: 1-10 diagram, tergantung pada kompleksitas proses bisnis.
- c. Diagram Kelas: 1-20 diagram, tergantung pada kompleksitas struktur data.
- d. Diagram State Machine: 1-5 diagram, tergantung pada kompleksitas perilaku sistem.
- e. Diagram Sequence: 1-10 diagram, tergantung pada kompleksitas interaksi antara objek.

SUMBER: MODUL MSIM4303 REKAYASA PERANGKAT LUNAK

**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Senin, 10 November 2025, 10:22

Penjelasan sudah mengarah dengan baik, tapi sebaiknya tambahkan kesimpulan singkat di akhir tulisan

[Tautan permanen](#) [Tampilkan induk](#)
**Re: Diskusi.5**oleh [MUHAMMAD HAIDAR ALESSANDRO ABROR 050757557](#) - Minggu, 9 November 2025, 22:17

diagram uml yang paling penting untuk sistem e-commerce quickbuy adalah structure diagram karena menggambarkan struktur statis sistem belanja online yang memungkinkan pengguna memilih produk, menambah ke keranjang, melakukan pembayaran, dan melacak pengiriman. diagram pendukungnya antara lain: class diagram untuk menggambarkan struktur data dan hubungan antar objek; object diagram untuk memperlihatkan objek dan relasinya pada sistem; component diagram untuk menunjukkan organisasi dan ketergantungan antar komponen seperti source code, database, dan framework (misalnya mvc); serta deployment diagram untuk memodelkan perangkat keras dan sistem terdistribusi seperti client-server.

diagram uml yang paling penting untuk sistem kesehatan medicare adalah behavior diagram karena menggambarkan perilaku dan perubahan dalam sistem, misalnya pada alat injeksi insulin otomatis yang memantau kadar gula darah dan memberi dosis insulin sesuai kebutuhan. diagram pendukungnya antara lain: use case diagram untuk menggambarkan interaksi pengguna dan sistem; activity diagram untuk memodelkan proses bisnis; sequence diagram untuk menunjukkan interaksi antar objek dalam urutan waktu tertentu; state machine diagram untuk memodelkan perubahan status alat; dan component diagram untuk menjelaskan hubungan antara sensor, pompa, dan perangkat lunak.

jumlah diagram yang dibutuhkan berbeda karena tingkat kompleksitas sistem tidak sama. sistem quickbuy relatif sederhana karena berfokus pada interaksi pengguna, transaksi, dan data produk, sehingga hanya memerlukan beberapa diagram utama. sedangkan medicare jauh lebih kompleks karena melibatkan perangkat keras, sensor, algoritma pemantauan, dan keamanan tinggi, sehingga membutuhkan lebih banyak diagram untuk memodelkan perilaku dan integrasi komponennya.

cara menentukan jumlah dan jenis diagram uml dilakukan dengan mempertimbangkan beberapa faktor seperti kompleksitas sistem, tujuan proyek, metodologi pengembangan, ukuran tim, dan teknologi yang digunakan. prinsipnya adalah cukup tetapi tepat: mulai dengan use case dan class diagram sebagai dasar, tambahkan activity atau sequence diagram bila perlu detail alur kerja atau komunikasi antar objek, gunakan state machine untuk sistem dengan perilaku dinamis, dan tambahkan component atau deployment diagram bila melibatkan perangkat keras atau integrasi sistem. setiap diagram harus memiliki tujuan jelas dan mendukung komunikasi dalam pengembangan perangkat lunak.

sumber :SUMBER: MODUL MSIM4303 REKAYASA PERANGKAT LUNAK

[Tautan permanen](#) [Tampilkan induk](#)
**Re: Diskusi.5**oleh [AJAY SUPRIADI 04001670](#) - Senin, 10 November 2025, 10:23

Jawaban Anda cukup jelas, hanya perlu diperkuat dengan ditutup dengan kesimpulan agar lebih akademis.

[Tautan permanen](#) [Tampilkan induk](#)
[◀ Materi Inisiasi](#)

[Tugas.2 ▶](#)
Navigasi

▼ [Dasbor](#) [Beranda situs](#)> [Laman situs](#)▼ [Kelasku](#)> [STSI4203.108](#)▼ [STSI4202.42](#)> [Peserta](#) [Nilai](#)> [Pendahuluan](#)> [Sesi 1](#)> [Sesi 2](#)> [Sesi 3](#)> [Sesi 4](#)▼ [Sesi 5](#) [Kehadiran Sesi ke-5](#) [Materi Inisiasi](#) [Diskusi.5](#) [Tugas.2](#)> [Sesi 6](#)> [STSI4103.119](#)> [MKKI4201.278](#)> [STSI4201.161](#)> [STSI4205.331](#)> [STSI4104.284](#)> [MKDI4202.1514](#)> [Kelas](#) [Administrasi](#)▼ [Forum administrasi](#)

Berlangganan dinonaktifkan

Follow Us:      

UNIVERSITAS TERBUKA ©2025

Anda masuk sebagai [INDRAWAN LISANTO 053724113](#) ([Keluar](#))[Dapatkan aplikasi seluler](#)